

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное
образовательное учреждение высшего образования
«Череповецкий государственный университет»

Институт (факультет)	информационных технологий
Направление подготовки (специальность)	09.04.04 Программная инженерия (Искусственный интеллект)
Выпускающая кафедра	Математическое и программное обеспечение ЭВМ

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Название работы	Программное обеспечение прогнозирования прочностных характеристик кокса по его гранулометрическому составу
-----------------	--

Студента	Богданова Александра Павловича Ф.И.О.
----------	--

ДОПУСТИТЬ К ЗАЩИТЕ

Директор института (декан факультета)	(Ершов Е.В.)
Заведующий выпускающей кафедрой	(Ершов Е.В.)
Руководитель выпускной квалификационной работы	(Ершов Е.В.)
Консультант по технико-экономическому обоснованию разработки	(Виноградова Л.Н.)
Нормоконтролер	(Виноградова Л.Н.)
Выпускник	(Богданов А.П.)

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное
образовательное учреждение высшего образования
«Череповецкий государственный университет»

АННОТАЦИЯ

выпускной квалификационной работы студента

по теме:

Программное обеспечение прогнозирования прочностных характеристик кокса по его
гранулометрическому составу

Студент

Богданов Александр Павлович

фамилия, имя, отчество

подпись

Руководитель работы

ФГБОУ ВО «Череповецкий государственный университет», кафедра МПО ЭВМ

место работы

д.т.н., профессор

должность

Ершов Евгений Валентинович

фамилия, имя, отчество

(Перечислить основные вопросы, которые рассматривались; результаты работы)

Диссертационная работа посвящена разработке математического, алгоритмического и программного обеспечения для прогнозирования прочностных характеристик кокса по его гранулометрическому составу. В работе проведен анализ существующих методов и средств прогнозирования, выявлены недостатки и ограничения, сформулированы требования к обеспечению. Разработана математическая модель, адаптированы и протестированы алгоритмы прогнозирования. Выполнено концептуальное моделирование и проектирование информационного и программного обеспечения системы. Проведена экспериментальная проверка предложенных решений, разработана методика настройки программного обеспечения. Результаты экспериментов подтвердили работоспособность и эффективность разработанной системы. Практическая ценность работы заключается в создании программного комплекса для прогнозирования прочностных характеристик кокса, востребованного в металлургической промышленности.

Введение	5
1 Характеристика проблемы прогнозирования прочностных характеристик кокса по его гранулометрическому составу	8
1.1 Анализ существующих моделей, методов и средств прогнозирования прочностных характеристик кокса по его гранулометрическому составу	8
1.2 Описание прочностных характеристик кокса и гранулометрического состава как объекта прогнозирования	13
1.3 Постановка задачи, разработка требований к информационному и аппаратно-программному обеспечению прогнозирования прочностных характеристик кокса по его гранулометрическому составу	17
1.4 Выводы по разделу 1	20
2 Математическое и алгоритмическое обеспечение прогнозирования прочностных характеристик кокса по его гранулометрическому составу	21
2.1 Разработка математической модели прогнозирования прочностных характеристик кокса по его гранулометрическому составу	21
2.2 Адаптация и тестирование математической модели прогнозирования прочностных характеристик кокса по его гранулометрическому составу	33
2.3 Описание результатов моделирования системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу	45
2.4 Разработка метода и алгоритмов прогнозирования прочностных характеристик кокса по его гранулометрическому составу	59
2.5 Выводы по разделу 2	63
3 Проектирование системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу	64

3.1 Концептуальное моделирование системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу	64
3.1.1 Модель «черного ящика»	64
3.1.2 Модель состава	65
3.1.3 Модель структуры	66
3.1.4 Модель «белого ящика»	67
3.2 Проектирование информационного обеспечения прогнозирования прочностных характеристик кокса по его гранулометрическому составу	68
3.2.1 Моделирование потоков информации в проектируемой системе	68
3.3 Проектирование программного обеспечения прогнозирования прочностных характеристик кокса по его гранулометрическому составу	69
3.3.1 Выбор технологии проектирования, среды и языка программирования	69
3.3.1.1 Выбор модели жизненного цикла	69
3.3.1.2 Выбор подхода к разработке	71
3.3.1.3 Выбор среды и языка программирования	72
3.3.2 Разработка логической модели	73
3.3.2.1 Диаграмма вариантов использования	73
3.3.2.2 Контекстная диаграмма классов	78
3.3.2.3 Диаграмма пакетов	80
3.3.2.3.1 Детальная диаграмма классов пакета «AI»	84
3.3.3 Разработка физической модели	85
3.3.3.1 Диаграмма компонентов	85
3.3.3.2 Диаграмма размещения	87
3.3.4 Разработка интерфейсов	88
3.3.4.1 Построение графа диалога	88

3.3.4.2 Разработка форм ввода вывода информации	89
3.4 Выводы по разделу 3	92
4 Экспериментальная проверка предложенных технических решений прогнозирования прочностных характеристик кокса по его гранулометрическому составу	93
4.1 Описание эксперимента и экспериментальных данных	93
4.2 Разработка методики настройки программного обеспечения	97
4.3 Результаты экспериментальной проверки модели определения гранулометрического состава	97
4.4 Результаты экспериментальной проверки модели прогнозирования прочностных характеристик	100
4.5 Тестирование программного обеспечения	105
4.6 Перспективы применения	111
4.7 Выводы по разделу 4	112
Заключение	114
Список литературы	116
Приложение 1. Устав проекта	123
Приложение 2. Текст программы	140
Приложение 3. Спецификации	154
Приложение 4. Руководство пользователя	157

Введение

Актуальность темы.

Определение гранулометрического состава кокса и расчёт его прочностных характеристик является ручным процессом, результаты которого непригодны для дальнейшего анализа. С развитием технологий машинного зрения и обучения задача распознавания гранулометрического состава кокса и прогнозирования его прочностных характеристик становится более реальной и актуальной.

Определение гранулометрического состава на линии конвейера является актуальной темой для разработки приложений, которые появляются в производственных областях, начиная от обнаружения и создания систем отчетности и заканчивая комплексной системой анализа.

Системы обнаружения гранулометрического состава, как и классические устройства, предназначенные для этой задачи, набирают свою популярность все в больших сферах, однако данные технологии и алгоритмы, применяемые в них, имеют ряд проблем и недостатков.

На данный момент в компании ПАО «Северсталь» на конвейерах КД-4 и К-19 имеется проблема с частотой расчёта прочностных характеристик кокса и определения его гранулометрического состава. Он производится два раза в день ручным механическим способом и при дальнейшем обработке существует вероятность отправки брака в сырьё.

Для устранения данной проблемы требуется разработать программное обеспечение прогнозирования прочностных характеристик кокса по его гранулометрическому составу.

Цель исследования.

Целью исследования является повышение уровня автоматизации контроля и сокращение времени анализа доменного кокса.

Для достижения поставленных целей решены следующие задачи:

- анализ существующих моделей, методов и средств определения гранулометрического состава и прогнозирования его прочностных характеристик;
- разработка математической модели прогнозирования прочностных характеристик кокса по его гранулометрическому составу;
- проектирование и реализация системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу;
- экспериментальная проверка разработанных решений.

Объект исследования

Объектом исследования является система прогнозирования прочностных характеристик кокса по его гранулометрическому составу.

Предмет исследования.

Предметом исследования являются методы и алгоритмы обнаружения гранулометрического состава кокса на изображениях, и методы и алгоритма прогнозирования прочностных характеристик кокса.

Методы исследования.

Для реализации поставленной задачи применялись методы математического моделирования, машинного обучения, регрессионного анализа, а также алгоритмы компьютерной обработки изображений.

Научная новизна.

Научная новизна работы заключается:

1. В создании метода прогнозирования прочностных характеристик кокса, отличающегося от имеющихся отсутствием дополнительных механических действий и позволяющего рассчитать прочностные характеристики в любой момент за короткое время;
2. В разработке алгоритма определения гранулометрического состава с помощью сегментационных нейронных сетей, отличающегося от имеющихся решений высокой точностью и адаптивностью и позволяющего определить гранулометрический состав с высокой степенью детализации и точности.

Практическая значимость

Практическая значимость состоит в обнаружении гранулометрического состава и накопления базы данных по обнаруженному размеру кокса, на основе которой будет производиться расчёт его прочностных характеристик.

Апробация результатов.

Результаты магистерской диссертационной работы обсуждены и получили положительные оценки на:

- Всероссийская научно-практическая конференция «Региональная наука: тенденции развития глазами молодежи» (Череповец, 2024 гг.);
- Студенческая научная конференция «Регион искусственного интеллекта» (Череповец, 2023 г.);
- VI Всероссийская научно-практическая конференция «Современные информационные технологии. Теория и практика» (Череповец, 2023 г.);
- XXI Международная научно-практическая конференция студентов, аспирантов и молодых ученых «Молодежь и современные информационные технологии» (Томск, 2024 г.).

Структура и объем диссертации.

Диссертационная работа состоит из введения, четырех разделов, заключения, списка литературы, включающего 57 наименований, и 4-х приложений. Объем диссертационной работы – 161 страница. В тексте диссертации содержатся 69 рисунков и 39 таблиц.

1 Характеристика проблемы прогнозирования прочностных характеристик кокса по его гранулометрическому составу

1.1 Анализ существующих моделей, методов и средств прогнозирования прочностных характеристик кокса по его гранулометрическому составу

На текущий момент на предприятии ПАО «Северсталь» в цехе доменной печи №4 имеется ряд следующих проблем:

- трата денежных на обслуживание механического устройства и риск выхода из строя;
- долгая длительность обработки данных и низкая периодичность ее обновления;
- наличие человеческого фактора при занесении данных о гранулометрическом составе для последующего визуального анализа;
- отсутствие системы автоматизированной системы анализа гранулометрического состава.

Другими словами, отсутствие автоматизированной системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу не позволяет в точности зафиксировать неоднородность сырья в определенных промежутках и откладывает задачу анализа на человека.

Ниже представлена информация о ряде систем, связанных с анализом и определением гранулометрического состава кокса.

1. ВИСКОНТ.Гранулы (ООО «Малленом Системс») (рис. 1) - интеллектуальная система контроля крупности гранулометрического состава сыпучих материалов на конвейерной ленте разработана на основе технологии сверточных нейронных сетей, что позволяет достигать высокой точности определения гранулометрического состава до 97%.

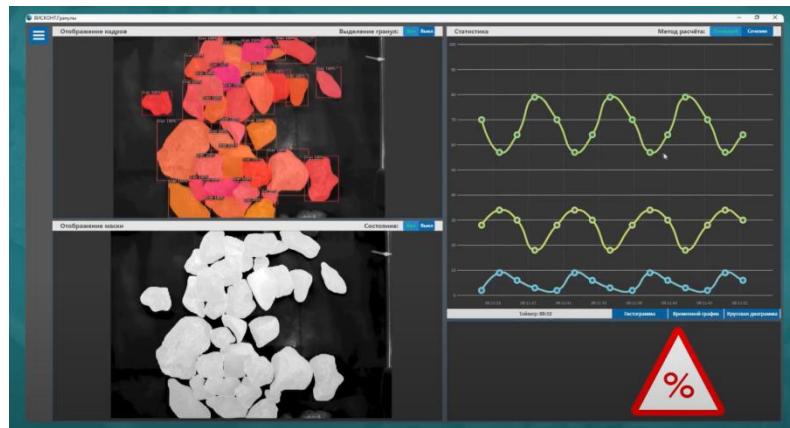


Рис. 1. ВИСКОНТ.Гранулы

Недостатки:

- закрытое программное обеспечение (ПО);
- отсутствие прогнозирования прочностных характеристик.

2. Система определения фракционного состава кокса (ООО «Малленом Системс») (рис. 2) - над транспортной лентой устанавливается телекамера для контроля фракционного состава кокса и выявления негабаритных объектов, видеопоток с телекамеры поступает для обработки на компьютер, установленный в закрытом помещении или в защитном шкафу, программное обеспечение, реализующее алгоритмы анализа изображений, передаёт результаты анализа в информационную систему Заказчика.

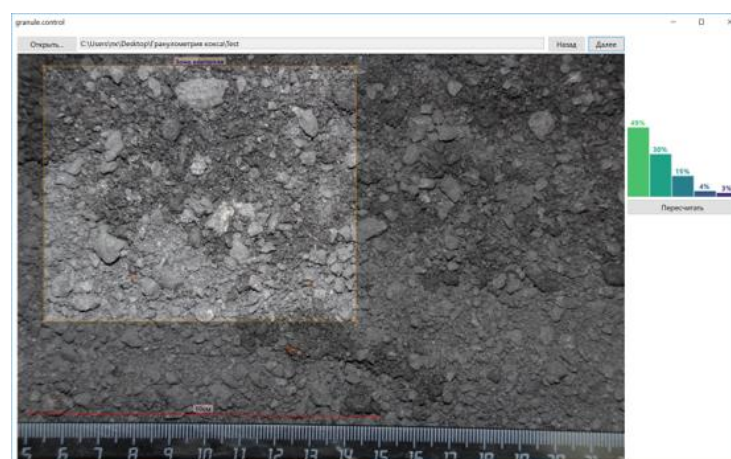


Рис. 2. Система определения фракционного состава кокса

Недостатки:

- закрытое ПО;
- отсутствие подсистемы анализа;
- отсутствие прогнозирования прочностных характеристик.

3. Комплекс «MachineVision» («НПО «РИВС») (рис. 3) - предназначен для контроля свойств пены, позволяет в режиме реального времени использовать визуальные характеристики пены, дополняя показания датчиков автоматизированной системы управления флотацией.

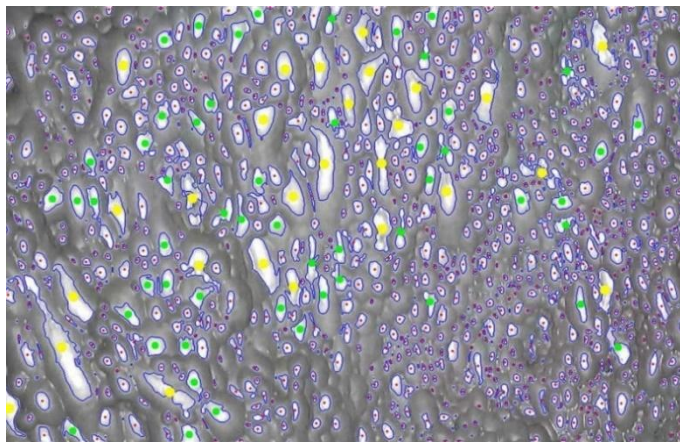


Рис. 3. Комплекс «MachineVision»

Недостатки:

- отсутствие подсистемы анализа;
- отсутствие прогнозирования прочностных характеристик.

4. VITес – система технического зрения для измерения объема и оценки гранулометрического состава сыпучих материалов (щебень, руда и др.) на ленте конвейера (рис. 4). Непрерывное измерение объема материала и оценка гранулометрического состава осуществляются на основании сканирования трехмерного профиля слоя на ленте конвейера синхронно со скоростью движения материала; результаты измерений (средневзвешенный класс крупности, выход объема за последнюю минуту, час и т. п.) могут быть представлены в графическом виде оператору, выведены на большое цифровое табло, а также переданы в информационную систему предприятия

в виде отчетов. Полученные в результате сканирования и расчетов данные сравниваются с заданными параметрами работы. В случае выхода за допуски или обнаружения на ленте крупных объектов система может включить предупредительный сигнал. Возможна работа в автономном режиме без непосредственного участия оператора.



Рис. 4. Система измерения объема и оценки гранулометрического состава VITec

Недостатки:

- закрытое ПО;
- отсутствие прогнозирования прочностных характеристик.

Таким образом, после выполненного анализа было выявлено, что продуктов, которые бы позволили спрогнозировать прочностные характеристики кокса, нет. К тому же практически все системы либо дорогие ввиду дополнительного оборудования, либо не учитывают все особенности технологического процесса и являются экспериментальными.

Для решения задачи прогнозирования прочностных характеристик могут быть использованы следующие методы:

1. Метод k-ближайших соседей (k-NN). Этот метод использует ближайших соседей для прогнозирования значений целевой переменной. Он основан на предположении, что объекты с похожими признаками имеют

схожие значения целевой переменной [5]. Для реализации этого метода необходимо хранить все объекты в памяти или в дисковом пространстве;

2. Регрессионный анализ. Этот метод используется для прогнозирования непрерывных значений целевой переменной. Он основан на анализе зависимостей между признаками и целевой переменной [5]. Регрессионный анализ может быть линейным или нелинейным;

3. Метод опорных векторов (SVM). Этот метод используется для классификации объектов на основе их признаков. Он основан на построении гиперплоскости, которая разделяет объекты разных классов [5]. SVM может быть линейным или нелинейным;

4. Градиентный бустинг. Данный метод используется для решения задач классификации и регрессии, который строит модель предсказания в форме ансамбля слабых предсказывающих моделей, обычно деревьев решений;

5. Нейронная сеть. Этот метод используется для прогнозирования значений целевой переменной на основе признаков.

Для того, чтобы произвести прогнозирование прочностных характеристик кокса, будут применены методы регрессионного анализа, градиентного бустинга и построения нейронных сетей, чтобы более точно выявить связь между прочностными характеристиками и гранулометрическим составом кокса.

Данные методы были выбраны, потому что они более оптимизированы и точны для решения задач поиска зависимости между числовыми значениями, в отличии от k-NN и SVM.

Традиционно задачи компьютерного зрения разделяют на четыре вида [6]:

1. Classification — классификация изображения по типу объекта, которое оно содержит;

2. Object detection — обнаружение всех объектов указанных классов и определение охватывающей рамки для каждого из них;

3. Semantic segmentation — определение всех пикселей объектов определённого класса или фона на изображении. Если несколько объектов одного класса перекрываются, их пиксели никак не отделяются друг от друга;

4. Instance segmentation — определение пикселей, принадлежащих каждому объекту каждого класса по отдельности.

В качестве выбранной технологии будут рассмотрены нейронные сети с типом Instance segmentation, потому что он предполагает получение контура отдельного объекта, что позволит определить размер гранулы кокса.

1.2 Описание прочностных характеристик кокса и гранулометрического состава как объекта прогнозирования

Кокс в доменной печи определяет газодинамическое сопротивление слоя шихты движущемуся потоку газа и должен быть крупным, а также сохранять размер кусков по мере его движения от колошника к фурменным очагам [2].

Гранулометрический состав кокса — это распределение его кусков по классам крупности, выраженное в процентах к массе исходной пробы.

ГОСТ 9434-75 [3] предусматривает следующую градацию кусков кокса по крупности: 25-, 25-40 мм, 40-60 мм, 60-80 мм и 80+ мм. Следует отметить, что фракция 80+ нежелательна. Увеличение фракции 80+ на 1 % приводит к увеличению удельного расхода кокса на 1,2 %. Фракция 25- отсеивается и не используется в доменных печах.

Кокс и железорудные материалы в доменную печь загружают чередующимися слоями. Структура слоя кокса способствует распределению газа по сечению и проникновению его в слои железорудных материалов.

Наиболее важными функциями кокса являются [2]:

- генерация в процессе горения на фурмах тепла и восстановленного газа, необходимого для восстановления и плавления железорудных материалов;

- создание в печи газопроницаемой структуры, обеспечивающей необходимое распределение газа по сечению печи и прохождение его через железорудные материалы;

- создание в нижней части печи прочной структуры (коксовой насадки), проницаемой для жидких продуктов плавки;

- кокс остается твердым и пористым материалом при его нагреве до высоких температур (более 2000 градусов Цельсия), что имеет особое значение в горне и зоне когезии. Под зоной плавления кокс остается единственным твердым материалом. Столб всей шихты, заполняющий доменную печь, поддерживается слоем кокса, заполняющего нижнюю часть печи - коксовой насадкой.

Качество кокса можно описать двумя широкими категориями:

- состав;
- механическая холодная и горячая прочность.

К физическим характеристикам качества кокса относят [2]:

- размеры, гранулометрический состав. Средний размер металлургического кокса обычно составляет от 45 до 55 миллиметров. Для обеспечения высокой проницаемости слоя кокса гранулометрический состав его должен быть узким. Повышенное содержание в коксе фракции более 80 мм указывает на плохое регулирования процесса коксования;

- сопротивление физическому разрушению при транспортировке и других механических воздействиях. Показателями, которые оценивают эту характеристику, являются - I40, M40, "Стабильность". Данные показатели указывают на гранулометрический состав кокса после его химической стабилизации;

- стойкость к истиранию, которая характеризуется показателями - I10, M10, "Твердость";

- реакционная способность кокса. Кокс может реагировать с CO₂, что приводит к его разупрочнению и увеличивает его расход в доменной печи.

Кокс с меньшей реакционной способностью (показатели реакционной способности кокса - CRI) и высокой горячей прочностью (показатель горячей прочности - CSR) имеет более высокую механическую прочность в нижней части печи.

Постоянство качественных характеристик кокса, особенно его размера и гранулометрического состава, оказывает большое влияние на работу доменной печи.

Прочность кокса — это важнейшая характеристика, определяющая в первую очередь газодинамические характеристики столба шихтовых материалов в доменной печи [2]. Появление мелочи при разрушении непрочного кокса не только ухудшает газопроницаемость шихты в печи, но и уменьшает размеры фурменного очага, а значит и газораспределение в поперечном сечении доменной печи. Под прочностью кокса понимают его способность противостоять разрушающим воздействиям всех видов. Пример изображения кокса показан на рис. 5.

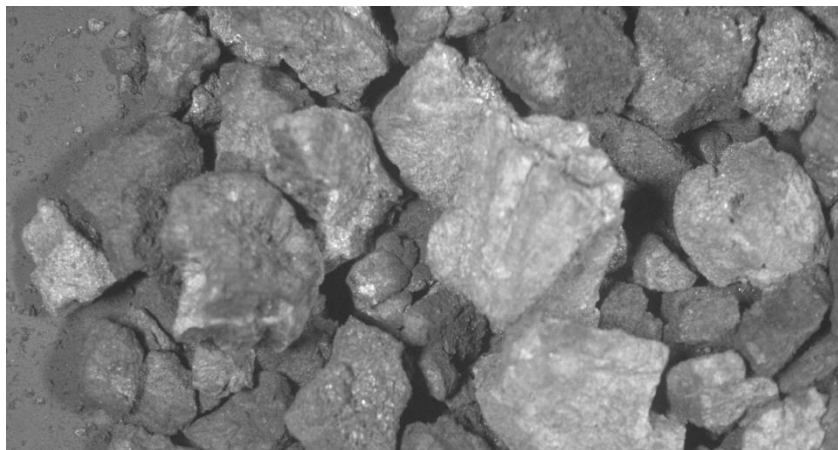


Рис. 5. Доменный кокс

В настоящее время контролировать изменение качества кокса, подаваемого в ДП № 4, в режиме реального времени практически невозможно. Корректировка рудной нагрузки происходит с запаздыванием, при этом зачастую имеют место значительные отклонения по качеству чугуна.

Применение системы для определения гранулометрического состава подаваемого кокса в реальном времени с помощью методов и алгоритмов технического зрения и искусственного интеллекта позволит решить данную проблему на конвейерах КД-4 и К-19 ДП № 4. Схема конвейера представлена на рис. 6, а его изображение на рис. 7.

Информация о гранулометрическом составе кокса должна передаваться на рабочие места газовщиков доменной печи.



Рис. 6. Схема конвейера



Рис. 7. Конвейер доменной печи

В рамках исследования будет рассмотрено движение кокса по следующему промежутку (рис. 8):

1. Коксо-сортировочная машина;
2. Конвейер К-19;
3. Перегрузочное устройство;
4. Конвейер К-20;
5. Перегрузочное устройство;
6. Конвейер К-22;
7. Перегрузочное устройство;
8. Конвейер КД-4.

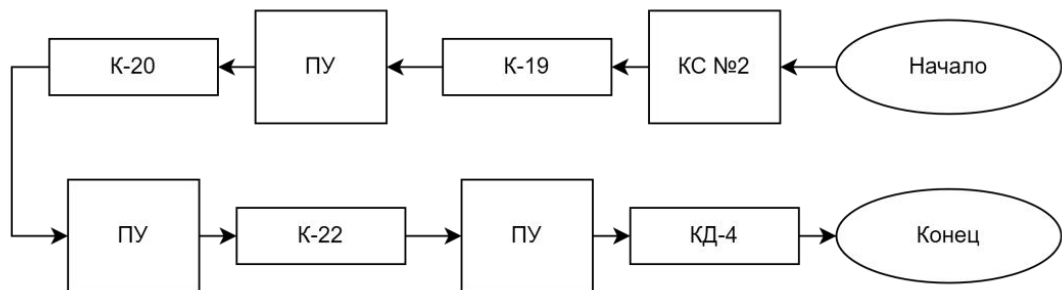


Рис. 8. Схема движения кокса

Среднее время прохождения кокса по данным технологическим узлам - 11 минут 30 секунд.

1.3 Постановка задачи, разработка требований к информационному и аппаратно-программному обеспечению прогнозирования прочностных характеристик кокса по его гранулометрическому составу

Показателями положительной работы системы являются правильный расчет прогноза прочностных характеристик кокса, определение гранулометрического состава кокса.

При достижении данных показателей может возникнуть ряд проблем, с которыми система может столкнуться:

- сбой при получении данных для расчета модели;
- низкая точность определения гранулометрического состава кокса и прочностных характеристик.

Для решения представленной задачи нужно выполнить следующие пункты:

- сбор набора изображений кокса;
- сбор набора данных прочностных характеристик;
- создание алгоритма определения гранулометрического состава;
- создание метода прогнозирования прочностных характеристик;
- создание модуля визуализации результатов анализа;
- адаптация и тестирование программного обеспечения.

Для реализации проекта были выбраны технологии, описание которых представлено в табл. 1.

Таблица 1

Технологии и инструментальные средства реализации проекта

Именованние	Назначение
Draw.IO	Средство создание диаграмм для проектирования и описания алгоритмов
Rational rose	Средство проектирования и разработки информационных систем
Visual Studio 2022	Среда разработки
Visual Studio Code	Текстовый редактор
Jira	Ведение задач и документации проекта
Microsoft Excel	Средство для ведения отчётности

Более подробный список использованных технологий и инструментальных средств расположен в п.3.3.1.

Постановка задачи – это процесс формулировки назначения программного обеспечения и основных требований к нему. Описываются функциональные требования, которые должно выполнять программное обеспечение, и эксплуатационные требования, определяющие характеристики его функционирования. Этап постановки задачи заканчивается разработкой технического задания с принятием основных проектных решений [7].

Необходимо создать программное обеспечение, реализующее новый метод прогнозирования прочностных характеристик кокса и алгоритм определения гранулометрического состава кокса, чтобы суммарное затраченное время на анализ кокса было минимальным, а степень автоматизации - максимальна. Максимально эффективно это можно сделать с помощью методов и алгоритмов компьютерного зрения и регрессионного анализа в отличие от ручного использования гранулометра.

Суть метода состоит в нахождении зависимости между разностью определенного гранулометрического состава кокса в конечных точках системы конвейеров и его прочностными характеристиками. Ввиду того, что ручной, классический метод расчёта прочностных характеристик подразумевает под собой механическое воздействие на кокс, то данный метод использует естественную разрушаемость кокса при его движении по технологическим узлам.

К математическому обеспечению предъявляются следующие требования:

- периодичность определения гранулометрического состава не реже одного раза в минуту;
- диапазон размеров кусков 0-100 мм;
- количество классов крупности кусков – 6;
- диапазоны классов крупности (мм): 0-10, 10-25, 25-40, 40-60, 60-80, 80-100;
- минимальная ширина класса крупности 10 мм;

- погрешность измерения линейных размеров не более 5 мм;
- погрешность определения доли контролируемого класса не более 10%;
- погрешность определения показателей М40 (стабильность) и М10 (твёрдость) не более 18 %.

Для того, чтобы решить задачу прогнозирования прочностных характеристик на основе разницы гранулометрического состава кокса в головной и хвостовой частях конвейера были выбраны следующие модели машинного обучения:

- полиномиальная регрессия;
- модели градиентного бустинга (CatBoost, XgbBoost);
- модели нейронных сетей;
- ансамбль моделей.

Для решения задачи определения гранулометрического состава будут применяться методы и алгоритмы компьютерного зрения. Задача распознавания объекта на изображении и получение его контура может быть решена с помощью архитектуры нейронной сети Mask R-CNN. В частности, сеть с данной архитектурой может выделять на изображении контуры («маски») различных объектов, даже если таких объектов несколько, и они имеют различный размер и частично перекрываются [1].

1.4 Выводы по разделу 1

В данном разделе представлены основные выводы по анализу предметной области программного обеспечения прогнозирования прочностных характеристик кокса по его гранулометрическому составу. Были проанализированы существующие системы в данной области и выделены главные преимущества разрабатываемого программного обеспечения. Также в текущем разделе была разобрана предметная область, сформированы задачи и требования к программному обеспечению, выполнен подбор математического аппарата.

2 Математическое и алгоритмическое обеспечение прогнозирования прочностных характеристик кокса по его гранулометрическому составу

2.1 Разработка математической модели прогнозирования прочностных характеристик кокса по его гранулометрическому составу

Для создания модели прогнозирования прочностных характеристик независимыми переменными выступает гранулометрический состав кокса:

$$C_{0_10}, C_{10_25}, C_{25_40}, C_{40_60}, C_{60_80}, C_{80_100}, \quad (1)$$

где

C_{0_10} – содержание фракции 0-10 мм;

C_{10_25} – содержание фракции 10-25 мм;

C_{25_40} – содержание фракции 25-40 мм;

C_{40_60} – содержание фракции 40-60 мм;

C_{60_80} – содержание фракции 60-80 мм;

C_{80_100} – содержание фракции более 80 мм.

Соответственно, разница между значениями данной фракции между головной и хвостовой частями конвейера будет записана следующим образом:

$$\Delta C_{0_10}, \Delta C_{10_25}, \Delta C_{25_40}, \Delta C_{40_60}, \Delta C_{60_80}, \Delta C_{80_100}, \quad (2)$$

где

ΔC_{0_10} – разница фракции 0-10 мм;

ΔC_{10_25} – разница фракции 10-25 мм;

ΔC_{25_40} – разница фракции 25-40 мм;

ΔC_{40_60} – разница фракции 40-60 мм;

ΔC_{60_80} – разница фракции 60-80 мм;

ΔC_{80_100} – разница фракции более 80 мм.

Ключевыми прочностными характеристиками кокса являются следующие показатели:

M10 - стойкость к истиранию (твердость);

M40 - сопротивление физическому разрушению при транспортировке и других механических воздействиях (стойкость).

Для того, чтобы спрогнозировать прочностные характеристики, будут использоваться следующие методы:

- метод линейной регрессии;
- метод градиентного бустинга;
- метод построения нейронной сети;
- метод построения ансамбля моделей машинного обучения.

Метод линейной регрессии — это статистический метод для анализа связи между переменными, когда одна переменная (независимая) влияет на другую переменную (зависимую) линейно [8]. Метод использует модель, которая представляет зависимую переменную как функцию от независимых переменных, где коэффициенты этой функции определяются через анализ данных.

Полиномиальная регрессия - форма регрессионного анализа, в которой связь между независимой переменной x и зависимой переменной y моделируется как полином n -й степени в x .

Соответственно формулы для определения твердости и стойкости кокса с помощью линейной регрессии с единичной степенью можно представить следующим образом:

$$P = c + x1 * \Delta c_{0-10} + x2 * \Delta c_{10-25} + x3 * \Delta c_{25-40} + x4 * \Delta c_{40-60} + x5 * \Delta c_{60-80} + x6 * \Delta c_{80-100}, \quad (3)$$

где

Δc_i – разница между фракцией в начале и в конце конвейера;

x_1 - x_6 – коэффициенты регрессии;

c – базовое значение;

P – параметр прочности кокса (M10 или M40).

Градиентный бустинг – это алгоритм, объединяющий несколько «слабых» обучающих моделей в «сильные», в котором каждая новая модель обучается минимизировать функцию потерь, такую как средняя квадратичная ошибка или кросс-энтропия предыдущей модели, используя градиентный спуск.

На каждой итерации алгоритм вычисляет градиент функции потерь относительно предсказаний текущего ансамбля (рис. 9), а затем обучает новую слабую модель, чтобы минимизировать этот градиент. Предсказания новой модели добавляются в ансамбль, и процесс повторяется до тех пор, пока не будет достигнут критерий остановки.

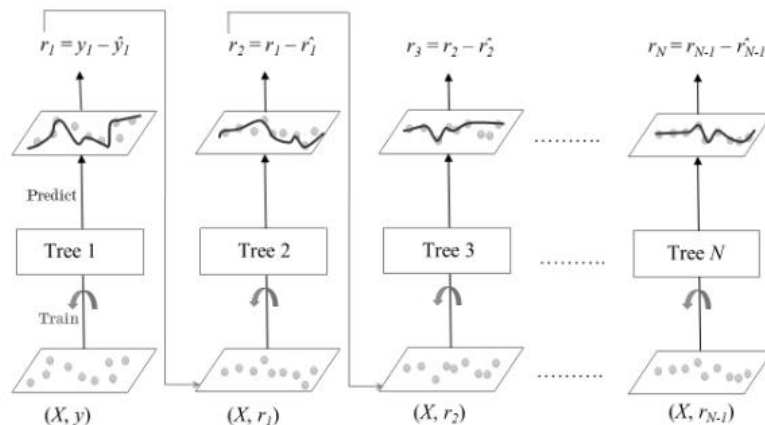


Рис. 9. Принцип работы градиентного бустинга

Ниже представлена формула обновления функции принятия решения в градиентном бустинге, которая описывает то, что на каждом шаге алгоритм ищет новую модель h_m , которая минимизирует суммарную функцию потерь по всем обучающим примерам. После этого эта модель добавляется к предыдущей модели для улучшения предсказания.

$$f_m(x) = f_{m-1}(x) + (\operatorname{argmin}[\sum_{i=1}^N L(y_i, f_{m-1}(x_i) + h_m(x_i))])(x), \quad (4)$$

где

$f_m(x)$ – функция, которую необходимо предсказать на шаге m для входного значения x ;

$f_{m-1}(x)$ – предсказание функции на предыдущем шаге;

$h_m(x_i)$ – новая модель, которая добавляется к предыдущей модели для улучшения предсказания;

L – функция потерь, которая измеряет разницу между предсказанными значениями и истинными значениями;

x_i, y_i – обучающие примеры.

Нейронная сеть представляет из себя совокупность нейронов, соединенных друг с другом определенным образом.

При построение нейронной сети стоит учитывать, что состояние нейрона вычисляется по следующей формуле:

$$S = \sum_{i=1}^n x_i w_i, \quad (5)$$

где

S – конечное состояние нейрона;

n – число входов нейрона;

x_i – значение входа нейрона;

w_i – вес связи нейрона.

Для обучения всех моделей предлагается использовать составную функцию ошибки с помощью применения гармонического среднего:

$$\bar{e} = \frac{n}{\sum \frac{1}{e}}, \quad (6)$$

где

$\bar{e}$ – среднее гармоническое ошибки;

n – число различных ошибок;

e – набор ошибок.

В качестве набора ошибок предлагается использовать RMSE (корень из среднеквадратичной ошибки), MAE(средняя абсолютная ошибка) и Huberloss (ошибка Хьюбера).

RMSE вычисляется следующим образом:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}, \quad (7)$$

где

n – количество наблюдений, по которым строится модель и количество прогнозов;

y_i – фактические значение зависимой переменной для i -го наблюдения;

$\hat{y}_i$ – значение зависимой переменной, предсказанное моделью.

MAE вычисляется следующим образом:

$$\text{MAE} = \frac{\sum |y_i - \hat{y}_i|}{n}, \quad (8)$$

где

n – количество наблюдений, по которым строится модель и количество прогнозов;

y_i – фактические значение зависимой переменной для i -го наблюдения;

$\hat{y}_i$ – значение зависимой переменной, предсказанное моделью.

Huberloss вычисляется следующим образом:

$$\text{Huberloss} = \begin{cases} \frac{1}{2} \sum (y_i - \hat{y}_i)^2, & \text{если } |y_i - \hat{y}_i| \leq \delta \\ \delta \sum |y_i - \hat{y}_i| - \frac{1}{2} \delta^2, & \text{в др. случаях} \end{cases}, \quad (9)$$

где

n – количество наблюдений, по которым строится модель и количество прогнозов;

δ – значение перехода от линейной ошибки к квадратичной;

y_i – фактические значение зависимой переменной для i -го наблюдения;

$\hat{y}_i$ – значение зависимой переменной, предсказанное моделью.

Конечные уравнения и результаты создания моделей машинного обучения расположены в п.2.3.

Для того, чтобы определить гранулометрический состав кокса, необходимо использовать методы компьютерного и машинного зрения. Так как выбранной архитектурой нейронной сети является Mask R-CNN, то следует описать принцип работы сверточной нейронной сети.

Сверточный (convolutional) слой позволяет объединять значения расположенных рядом пикселей и выделять более обобщённые признаки изображения. Для этого по изображению последовательно «скользят» квадратным окном небольшого размера (3x3, 5x5, 7x7 пикселей и т.п.) называемым ядром (kernel). Каждый элемент ядра имеет свой весовой коэффициент, умножаемый на значение яркости того пикселя изображения, на который в данный момент наложен элемент ядра. Затем полученные для всего окна значения складываются, и эта взвешенная сумма даёт величину очередного признака. Для получения матрицы («карты») признаков всего изображения, ядро последовательно сдвигается по горизонтали и вертикали.

В следующих слоях операция свертки применяется уже к картам признаков, полученным из предыдущих слоёв.

Графически процесс проиллюстрирован на рис. 10.

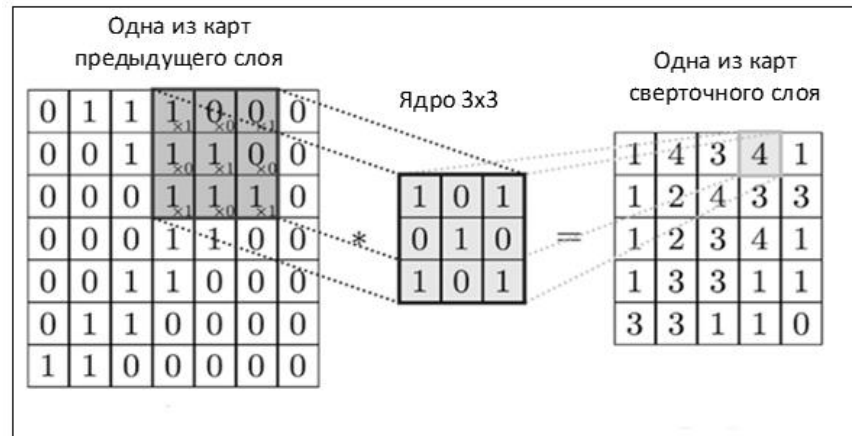


Рис. 10. Процесс свёртки

Изображения или карты признаков в рамках одного слоя могут сканироваться не одним, а несколькими независимыми фильтрами, давая таким образом на выход не одну карту, а несколько (их ещё называют «каналами»). Настройка весов каждого фильтра происходит при помощи процедуры backpropagation.

Очевидно, если ядро фильтра при сканировании не выходит за пределы изображения, размерность карты признаков будет меньше, чем у исходной карты (изображения). Если необходимо сохранить тот же размер, применяют так называемые *paddings* — значения, которыми дополняется изображение по краям и которые потом захватываются фильтром вместе с реальными пикселями картинки. Процесс обработки «граничной» области представлен на рис. 11.

Помимо *paddings* на изменение размерности так же влияют *strides* — значения шага, с которым окно перемещается по изображению/карте.

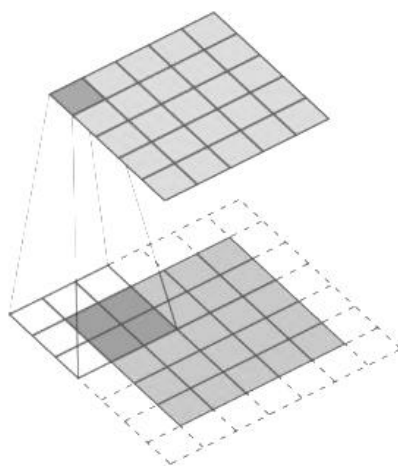


Рис. 11. Обработка «граничной» области

Свертка не является единственным способом получения обобщённой характеристики группы пикселей. Самый простой способ для этого — выбрать один пиксель по заданному правилу, например — максимальный. Именно это и делает слой MaxPooling (рис. 12).

12	20	30	0
8	12	2	0
34	70	37	4
112	100	25	12

 2×2 Max-Pool

20	30
112	37

Рис. 12. Слой «MaxPooling»

В отличие от Convolution, MaxPooling обычно применяется к непересекающимся группам пикселей.

Mask R-CNN развивает архитектуру Faster R-CNN путём добавления ещё одной ветки, которая предсказывает положение маски, покрывающей найденный объект, и, таким образом решает уже задачу instance segmentation [1]. Маска представляет собой просто прямоугольную матрицу, в которой 1 на некоторой позиции означает принадлежность соответствующего пикселя объекту заданного класса, 0 — то, что пиксель объекту не принадлежит (рис. 13).

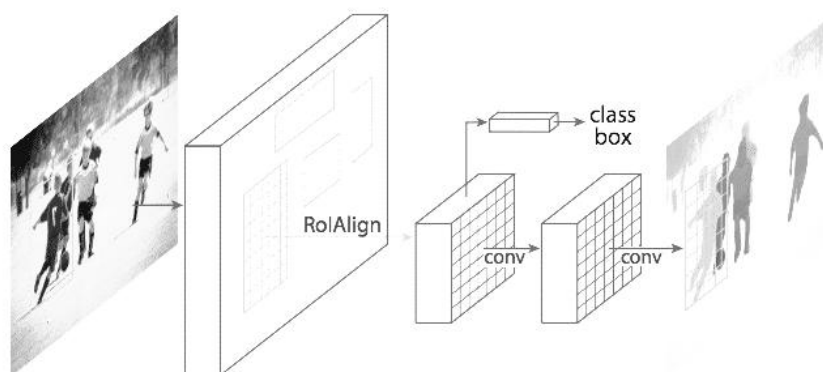


Рис. 13. Ветка сети Mask R-CNN

Авторы архитектуры условно разделяют разработанную архитектуру на CNN-сеть вычисления признаков изображения, называемую ими *backbone*, и *head* — объединение частей, отвечающих за предсказание охватывающей рамки, классификацию объекта и определение его маски. Loss функция для них общая и включает три компонента:

$$L = L_{cls} + L_{box} + L_{mask}, \quad (10)$$

где

L_{cls} — потеря, которая измеряет правильность классификации каждого объекта;

L_{box} — потеря, которая измеряет насколько «тесны» предсказанные ограничивающие рамки относительно основного объекта;

L_{mask} — потеря, которая измеряет, насколько правильна предсказанная сегментирующая маска для изображения.

Выделение маски происходит в *class-agnostic* стиле: маски предсказываются отдельно для каждого класса, без предварительного знания, что изображено в регионе, и потом просто выбирается маска класса, победившего в независимом классификаторе. Утверждается, что такой подход более эффективен, чем опирающийся на априорное знание класса.

Одна из основных модификаций, возникших из-за необходимости предсказывать маску — изменение процедуры RoIPool (вычисляющей матрицу признаков для региона-кандидата) на так называемую RoIAlign. Дело в том, что карта признаков, полученная из CNN (рис. 14), имеет меньший размер, чем исходное изображение, и регион, охватывающий на изображении целочисленное количество пикселей, не получается отобразить в пропорциональный регион карты с целочисленным количеством признаков.



Рис. 14. Карта признаков

В RoIPool проблема решалась просто округлением дробных значений до целых. Такой подход нормально работает при выделении охватывающей рамки, но вычисленная на основе таких данных маска получается слишком неточной.

В противоположность этому, в RoIAlign не используется округление, все числа остаются действительными, а для вычисления значений признаков используется билинейная интерполяция по четырём ближайшим целочисленным точкам.

Разница между признаками региона-кандидата поясняется представленным рис. 15.

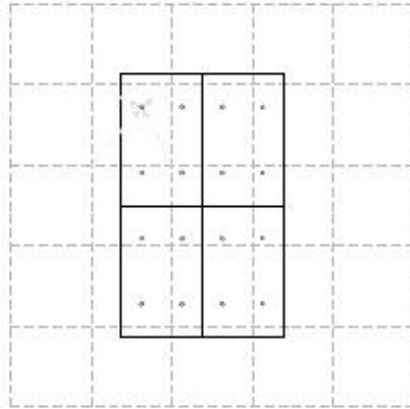


Рис. 15. Разница в вычислении матрицы признаков для региона-кандидата

Здесь штрихованной сеткой обозначена карта признаков, а непрерывной — отображение на карту признаков региона-кандидата с исходной фотографии. В данный регион должно попасть 4 группы для MaxPooling по 4 признака, обозначенных на рисунке точками. В отличие от процедуры RoIPool, которая за счёт округления просто бы выровняла регион по целочисленным координатам, RoIAlign оставляет точки в их текущих местах, но вычисляет значения каждой из них при помощи билинейной интерполяции по четырём ближайшим признакам.

Билинейная интерполяция функции двух переменных производится за счёт применения линейной интерполяции сначала в направлении одной из координат, затем — в направлении другой.

Пусть нужно интерполировать значение функции $f(x, y)$ в точке P при известных значениях функции в окружающих точках (рис. 16):

$$\begin{aligned}
 Q_{11} &= (x_1, y_1) \\
 Q_{12} &= (x_1, y_2) \\
 Q_{21} &= (x_2, y_1) \\
 Q_{22} &= (x_2, y_2)
 \end{aligned}
 \tag{11}$$

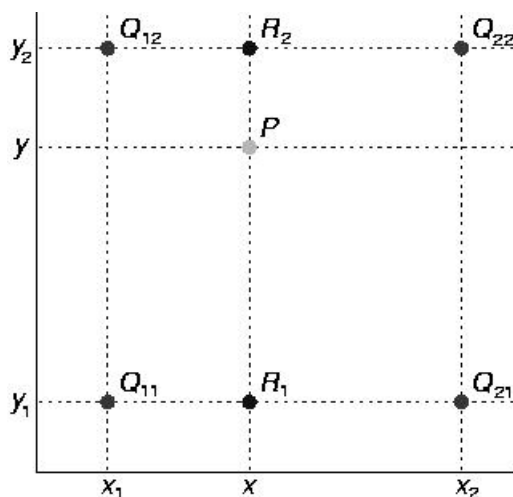


Рис. 16. Билинейная интерполяция функции двух переменных

Для этого сначала интерполируются значения вспомогательных точек R_1 и R_2 , а затем на их основе интерполируется значение в точке P .

Тогда,

$$\begin{aligned} R_1 &= (x, y_1) \\ R_2 &= (x, y_2) \end{aligned} \quad (12)$$

$$f(R_1) = \frac{(x_2 - x)}{(x_2 - x_1)} f(Q_{11}) + \frac{(x - x_1)}{(x_2 - x_1)} f(Q_{21}) \quad (13)$$

$$f(R_2) = \frac{(x_2 - x)}{(x_2 - x_1)} f(Q_{12}) + \frac{(x - x_1)}{(x_2 - x_1)} f(Q_{22}) \quad (14)$$

$$f(P) = \frac{(y_2 - y)}{(y_2 - y_1)} f(R_1) + \frac{(y - y_1)}{(y_2 - y_1)} f(R_2) \quad (15)$$

Конечным результатом является выделение контуров гранул кокса и определение их размера.

Для решения задачи определения размера гранул кокса был создан алгоритм поиска максимального диаметра.

С учетом того, что контур может содержать более 1000 точек, а количество гранул может быть более 30, для ускорения расчетов был использован алгоритм аппроксимации контура.

В методе полигональной аппроксимации итеративно строится такой многоугольник, который содержал бы минимальное число вершин и в то же время представлял фигуру с заданной точностью.

Метод аппроксимации контуров заданными кривыми предполагает представление анализируемых фигур в виде набора стандартных фигур, близких к аппроксимируемым по форме и размерам (прямоугольник, эллипс, ромб, равнобедренный треугольник). Тогда, для дискретного замкнутого контура $B = \{b_1, b_2, \dots, b_n\}$ с координатами контурных точек $b_i = (x_i, y_i)$ из множества стандартных фигур подбирается такая, для которой сумма квадратов расстояний до точек контура минимальна.

При построении отрезка ломаной контура из начала координат проводятся лучи через соседние с (x_i, y_i) точки реестра. Лучи W_j выбираются таким образом, чтобы образованный ими угол имел максимальное значение. Номер луча равен номеру вершины, вносящей соответствующее ограничение на область, которой должен принадлежать узел аппроксимации. После уменьшения количества точек, простым перебором можно определить размер гранулы и её класс.

2.2 Адаптация и тестирование математической модели прогнозирования прочностных характеристик кокса по его гранулометрическому составу

Адаптация математической модели – это процесс настройки параметров модели для достижения оптимальных результатов.

Для того, чтобы получить данные по гранулометрическому составу, в первую очередь необходимо создать модель определения гранулометрического состава доменного кокса по изображениям с конвейера.

Обучение сегментационной нейронной сети Mask R-CNN является процессом с определенным количеством настраиваемых гиперпараметров. В табл. 2 приведены параметры, которые использовались для обучения сегментационной нейронной сети обнаружения кокса.

Таблица 2

Параметры обучения нейронной сети

Параметр	Описание	Значение
DEVICE	Устройство, используемое для обучения нейронной сети	GPU
NUM_WORKERS	Количество процессов, преобразующих, загружающих изображения и данные в устройство для обучения	4
IMS_PER_BATCH	Количество изображений для проведения одной итерации, увеличение данного параметра требует больших объемов памяти	4
MAX_ITER	Количество итераций	25000
WARMUP_ITERS	Количество итераций для корректировки скорости обучения	2000
BASE_LR	Скорость обучения. Чем выше данный параметр, тем быстрее нейронная сеть поймет основы решения задачи, но слишком большие значения приведут к тому, что нейронная сеть не сможет изучить все признаки	0.001
CHECKPOINT_PERIOD	Период проверки нейронной сети на тестовой выборке и сохранения весов	5000

Для обучения и проверки входную базу изображений необходимо разделить на обучающую и тестовую выборку. Общепринятой пропорцией в данном направлении является 70 % на 30 % соответственно. Информация о разделении базы изображений представлена в табл. 3.

Соотношение обучающей и тестовой выборок

Тип	Процент от общей выборки, %	Количество, шт.
Обучающая	70	945
Тестовая	30	405

Анализируя полученные изображения, были выделены следующие варианты загруженности конвейера:

- полная загруженность;
- частичная загруженность;
- низкая загруженность.

На рис. 17 представлен пример изображения, демонстрирующий полную загруженность конвейера. Данное состояние является самым распространенным и принято считать рабочим.

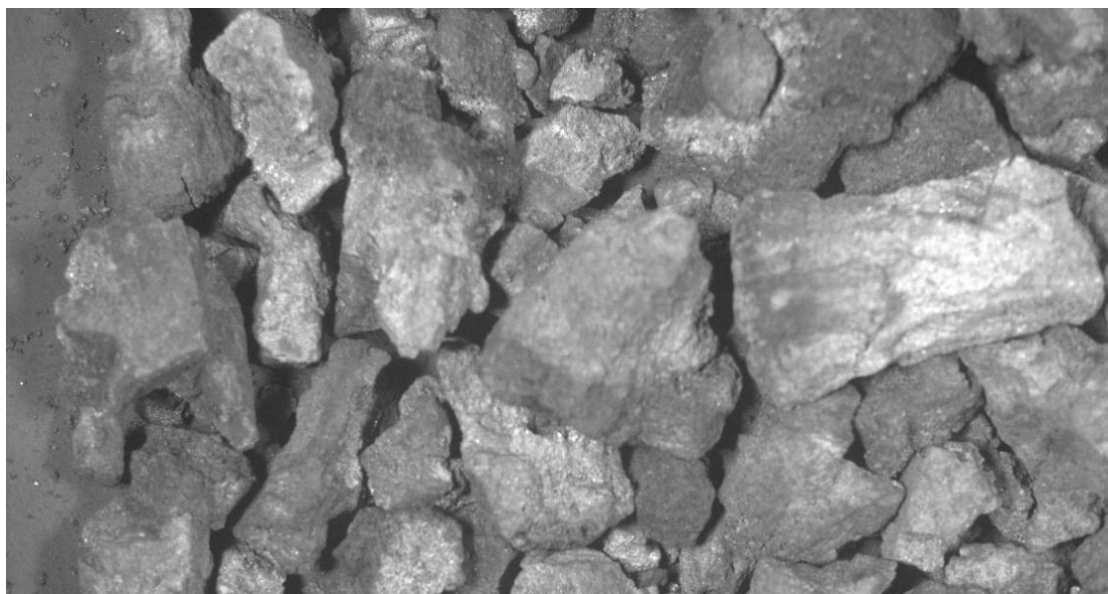


Рис. 17. Полная загруженность конвейера

На рис. 18 представлен пример изображения, демонстрирующий частичную загруженность конвейера. Данное состояние является переходным, что означает начало или конец периода полной загруженности.

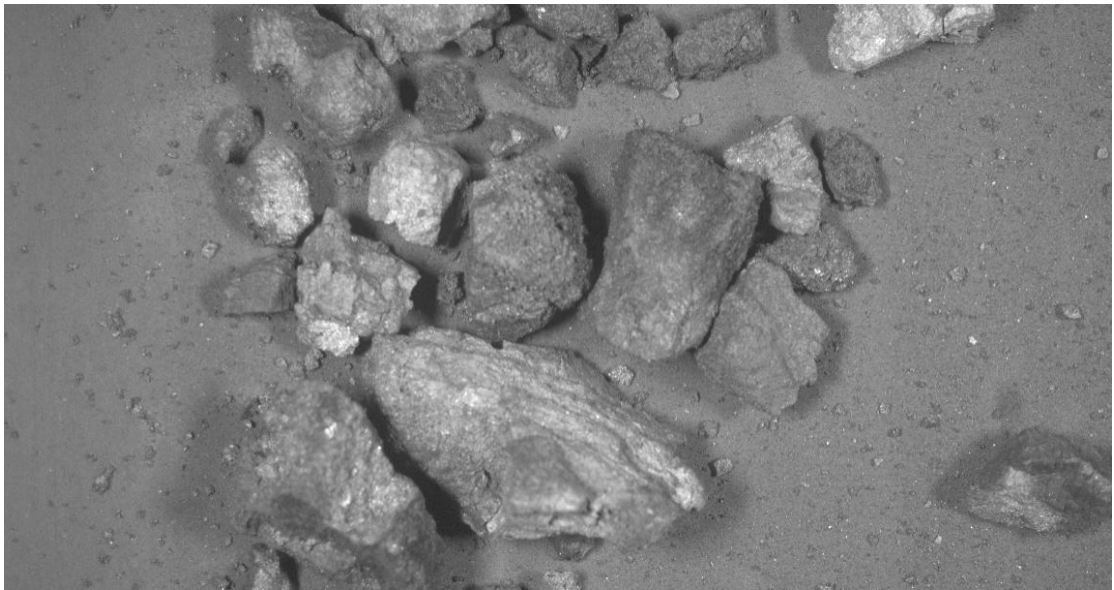


Рис. 18. Частичная загруженность конвейера

На рис. 19 представлен пример изображения, демонстрирующий низкую загруженность конвейера. Данное состояние означает практически полное отсутствие работы конвейера.

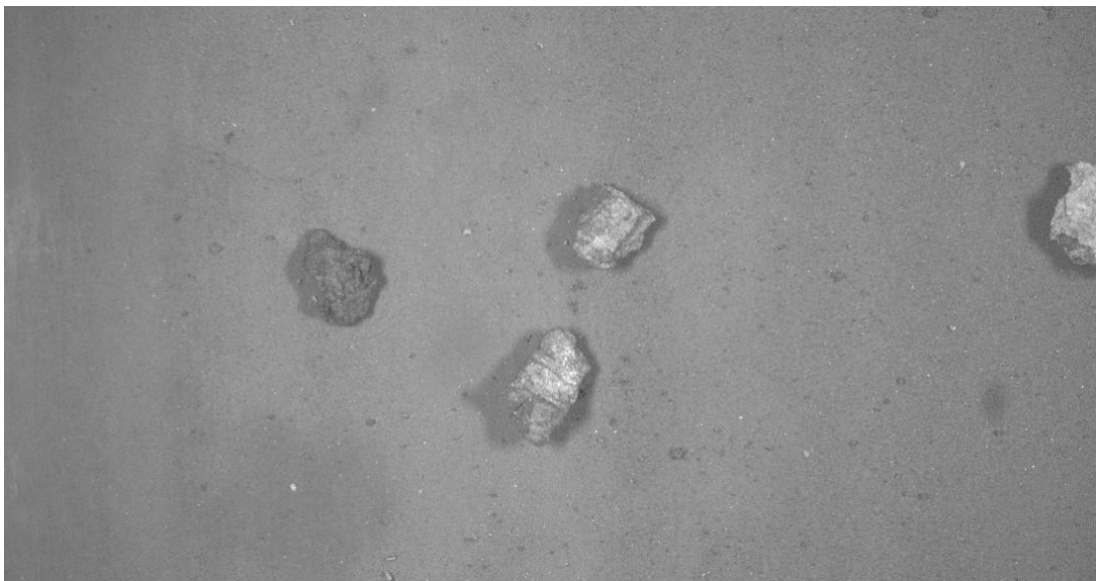


Рис. 19. Низкая загруженность конвейера

Аугментация данных — это методика создания дополнительных обучающих данных из имеющихся. Для достижения хороших результатов глубокие сети должны обучаться на очень большом объеме данных. Следовательно, если исходный обучающий набор содержит ограниченное

количество изображений, необходимо выполнить аугментацию, чтобы улучшить результаты модели.

Существует множество вариантов аугментации. Самыми популярными являются следующие: отражение по горизонтали, случайное кадрирование и изменение цвета. Можно применять различные комбинации, например, одновременно выполнять поворот и случайное масштабирование. Кроме того, можно варьировать величину насыщенности и значения всех пикселей (компоненты S и V цветового пространства HSV). В частности, можно возвести эти компоненты в степень из интервала от 0,25 до 4, умножить их на коэффициент из интервала от 0,7 до 1,4, или прибавить к ним величину из интервала от -0,1 до 0,1. Также можно прибавить величину из интервала от -0,1 до 0,1 к величине тона всех пикселей (компонента H цветового пространства HSV). Аналогичные преобразования можно применить к фрагментам изображений.

Описание методик расширения:

- случайное применение обрезания области изображения от 40 % до 100 %;
- случайное применение поворота изображения от 10 до 180 градусов с шагом 10;
- изменение размера изображения до 850x850;
- возможность случайного отражения по горизонтали;
- возможность случайного отражения по вертикали;
- добавление случайной яркости и контрастности.

Во время процесса обучения были получены графики по параметрам, описание которых представлено в табл. 4.

Таблица 4

Параметры результата обучения нейронной сети

Параметр	Описание	Итоговое значение
loos_cls	потеря, которая измеряет правильность классификации каждого объекта	0.16
loss_mask	потеря, которая измеряет, насколько правильна предсказанная сегментирующая маска для изображения	0.16
loss_box_reg	потеря, которая измеряет, насколько "тесны" предсказанные ограничивающие рамки относительно основного объекта	0.27

На рис. 20 представлен график потери по классификации объекта. Стремительное уменьшение ошибки связано с наличием всего лишь двух классов: объект «гранула» и фон.

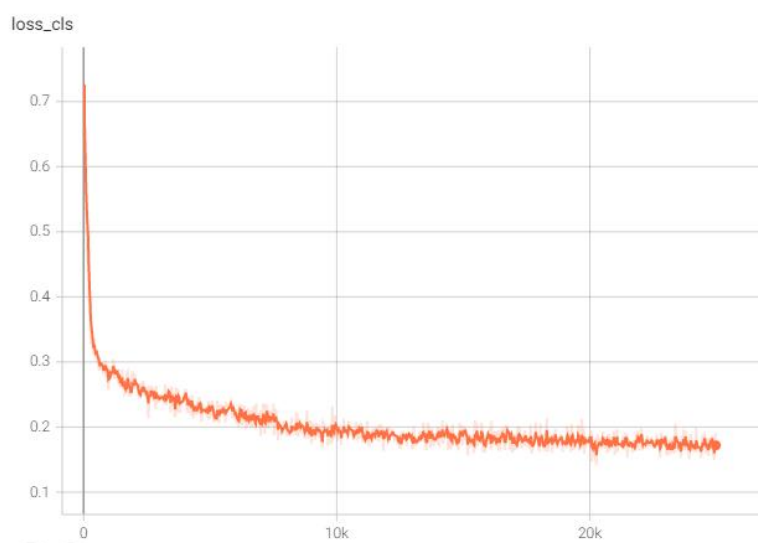
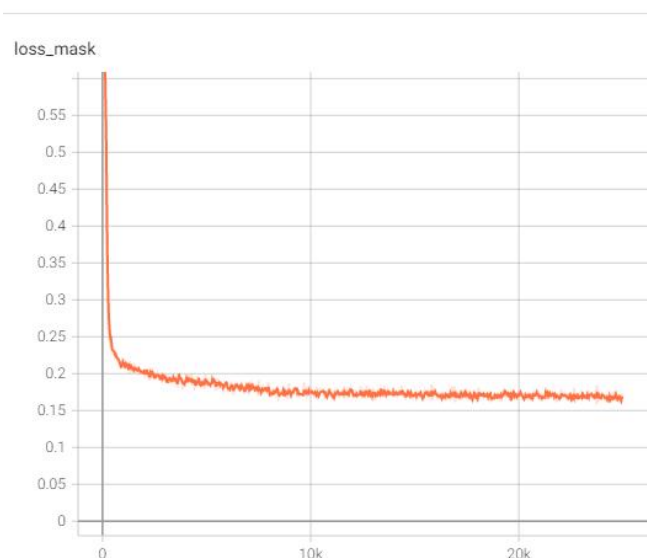
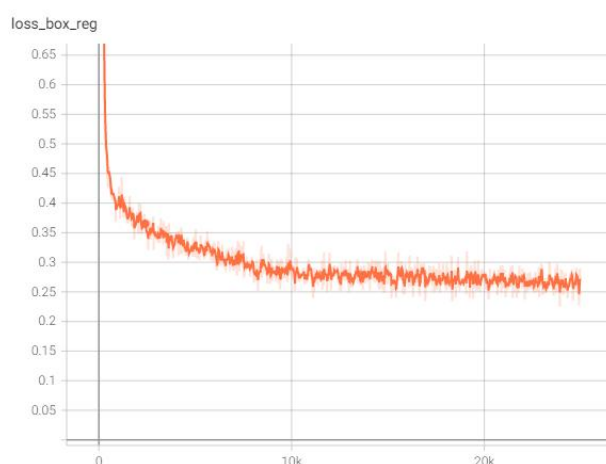


Рис. 20. График показателя loos_cls

На рис. 21 представлен график потери по обнаружению маски классов. Уже на 8 тысячах итераций нейронная сеть смогла понять общие закономерности, а далее лишь изучала особенности формы гранул кокса.

Рис. 21. График показателя `loos_mask`

На рис. 22 представлен график потери по обнаружению областей объектов классов. Значения ошибки находятся в районе 0.25, что связано с близкой расположенностью гранул кокса и их взаимном перекрытии.

Рис. 22. График показателя `loos_box_reg`

В результате тестирования выяснилось, что точность определения класса гранулы кокса более 93 %, что выше целевых показателей в 90%. Подробные результаты тестирования расположены в п.4.3.

Для создания модели прочностных характеристик было собрано два набора данных: измерения гранулометрического состава (табл. 5) и лабораторные измерения прочностных характеристик (табл. 6)

Таблица 5

Измерения гранулометрического состава

Тип	Процент от общей выборки, %	Количество, шт.
Обучающая	80	40172
Тестовая	20	19957

Таблица 6

Лабораторные измерения прочностных характеристик

Тип	Процент от общей выборки, %	Количество, шт.
Обучающая	80	75
Тестовая	20	19

Набор данных с лабораторными измерениями прочностных характеристик был сопоставлен с измерениями гранулометрического состава для создания модели.

В первичном анализе были составлены графики зависимости прочностной характеристики от фракции (рис. 23, рис. 24) и был сделан вывод о том, что для параметра M10 увеличение фракции обратно пропорционально, а для параметра M40 - прямо пропорционально.

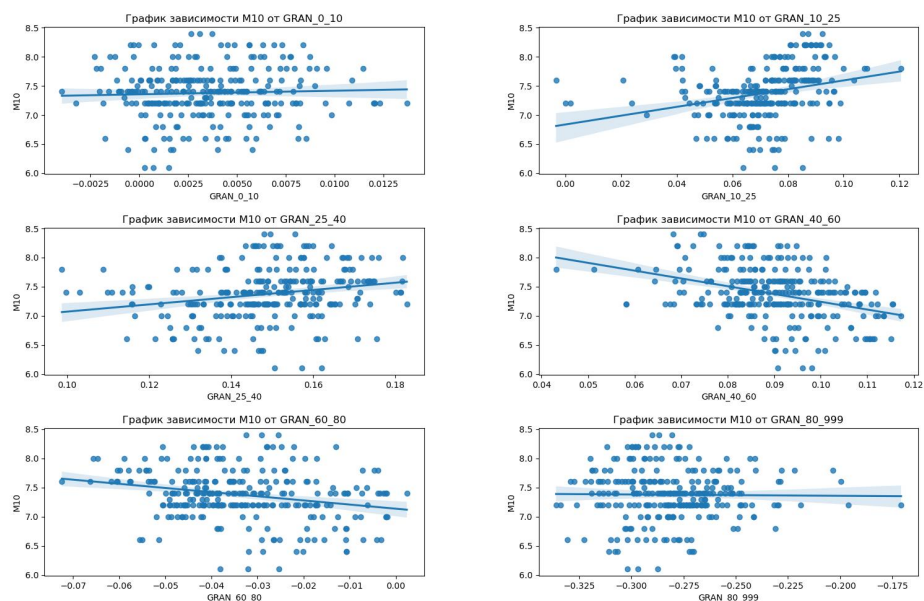


Рис. 23. Зависимость M10 от гранулометрического состава

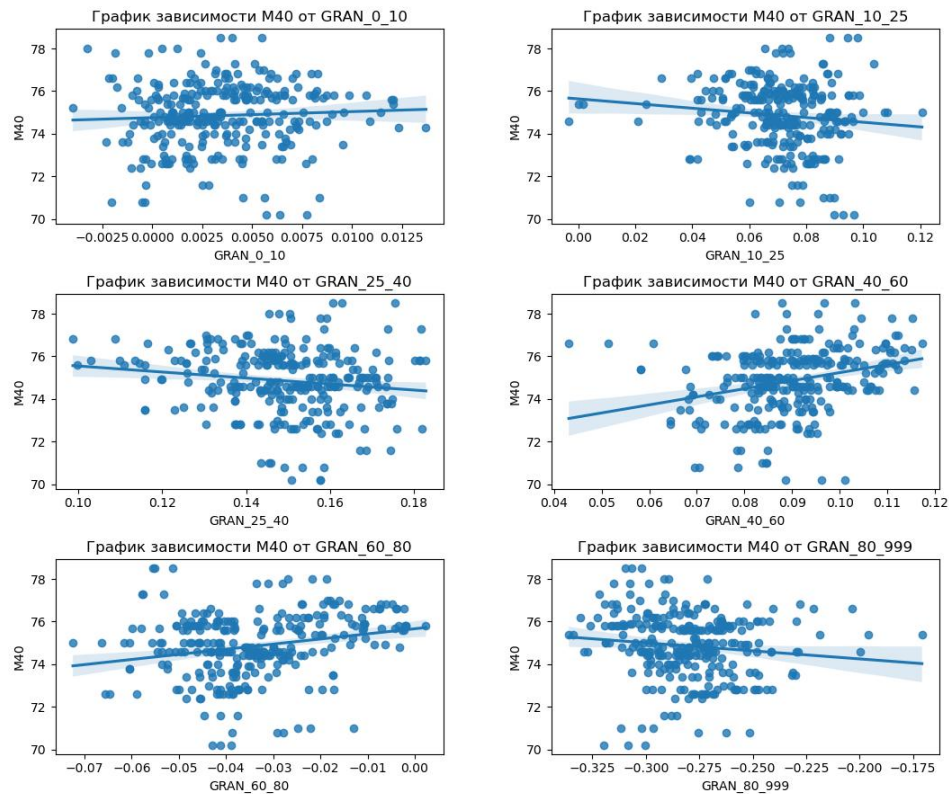


Рис. 24. Зависимость M40 от гранулометрического состава

Архитектура предлагаемой модели будет состоять из двух частей (рис. 25). В первой части будут находиться четыре модели, включающие в себя линейную регрессию, нейронные сети, а также градиентные бустинги от CatBoost и XGBoost. Каждая из этих моделей будет обучаться независимо, а подбор параметров будет осуществляться с использованием метода кросс-валидации.

Метод кросс-валидации — это статистический метод оценки производительности модели машинного обучения и её способности обобщения на новые данные. Он используется для измерения того, насколько успешно модель способна предсказывать данные, которые не участвовали в процессе обучения.

Принцип метода кросс-валидации заключается в разделении доступных данных на две части: набор для обучения и набор для тестирования. Данные разбиваются на несколько подмножеств, и модель обучается на одной части данных (наборе для обучения) и тестируется на другой части (наборе для

тестирования). Этот процесс повторяется несколько раз с различными комбинациями обучающих и тестовых наборов.

Основная цель кросс-валидации — оценить производительность модели на данных, которые она ранее не видела, чтобы избежать переобучения и получить более объективную оценку качества модели.

Во второй части будет представлена обобщающая модель линейной регрессии, которая будет использовать выводы предшествующих моделей вместе с исходными входными данными. Это позволит совместно агрегировать результаты вычислений моделей на основе входных данных.

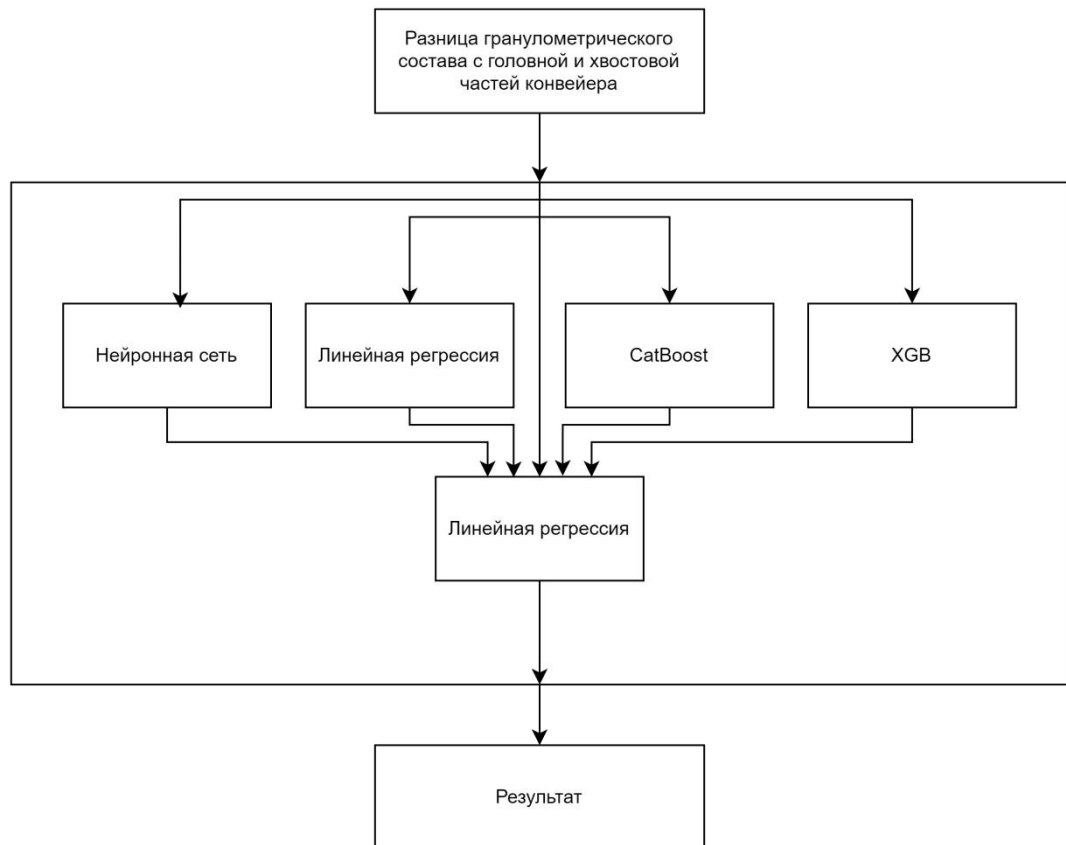


Рис. 25. Архитектура модели прогнозирования прочностных характеристик

Для создания моделей был использован метод поиска гиперпараметров. В табл. 7 представлены границы гиперпараметров для выбранных моделей машинного обучения.

Границы гиперпараметров

Модель	Параметр	Описание	Тип	Значение
1	2	3	4	5
Нейронная сеть	hd_layer_size_i	Размер скрытого слоя	Целое число	от 2 до 32
	n_layers	Количество скрытых слоев	Целое число	от 5 до 10
	activation	Функция активации	Категория	'identity', 'logistic', 'tanh', 'relu'
	solver	Оптимизатор	Категория	'lbfgs', 'sgd', 'adam'
	alpha	L2 регуляризация	Вещественное число	от 0.0001 до 0.1
CatBoost	depth	Глубина дерева	Целое число	От 1 до 7
	learning_rate	Скорость обучения	Вещественное число	От 0.001 до 0.1
	iterations	Количество итераций	Целое число	От 10 до 150
	l2_leaf_reg	L2 регуляризация на листьях	Вещественное число	От 1e-5 до 3
	boosting_type	Тип бустинга	Категория	'Plain', 'Ordered'
	bootstrap_type	Тип бутстрапа	Категория	'Bernoulli', 'MVS'
	leaf_estimation_method	Метод оценки листьев	Категория	'Gradient', 'Newton', 'Exact', 'Hist'
	loss_function	Функция потерь	Категория	'RMSE', 'MAE', 'MAPE'
	subsample	Доля выборки для обучения каждого дерева	Вещественное число	От 0.5 до 1
	grow_policy	Политика роста деревьев	Категория	'Depthwise', 'Lossguide'
	rsm	Случайный признак выбора	Вещественное число	От 0.5 до 1

Продолжение табл. 7

1	2	3	4	5
	Random_strength	Случайная сила	Вещественное число	От 0.5 до 1
XGB	max_depth	Глубина дерева	Целое число	От 1 до 7
	learning_rate	Скорость обучения	Вещественное число	От 0.001 до 0.1
	n_estimators	Количество итераций	Целое число	От 10 до 150
	subsample	Доля выборки для обучения каждого дерева	Вещественное число	От 0.5 до 1
	colsample_bytree	Доля признаков, используемых при построении каждого дерева	Вещественное число	От 0.5 до 1
	reg_alpha	Коэффициент L1-регуляризации	Вещественное число	От 1e-5 до 3
	reg_lambda	Коэффициент L2-регуляризации	Вещественное число	От 1e-5 до 3
	min_child_weight	минимальный вес образца для создания нового узла	Целое число	От 1 до 20
	gamma	Параметр, определяющий минимальное снижение функции потерь для разбиения узла	Вещественное число	От 0 до 1
	booster	Тип бустинга	Категория	'gbtree', 'gblinear'
	tree_method	Метод построения деревьев	Категория	'auto', 'exact', 'approx', 'hist', 'gpu_hist'
Линейная регрессия	degree	Степень полинома для преобразования признаков	Целое число	От 1 до 7
	interaction_only	Должны ли быть преобразованы только взаимодействия между признаками	Логический	
	include_bias	Наличие свободного значения	Логический	
	positive	Положительность коэффициентов	Логический	

После проведения обучения модели прогнозирования M10 были получены следующие ошибки (табл. 8).

Таблица 8

Ошибки при обучении модели прогнозирования M10

Модель	Ошибка
Нейронная сеть	0.177
Линейная регрессия	0.195
CatBoost	0.162
XGB	0.166
Общая модель	0.02

После проведения обучения модели прогнозирования M40 были получены следующие ошибки (табл. 9).

Таблица 9

Ошибки при обучении модели прогнозирования M40

Модель	Ошибка
Нейронная сеть	1.38
Линейная регрессия	1.41
CatBoost	1.31
XGB	1.33
Общая модель	0.29

Более подробно оценка полученных результатов представлена в п.4.4.

2.3 Описание результатов моделирования системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу

В результате, была получена нейронная сеть, способная выполнять сегментацию изображения по поиску гранул кокса на конвейере.

Пример изображения, обработанного нейронной сетью, с полной загрузкой конвейера представлен на рис. 26. Отчетливо видно, что нейронная

сеть обнаруживает более 95 % всех объектов и практически во всех уверена, изредка встречаются объекты с уверенностью ниже 70 %.

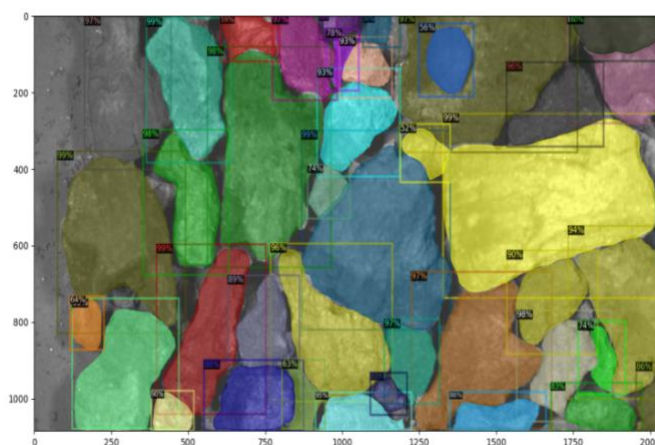


Рис. 26. Сегментация изображения с полной загрузкой конвейера

Пример изображения, обработанного нейронной сетью, с частичной загрузкой конвейера представлен на рисунке рис. 27. Видно, что нейронная сеть распознает как отдельно расположенные гранулы, так и близко лежащие.

Пример изображения, обработанного нейронной сетью, с низкой загрузкой конвейера представлен на рис. 28. Нейронная сеть не распознает слишком мелкие гранулы и тени от гранул, что положительно сказывается на дальнейшей обработке.

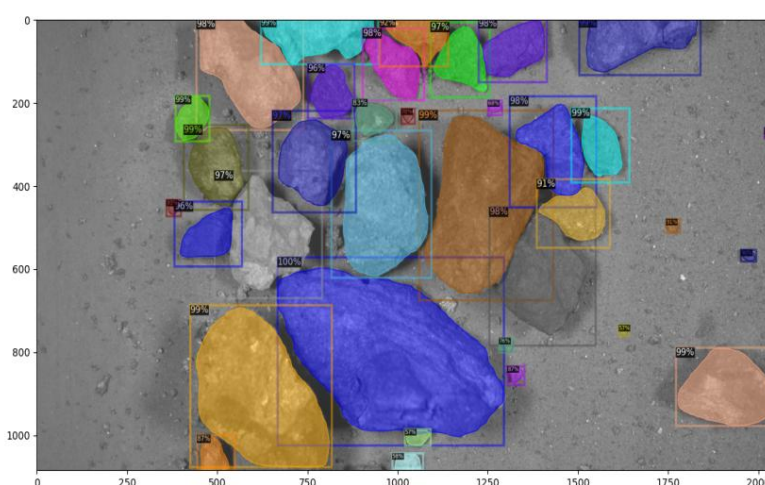


Рис. 27. Сегментация изображения с частичной загрузкой конвейера

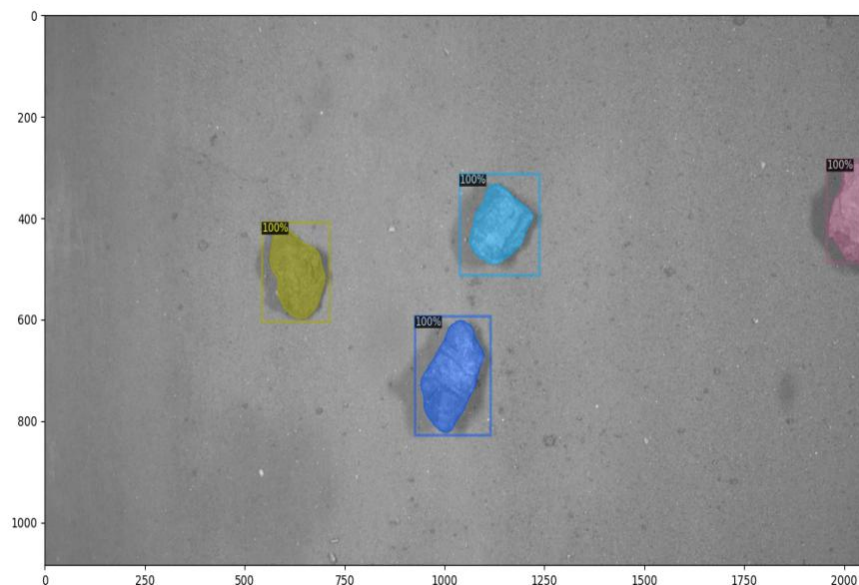


Рис. 28. Сегментация изображения с низкой загрузкой конвейера

Пример изображения, обработанного нейронной сетью, с большим количеством объектов представлен на рис. 29. Данный пример демонстрирует максимальные возможности по обнаружению большого количества объектов, которые находятся близко друг к другу.

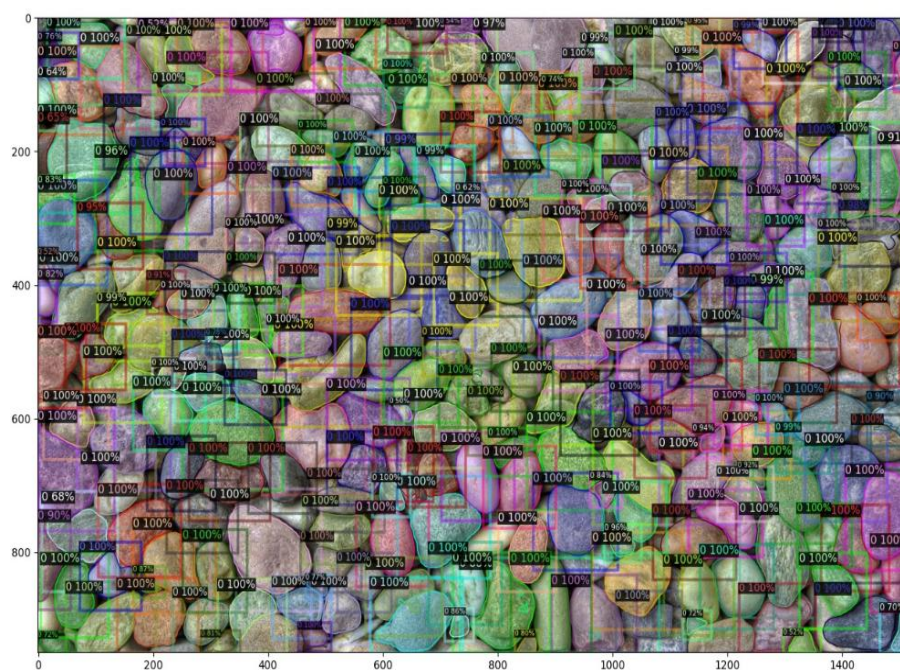


Рис. 29. Сегментация изображения с большим количеством объектов

После создания модели определения гранулометрического состава была создана модель прогнозирования M10 и M40. Далее будут представлены интерпретации обученных моделей с их первичным анализом.

1) Интерпретация моделей прогнозирования M10.

1.1) Полиномиальная регрессия для параметра M10 обучилась со степенью 1, что позволяет интерпретировать её как линейное уравнение, представленное в табл. 10.

Таблица 10

Коэффициенты множителей линейной регрессии M10

Коэффициент	Множитель
7.5349	1
4.2527	GRAN_10_25
0.7666	GRAN_25_40
-12.7379	GRAN_40_60
-0.4166	GRAN_60_80
-2.0029	GRAN_80_999

График сравнения фактического M10 с предсказанием регрессии представлен на рис. 30. Можно сделать первичный вывод, о том, что данная модель тяжело реагирует на отклонения.

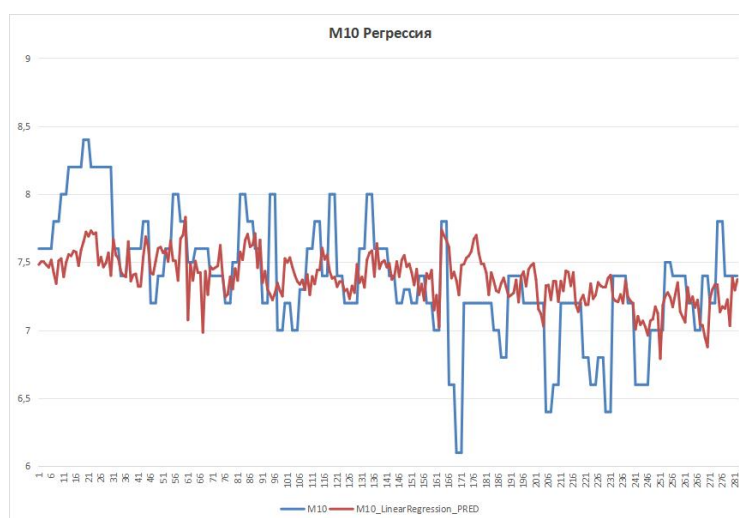


Рис. 30. Сравнение M10 с предсказанием регрессии

1.2) Модель градиентного бустинга CatBoost предоставляет коэффициенты важности параметров (табл. 11), по которой можно сделать вывод о том, что основными показателями являются содержания фракций 10-25 мм и 40-60 мм.

Таблица 11

Важность параметров в CatBoost M10

Поле	Важность
GRAN_0_10	13 %
GRAN_10_25	24 %
GRAN_25_40	14 %
GRAN_40_60	21 %
GRAN_60_80	18 %
GRAN_80_999	11 %

График сравнения фактического M10 с предсказанием CatBoost представлен на рис. 31. Можно сделать первичный вывод, о том, что данная модель приемлемо справляется с задачей предсказания.

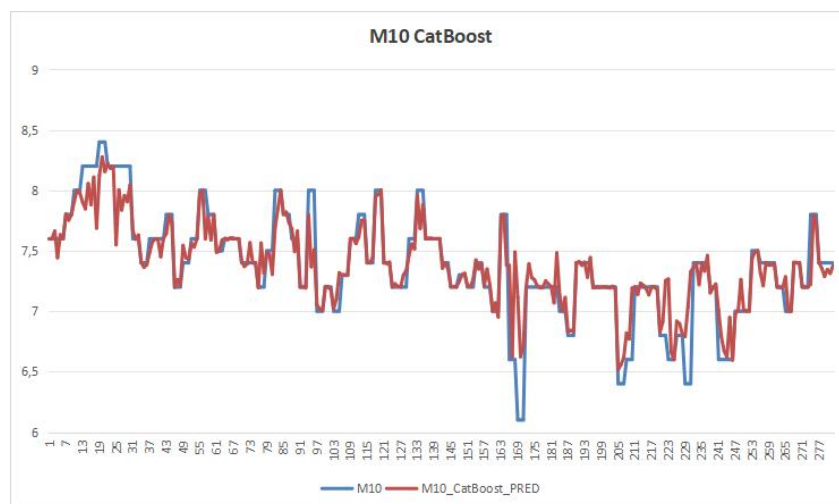


Рис. 31. Сравнение M10 с предсказанием CatBoost

1.3) Модель градиентного бустинга XgbBoost предоставляет коэффициенты важности параметров (табл. 12), по которой можно сделать вывод о том, что основными показателями являются содержания фракций 10-25 мм и 40-60 мм.

Важность параметров в XgbBoost M10

Поле	Важность
GRAN_0_10	10 %
GRAN_10_25	25 %
GRAN_25_40	14 %
GRAN_40_60	19 %
GRAN_60_80	13 %
GRAN_80_999	15 %

График сравнения фактического M10 с предсказанием Xgb представлен на рис. 32. Можно сделать первичный вывод, о том, что данная модель относительно справляется с задачей предсказания, но обладает достаточно большим шумом.

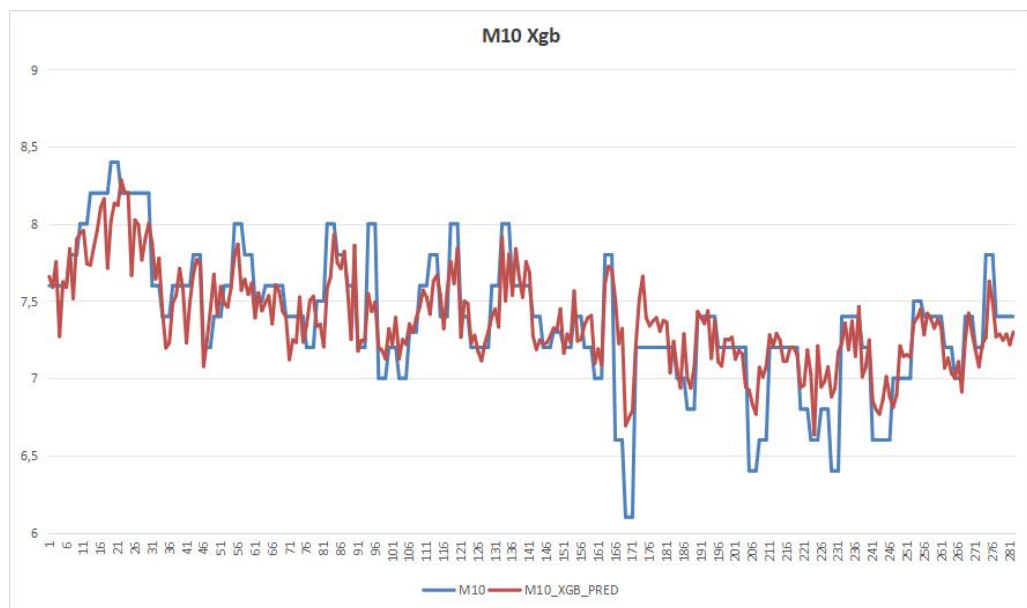


Рис. 32. Сравнение M10 с предсказанием XgbBoost

1.4) Модель полносвязной нейронной сети предоставляет получившуюся архитектуру (табл. 13), по которой можно сказать, что оптимальное число слоев для предсказания параметра M10 — 7.

Слои в нейронной сети M10

Номер скрытого слоя	Количество нейронов
1	26
2	13
3	19
4	23
5	13
6	13
7	5

График сравнения фактического M10 с предсказанием нейронной сети представлен на рис. 33. Можно сделать первичный вывод, о том, что данная модель не смогла определить все выбросы в данных.

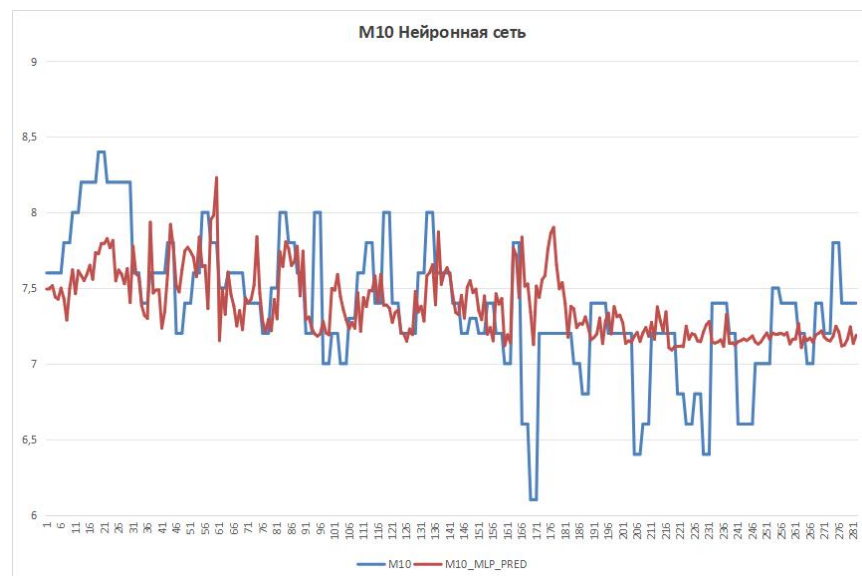


Рис. 33. Сравнение M10 с предсказанием нейронной сети

1.5) Обобщающая модель представляет собой полиномиальную регрессию. Для параметра M10 и других моделей она обучилась со степенью 2, что позволяет интерпретировать её как уравнение, с зависимостями между моделями и входными данными, коэффициенты которого представленные в табл. 14.

Коэффициенты множителей общей модели M10

Коэффициент	Множитель
0.0582	1
0.9796	CatBoost
0.0560	XGB
-0.0407	MLP
-0.0093	LinearRegression
0.0008	GRAN_0_10
0.0013	GRAN_10_25
0.0016	GRAN_80_999
0.0167	CatBoost*XGB
0.0051	CatBoost*GRAN_0_10
0.0120	CatBoost*GRAN_10_25
0.0057	CatBoost*GRAN_80_999
0.0050	XGB*GRAN_0_10
0.0122	XGB*GRAN_10_25
-0.0104	XGB*GRAN_60_80
0.0075	MLP*GRAN_80_999
0.0058	LinearRegression*GRAN_0_10
0.0105	LinearRegression*GRAN_10_25
-0.0109	LinearRegression*GRAN_25_40
-0.0076	LinearRegression*GRAN_40_60
-0.0082	LinearRegression*GRAN_60_80
0.0177	LinearRegression*GRAN_80_999

График сравнения фактического M10 с предсказанием обобщённой модели представлен на рис. 34. Можно сделать первичный вывод, о том, что данная модель хорошо справляется с предсказанием параметра M10, успешно обрабатывая более 80% всех выбросов.

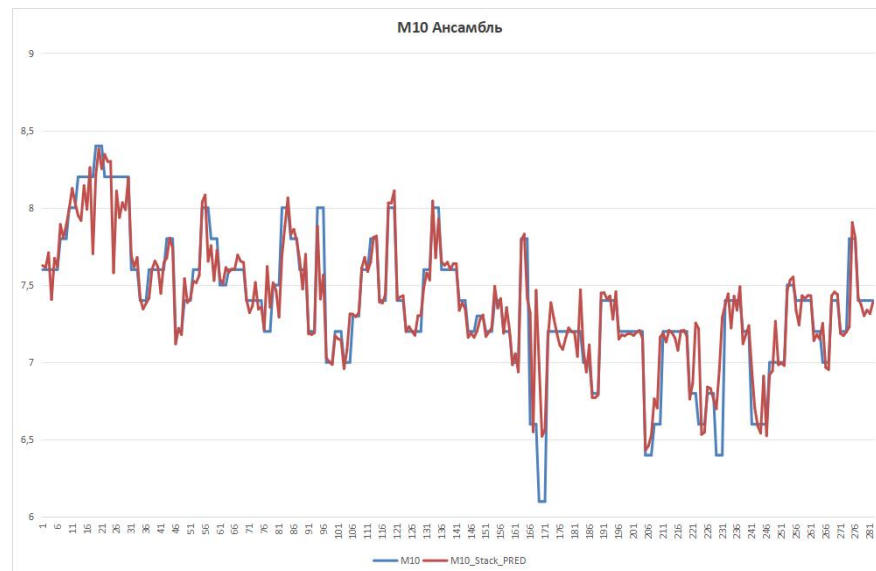


Рис. 34. Сравнение M10 с предсказанием общей модели

2) Интерпретация моделей прогнозирования M40.

2.1) Полиномиальная регрессия для параметра M40 обучилась со степенью 2, что позволяет интерпретировать её как уравнение, представленное в табл. 15.

Таблица 15

Коэффициенты множителей линейной регрессии M40

Коэффициент	Множитель
1	2
73.1818	1
39.2652	GRAN_0_10
-5.5639	GRAN_10_25
3.4624	GRAN_25_40
12.7041	GRAN_40_60
20.5262	GRAN_60_80
0.7337	GRAN_80_999
3.5243	GRAN_0_10*GRAN_10_25
8.4453	GRAN_0_10*GRAN_25_40
3.8574	GRAN_0_10*GRAN_40_60
-2.9975	GRAN_0_10*GRAN_60_80
-12.9617	GRAN_0_10*GRAN_80_999

Продолжение табл. 15

1	2
-4.2519	GRAN_10_25*GRAN_25_40
13.4120	GRAN_10_25*GRAN_40_60
-21.3388	GRAN_10_25*GRAN_60_80
7.6289	GRAN_10_25*GRAN_80_999
52.1940	GRAN_25_40*GRAN_40_60
-27.4819	GRAN_25_40*GRAN_60_80
10.5705	GRAN_25_40*GRAN_80_999
-0.1087	GRAN_40_60*GRAN_60_80
-41.0047	GRAN_40_60*GRAN_80_999
-24.1110	GRAN_60_80*GRAN_80_999

График сравнения фактического М40 с предсказанием регрессии представлен на рис. 35. Можно сделать первичный вывод, о том, что данная модель тяжело реагирует на отклонения.

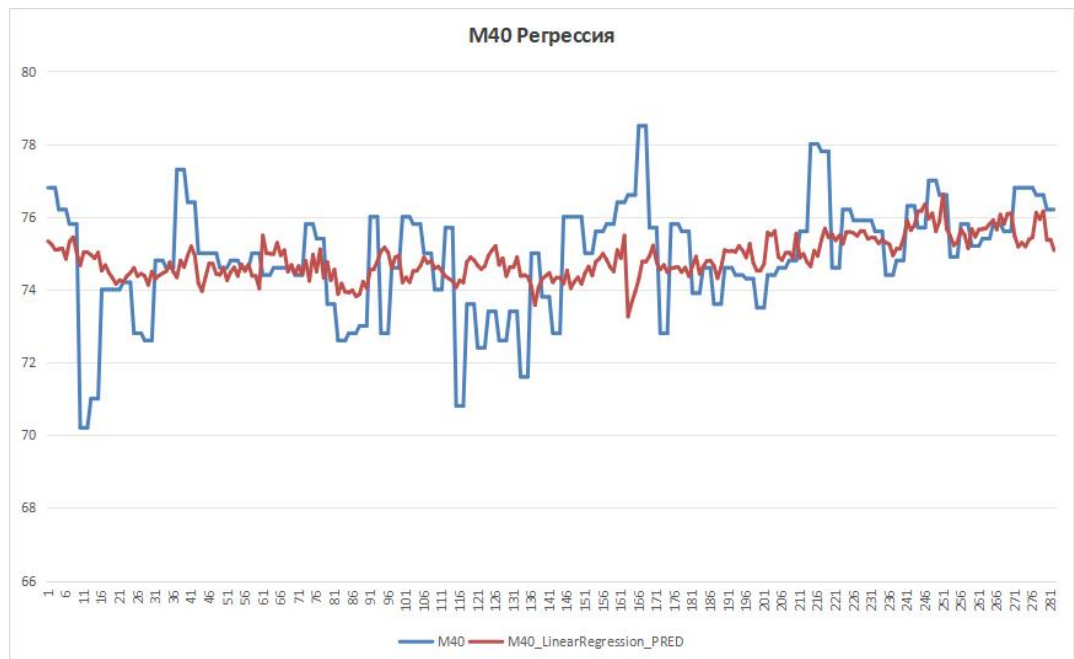


Рис. 35. Сравнение М40 с предсказанием регрессии

2.2) Модель градиентного бустинга CatBoost предоставляет коэффициенты важности параметров (табл. 16), по которой можно сделать вывод о том, что основными показателями являются содержания фракций 40-60 мм и 60-80 мм.

Таблица 16

Важность параметров в CatBoost M40

Поле	Важность
GRAN_0_10	13 %
GRAN_10_25	12 %
GRAN_25_40	13 %
GRAN_40_60	20 %
GRAN_60_80	27 %
GRAN_80_999	13 %

График сравнения фактического M40 с предсказанием CatBoost представлен на рис. 36. Можно сделать первичный вывод, о том, что данная модель приемлемо справляется с задачей предсказания.

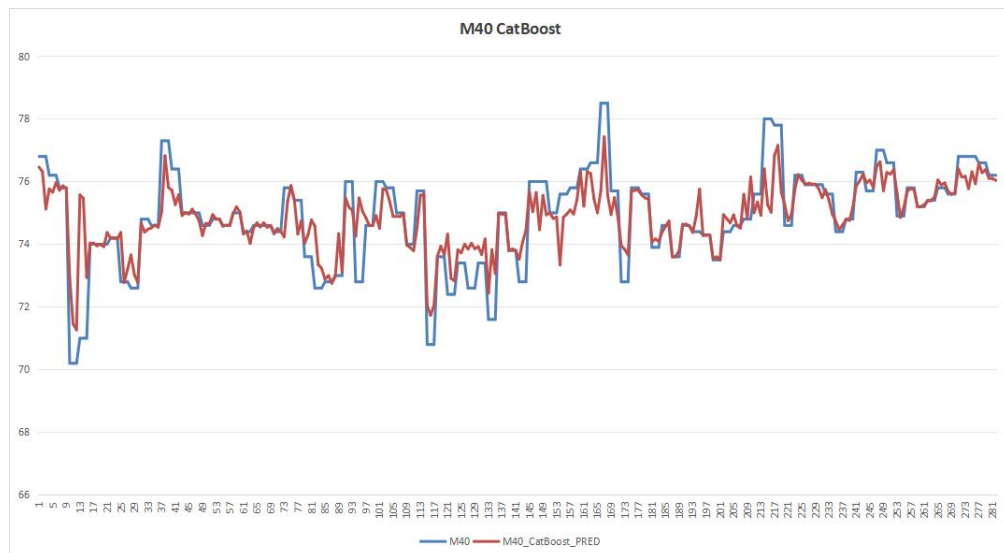


Рис. 36. Сравнение M40 с предсказанием CatBoost

2.3) Модель градиентного бустинга XgbBoost предоставляет коэффициенты важности параметров (табл. 17), по которой можно сделать вывод о том, что основными показателями являются содержания фракций 40-60 мм и 60-80 мм.

Важность параметров в Xgb M40

Поле	Важность
GRAN_0_10	14 %
GRAN_10_25	15 %
GRAN_25_40	13 %
GRAN_40_60	20 %
GRAN_60_80	21 %
GRAN_80_999	14 %

График сравнения фактического M40 с предсказанием Xgb представлен на рис. 37. Можно сделать первичный вывод, о том, что данная модель относительно справляется с задачей предсказания, но обладает небольшой устойчивостью к выбросам и шуму.

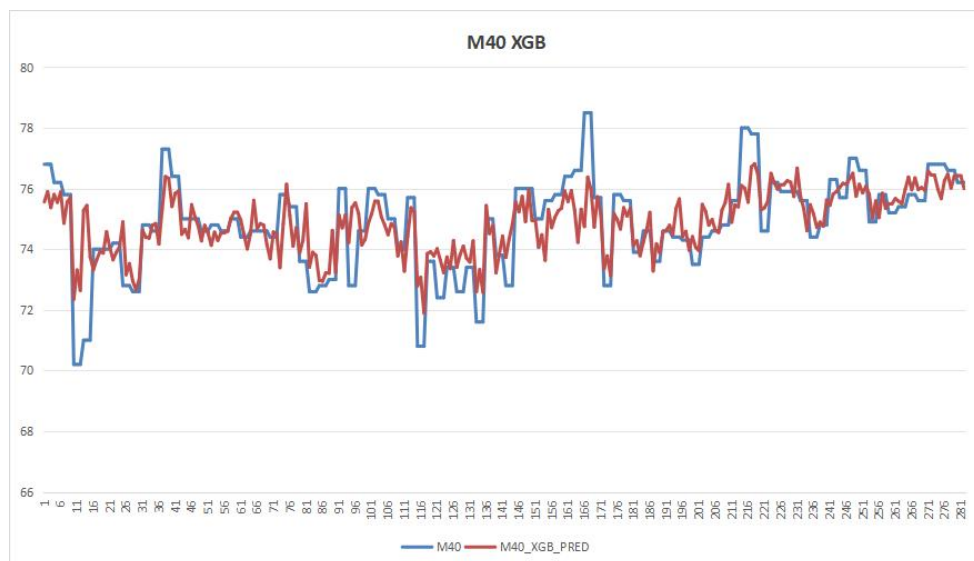


Рис. 37. Сравнение M40 с предсказанием XgbBoost

2.4) Модель полносвязной нейронной сети предоставляет получившуюся архитектуру (табл. 18), по которой можно сказать, что оптимальное число слоев для предсказания параметра M40 — 7.

Слои в нейронной сети М40

Номер скрытого слоя	Количество нейронов
1	18
2	19
3	17
4	21
5	28
6	17
7	16

График сравнения фактического М40 с предсказанием нейронной сети представлен на рис. 38. Можно сделать первичный вывод, о том, что данная модель не смогла определить все выбросы в данных.

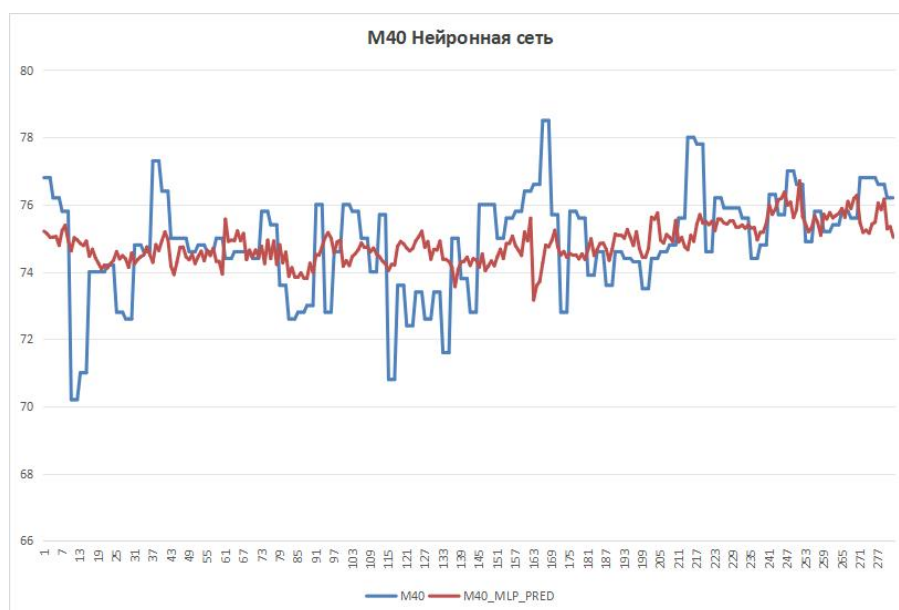


Рис. 38. Сравнение М40 с предсказанием нейронной сети

2.5) Обобщающая модель представляет собой полиномиальную регрессию. Для параметра М40 и других моделей она обучилась со степенью 2, что позволяет интерпретировать её как уравнение, с зависимостями между моделями и входными данными, коэффициенты которого представленные в табл. 19.

Коэффициенты множителей общей модели M40

Коэффициент	Множитель
9.1217	1
1.0233	CatBoost
- 0.0217	XGB
- 0.1238	MLP
- 0.0165	GRAN_25_40
- 0.0076	GRAN_40_60
0.0337	GRAN_80_999
0.0036	CatBoost*XGB
- 0.0016	CatBoost*MLP
- 0.0667	CatBoost*GRAN_0_10
0.0932	CatBoost*GRAN_10_25
- 0.0327	CatBoost*GRAN_25_40
- 0.0149	CatBoost*GRAN_40_60
- 0.1642	CatBoost*GRAN_60_80
0.0146	CatBoost*GRAN_80_999
- 0.0776	XGB*GRAN_0_10
0.0492	XGB*GRAN_10_25
- 0.0136	XGB*GRAN_25_40
- 0.1127	XGB*GRAN_60_80
0.0142	XGB*GRAN_80_999
- 0.0777	MLP*GRAN_10_25
0.0208	MLP*GRAN_25_40
0.0741	MLP*GRAN_40_60
0.1824	MLP*GRAN_60_80
0.0157	LinearRegression*GRAN_0_10
0.0613	LinearRegression*GRAN_10_25
0.0188	LinearRegression*GRAN_25_40
0.0702	LinearRegression*GRAN_40_60
0.1529	LinearRegression*GRAN_60_80

График сравнения фактического M40 с предсказанием обобщённой модели представлен на рис. 39. Можно сделать первичный вывод, о том, что данная модель хорошо справляется с предсказанием параметра M10, успешно обрабатывая более 80% всех выбросов, но обладает небольшим шумом.

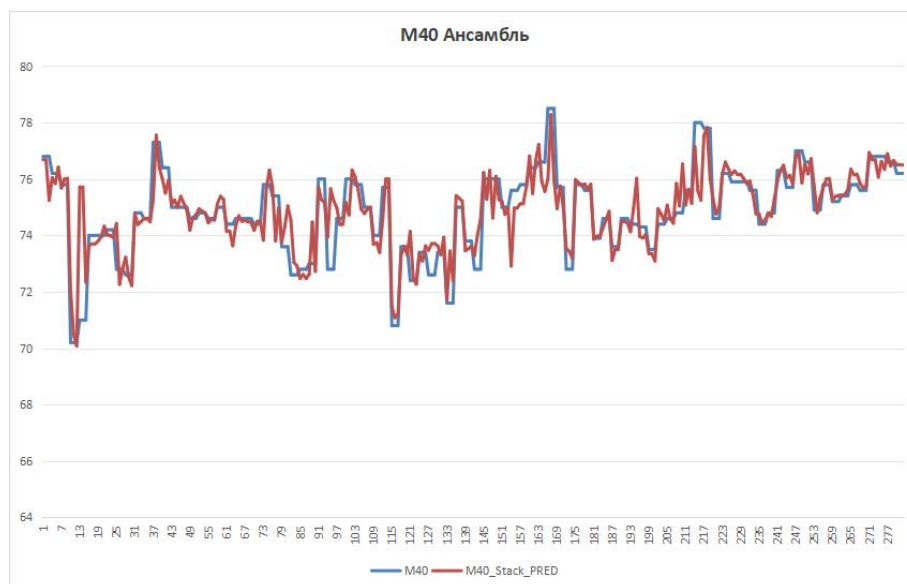


Рис. 39. Сравнение M40 с предсказанием общей модели

Следующим шагом после разработки модели идёт разработка методов и алгоритмов.

2.4 Разработка метода и алгоритмов прогнозирования прочностных характеристик кокса по его гранулометрическому составу

Предлагаемый метод можно описать следующим образом. На первом и втором этапах определяется гранулометрический состав кокса с головной и хвостовой частей конвейера соответственно, за данный процесс отвечает подсистема определения гранулометрического состава.

Далее для каждой фракции идёт расчет разности и после результат подается на вход в подсистему прогнозирования прочностных характеристик. В данной подсистеме идёт расчёт с помощью метода линейной регрессии показателей M10 (твёрдость) и M40 (стабильность).

Как результат, метод предоставляет определенный гранулометрический состав кокса, представленный в виде содержаний отдельно взятых фракций согласно требованиям и рассчитанные параметры M10, M40.

Описанный ранее метод, показан с помощью алгоритма, отображенного на рис. 40.



Рис. 40. Алгоритм прогнозирования прочностных характеристик кокса

Алгоритм прогнозирования прочностных характеристик кокса включает в себя следующие шаги:

1. Получение изображения с хвостовой и головной частей конвейера;
2. Определение гранулометрического состава для изображения с головной части конвейера;

3. Определение гранулометрического состава для изображения с хвостовой части конвейера;

4. Расчёт разности для каждой фракции между головной и хвостовой частей конвейера;

5. Прогнозирование M10;

6. Прогнозирование M40.

Алгоритм определения гранулометрического состава кокса отображен на рис. 41.



Рис. 41. Алгоритм определения гранулометрического состава кокса с изображения

Алгоритм определения гранулометрического состава кокса включает в себя следующие действия шага:

1. Получение изображения на вход;
2. Получение контуров с помощью нейронной сети Mask R-CNN;
3. Далее для каждого контура необходимо применить описанный ранее метод аппроксимации контуров и произвести расчёт размера гранулы путём поиска наибольшей диагонали;
4. После необходимо произвести расчёт содержания фракций с помощью соответствующего алгоритма.

Алгоритм для вычисления содержания фракций представлен на рис. 42.

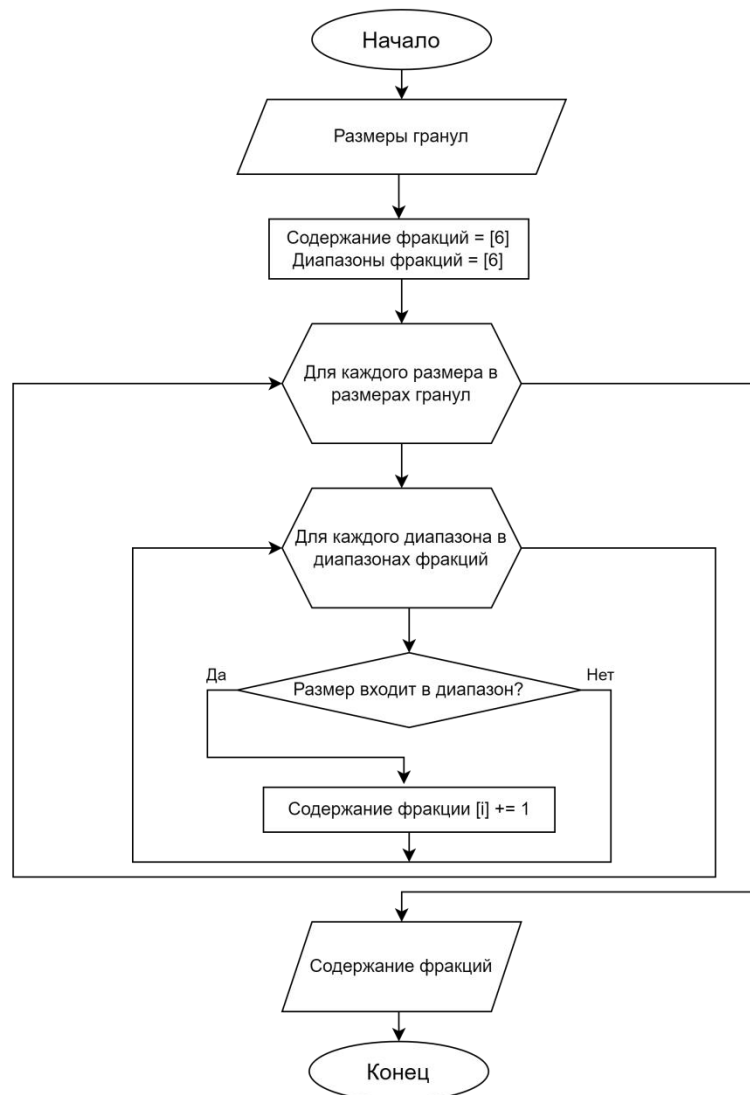


Рис. 42. Алгоритм вычисления содержания фракций

Алгоритм вычисления содержания фракций включает в себя следующие операции:

1. Получения размеров гранул на вход;
2. Создание списка нулей содержание фракций и списка диапазонов фракций согласно п.1.3;
3. Далее происходит полный перебор комбинаций полученных размеров и диапазонов, который увеличивает на единицу соответствующий элемент списка содержания фракций, который в конечном счете является результатом работы.

После разработки метода и алгоритмов необходимо перейти к проектированию системы.

2.5 Выводы по разделу 2

В данном разделе были построены модель определения гранулометрического состава кокса на основе сверточной сегментационной нейронной сети, а также, модель прогнозирования прочностных характеристик кокса на основе ансамбля моделей градиентного бустинга, линейными моделями и моделями нейронных сетей. Произведена адаптация и тестирования моделей. Разработаны методы и алгоритмы, позволяющие определять гранулометрический состав и прогнозировать прочностные характеристики в реальном времени.

3 Проектирование системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу

3.1 Концептуальное моделирование системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу

Концептуальная (содержательная) модель — это абстрактная модель, определяющая структуру моделируемой системы, свойства её элементов и причинно-следственные связи, присущие системе и существенные для достижения цели моделирования. Концептуальная модель относится к моделям, которые формируются после процесса концептуализации или обобщения [7].

3.1.1 Модель «черного ящика»

Модель системы, представляющая собой «черный ящик» с известными выходными и входными параметрами и неизвестным внутренним устройством [9]. С помощью такой модели можно получить представление о внутреннем устройстве. Пример черного ящика разрабатываемой системы представлен на рис. 43.



Рис. 43. Модель «Черного ящика»

В табл. 20 представлено описание входных и выходных параметров у данной модели.

Таблица 20

Описание входных и выходных параметров модели черного ящика

Входы/выходы	Описание
Кадр с хвостовой части	Кадр, полученный с камеры, расположенной в конце конвейера
Кадр с головной части	Кадр, полученный с камеры, расположенной в начале конвейера
Алгоритмы определения гранулометрического состава	Алгоритмы, с помощью которых будет производится определение гранулометрического состава
Алгоритмы прогнозирования прочностных характеристик	Алгоритмы, с помощью которых будет производится прогнозирование прочностных характеристик
Гранулометрический состав	Распределение кокса по классам крупности, выраженное в процентах к массе исходной пробы
Прочностные характеристики кокса	M10 (твёрдость) и M40 (стабильность)

3.1.2 Модель состава

Модель «черного ящика» отражает внешние свойства системы. Но у любой системы есть свой состав, ее отдельные части. При более детальном рассмотрении некоторые части системы могут быть в свою очередь разбиты на составные части и т. д., те части системы, которые рассматриваются как неделимые, будут называться элементами [9]. Части системы, состоящие более чем из одного элемента, называются подсистемами. В результате получается модель состава системы, описывающая, из каких подсистем и элементов она состоит.

Модель состава разрабатываемой системы представлена на рис. 44.

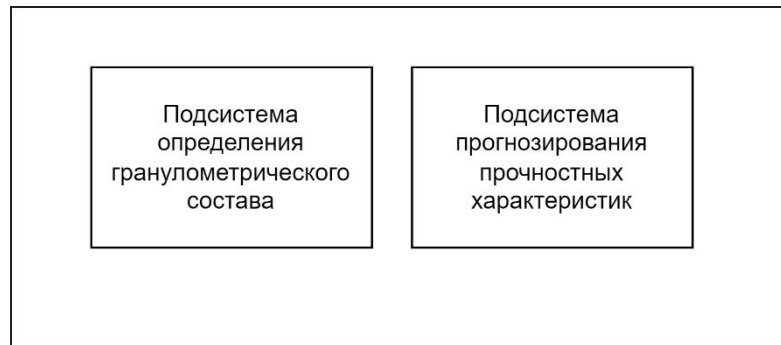


Рис. 44. Модель состава

В табл. 21 представлено краткое описание подсистем проектируемой модели состава. Таблица включает в себя указанные выше подсистемы и их краткое описание в рамках проектируемой системы.

Таблица 21

Описание подсистем

Подсистема	Описание
Подсистема определения гранулометрического состава	Производит определение гранулометрического состава с помощью применения сегментационной сверточной нейронной сети
Подсистема прогнозирования прочностных характеристик	Производит прогнозирование прочностных характеристик твердости и стойкости используя методы регрессионного анализа

3.1.3 Модель структуры

После построения модели состава системы производится этап построения структуры моделируемой системы, в результате получается модель структуры системы. Данная модель отражает внутренние связи составных частей системы [9].

Модель структуры представлена на рис. 45.

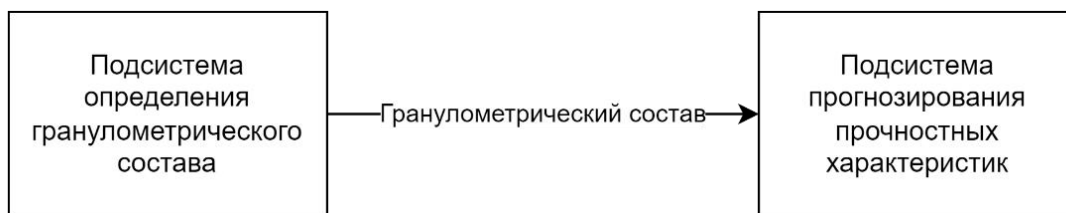


Рис. 45. Модель структуры

Подсистема определения гранулометрического состава передает в подсистему прогнозирования прочностных характеристик гранулометрический состав для прогнозирования характеристик твердости и стойкости.

3.1.4 Модель «белого ящика»

Одной из важных моделей на данном этапе является модель «белого ящика» системы. Входы и выходы «черного ящика» связываются с элементами и подсистемами модели структуры системы. Модель «белого ящика» представлена на рис. 46.

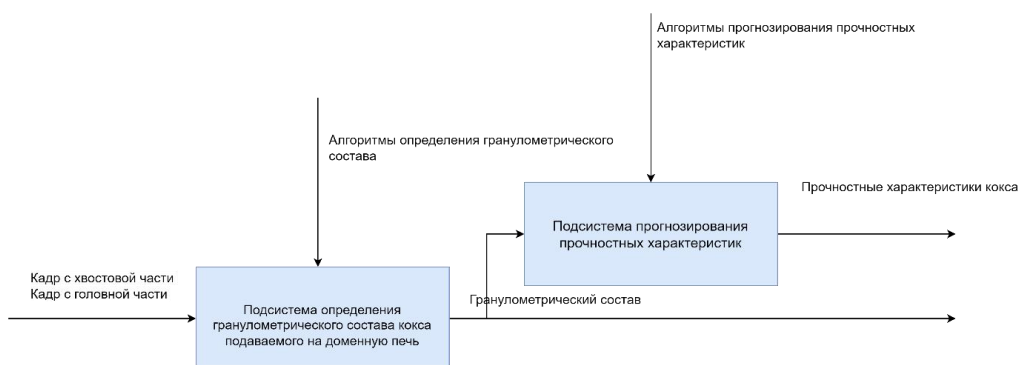


Рис. 46. Модель «белого ящика»

В «белом ящике» указываются все элементы системы, все связи между элементами внутри системы и связи определенных элементов с окружающей средой (входы и выходы системы) [10].

Получаемые на вход данные обрабатываются подсистемой определения гранулометрического состава. В дальнейшем результат работы данной

подсистемы переходит к подсистеме прогнозирования прочностных характеристик.

И как результат система предоставляет определенный гранулометрический состав и спрогнозированные прочностные характеристики.

3.2 Проектирование информационного обеспечения прогнозирования прочностных характеристик кокса по его гранулометрическому составу

3.2.1 Моделирование потоков информации в проектируемой системе

Диаграммы потоков данных обеспечивают удобный способ описания передаваемой информации между частями моделируемой системы. Они содержат два типа объектов: собирающие и хранящие информацию (хранилища данных и внешние сущности) – объекты, которые моделируют взаимодействие с теми частями системы, которые выходят за границы моделирования.

Диаграмма представлена на рис. 47. Детализация контекстной диаграммы представлена на рис. 48.

Она состоит из следующих процессов: определение гранулометрического состава, визуализация данных и прогнозирования прочностных характеристик.

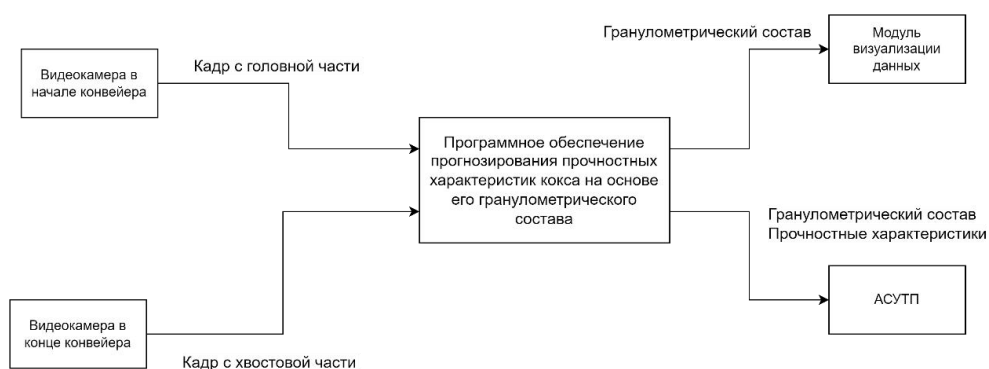


Рис. 47. Обобщенная модель потоков данных

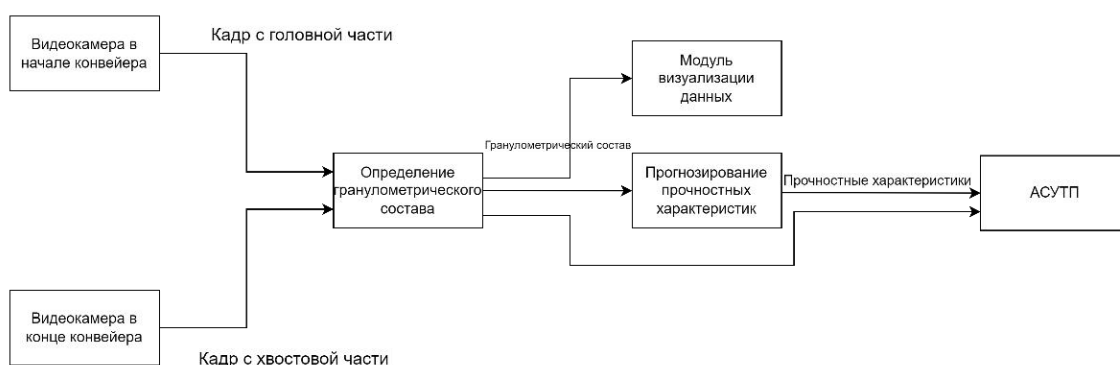


Рис. 48. Детализация диаграммы потоков данных

На вход системе подаются кадры с хвостовой и головной частей конвейера. В первую очередь они обрабатываются в блоке, отвечающего за определение гранулометрического состава, после которого результаты отправляются в модуль визуализации, АСУТП и в следующий блок. В блоке прогнозирования прочностных характеристик происходит расчёт параметров твердости и стойкости, которые отправляются в АСУТП.

3.3 Проектирование программного обеспечения прогнозирования прочностных характеристик кокса по его гранулометрическому составу

3.3.1 Выбор технологии проектирования, среды и языка программирования

3.3.1.1 Выбор модели жизненного цикла

Жизненный цикл программного обеспечения (ПО) — период времени, который начинается с момента принятия решения о необходимости создания программного продукта и заканчивается в момент его полного изъятия из эксплуатации [12]. Среди моделей жизненного цикла программного обеспечения наиболее известны следующие:

- каскадная модель;
- итерационная модель;

- инкрементная модель;
- спиральная модель;
- гибкая модель разработки Agile.

Исходя из требований заказчика к реализации программного продукта была выбрана гибкая модель разработки (рис. 49). Agile — это итеративный подход к управлению проектами и разработке программного обеспечения, который помогает командам быстрее и с меньшими проблемами поставлять ценность клиентам [13].

Вместо того чтобы выпускать весь продукт целиком, команда, следующая принципам Agile, выполняет работу в рамках небольших, но удобных инкрементов. Требования, планы и результаты оцениваются непрерывно, благодаря чему команды могут быстро реагировать на изменения.

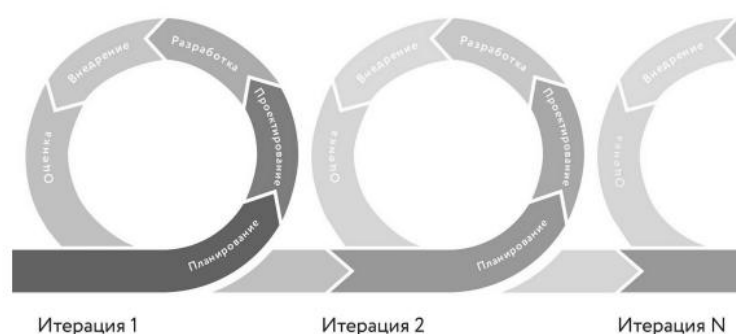


Рис. 49. Гибкая модель разработки

Выбор данной модели жизненного цикла для разработки системы обусловлен следующими причинами:

- данный тип модели ЖЦ позволяет вносить изменения на каждом этапе проекта, адаптировать проект под требования владельца продукта, снижать финансовые риски и быстро запускать продукт;
- на каждом этапе разработки формируется законченный набор проектной документации, проверенный на полноту и согласованность;

- гибкая модель предоставляет возможность начать разработку сразу после согласования бизнес-модели, общей стратегии и функциональных требований.

3.3.1.2 Выбор подхода к разработке

На текущий момент существует два основных подхода: структурный и объектно-ориентированный.

Сущность структурного подхода к разработке заключается в декомпозиции информационной системы на автоматизируемые функции: система разбивается на функциональные подсистемы, которые в свою очередь делятся на подфункции, подразделяемые на задачи и так далее. Процесс разбиения продолжается вплоть до конкретных процедур. При этом автоматизируемая система сохраняет целостное представление, в котором все составляющие компоненты взаимосвязаны [14].

Объектно-ориентированное программирование (ООП) — методология программирования, основанная на представлении программы в виде совокупности объектов, каждый из которых является экземпляром определённого класса, а классы образуют иерархию наследования [15].

В качестве подхода к разработке программного обеспечения было выбрано Объектно-ориентированное программирование (ООП). Выбор данного подхода обусловлен двумя факторами:

- относительная простота разработки программного обеспечения и дальнейшая модернизация;
- данный подход позволит достаточно легко и точно декомпозировать объекты предметной области.

В качестве стандарта проектирования и написания программной документации и спецификации был выбран язык моделирования UML.

UML – унифицированный язык моделирования. Он представляет собой систему обозначений, которую можно применять для объектно-

ориентированного анализа и проектирования [16]. Его можно использовать для визуализации, спецификации, конструирования и документирования программных систем. UML обладает следующим преимуществами:

- UML объектно-ориентирован, в результате чего методы описания результатов анализа и проектирования семантически близки к методам программирования на современных объектно-ориентированных языках;
- UML позволяет описать систему практически со всех возможных точек зрения и в различных аспектах поведения системы;
- диаграммы UML сравнительно просты для чтения после достаточно быстрого ознакомления с его синтаксисом;
- UML получил широкое распространение и быстро развивается.

Исходя из требований и сформированных задач, выбранного подхода к разработке программного обеспечения необходимо выбрать набор инструментов, с помощью которых будет создаваться предлагаемая информационная система.

3.3.1.3 Выбор среды и языка программирования

Для реализации серверной части будет использоваться язык программирования C# с его технологией создания сервисов ASP.NET 8. Для хранения данных будет использоваться база данных Oracle с её встроенным языком PLSQL, с которым будет работать библиотека Entity Framework.

Для того, чтобы работать с нейронными сетями и математическими моделями, будет использоваться язык программирования python и соответственно библиотеки Pytorch и Scipy.

Для реализации клиентского приложения будет использоваться язык программирования TypeScript с фреймворком React, имеющий библиотеку Material Design для создания стилизованных компонентов.

Разработка программной документации проводилась в соответствии с Единой системой программной документации (ЕСПД), позволяющая

облегчить процесс сопровождения программного решения как самим создателем ПО, так и сторонним лицам, сопровождающим разработанный программный продукт.

3.3.2 Разработка логической модели

3.3.2.1 Диаграмма вариантов использования

Одним из стандартных элементов языка UML является диаграмма вариантов использования, которая позволяет наглядно представить ожидаемое поведение системы. Основными понятиями диаграмм вариантов использования являются: действующее лицо, вариант использования, связь. Диаграмма вариантов может дополняться пояснительным текстом, который раскрывает смысл или семантику составляющих ее компонентов. Такой пояснительный текст получил название примечания или сценария.

Отдельный вариант использования обозначается на диаграмме эллипсом, внутри которого содержится его краткое название или имя в форме глагола с пояснительными словами.

Действующее лицо (actor) – внешняя по отношению к разрабатываемому программному обеспечению сущность, которая взаимодействует с ним с целью получения или предоставления какой-либо информации [17]. Действующими лицами могут быть пользователи, другое программное обеспечение или какие-либо технические средства, взаимодействующие с разрабатываемым программным обеспечением.

Вариант использования - некоторая очевидная для действующего лица процедура, решающая его конкретную задачу [17]. Все варианты использования, так или иначе, связаны с требованиями к функциональности разрабатываемой системы и могут сильно отличаться по объему выполняемой работы. Связь - взаимодействие действующих лиц и соответствующих вариантов использования. Варианты использования также

могут быть связаны между собой. При этом фиксируют связи использования и расширения.

Диаграмма вариантов использования для программного обеспечения прогнозирования прочностных характеристик кокса представлена на рис. 50.

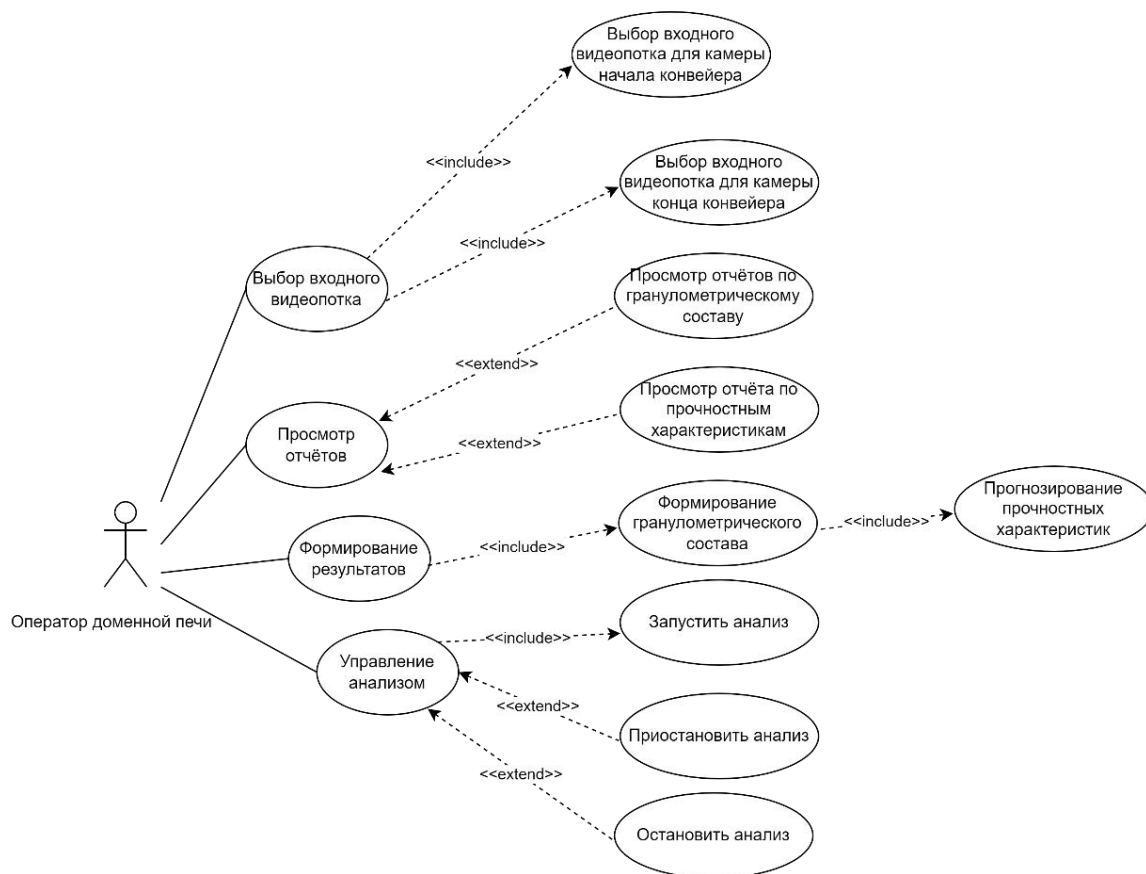


Рис. 50. Диаграмма вариантов использования

Действующим лицом является оператор доменной печи, который является пользователем системы. Он осуществляет работу с программой.

Пользователь может сформировать входной видеопоток, что включает в себя выбор камеры для начала и конца конвейера. Также пользователь может просматривать отчёт и формирование результатов по гранулометрическому составу и его прочностным характеристикам. Одним из ключевых действием пользователя является управление анализом, что включает в себя его запуск, приостановление и остановку.

В табл. 22 представлено краткое описание варианта «Выбор входного видеопотока»

Таблица 22

Краткое описание варианта использования «Выбор входного видеопотока»

Название	Выбор входного видеопотока
Цель	Выбрать входной видеопоток
Действующие лица	Пользователь
Краткое описание	Пользователь выбирает видеопоток, на котором будет происходить обнаружение людей в опасности
Тип варианта	Основной

Типичный ход событий для данного варианта представлен в табл. 23.

Таблица 23

Типичный ход событий для варианта использования «Выбор входного видеопотока»

Действие исполнителя	Отклик системы
1 Пользователь обращается к выбору видеопотока	2 Система предоставляет работу с видеопотоком
3 Пользователь просматривает возможные варианты загрузки видеопотока	4 Система отображает настройку видеопотока
5 Пользователь загружает видеопоток	6 Система принимает видеопоток и загружает его в программу

Альтернатива

1. Если пользователь загружает видеопоток некорректного формата, будет выведено сообщение об ошибке.

В табл. 24 представлено краткое описание варианта «Просмотр отчётов».

Таблица 24

Краткое описание варианта использования «Просмотр отчётов»

Название	Просмотр отчётов
Цель	Просмотреть отчёт
Действующие лица	Пользователь
Краткое описание	Пользователь получает информацию об отчёте по гранулометрическому составу и прочностным характеристикам
Тип варианта	Основной

Типичный ход событий для данного варианта представлен в табл. 25.

Таблица 25

Типичный ход событий для варианта использования «Просмотр отчётов»

Действие исполнителя	Отклик системы
1 Пользователь обращается к просмотру отчётов	2 Система предоставляет работу с просмотром отчётов
3 Пользователь просматривает возможные варианты просмотра отчётов	4 Система отображает настройку отчёта
5 Пользователь настраивает отчёт	6 Система показывает настроенный отчёт

Альтернатива

2. Если пользователь просматривает отчёт по несуществующим данным, система отобразит сообщение, что за данный период не существует отчётов.

В табл. 26 представлено краткое описание варианта «Формирование результатов».

Таблица 26

Краткое описание варианта использования «Формирование результатов»

1	2
Название	Формирование результатов
Цель	Сформировать результат анализа

Продолжение табл. 26

1	2
Действующие лица	Пользователь
Краткое описание	Система формирует результат по гранулометрическому составу и прочностным характеристика
Тип варианта	Основной

Типичный ход событий для данного варианта представлен в табл. 27.

Таблица 27

Типичный ход событий для варианта использования «Формирование результатов»

Действие исполнителя	Отклик системы
1 Пользователь обращается к формированию результату	2 Система предоставляет настройку формирования анализа
3 Пользователь настраивает формирование результата	4 Система формирует результат

Альтернатива

3. Если при формировании результатов возникнет ошибка обработки данных, система выведет сообщение об этом и за данный период не будет сформировано результата.

В табл. 28 представлено краткое описание варианта «Управление анализом».

Таблица 28

Краткое описание варианта использования «Управление анализом»

1	2
Название	Управление анализом
Цель	Настроить анализ
Действующие лица	Пользователь

Продолжение табл. 28

1	2
Краткое описание	Система управляет анализом по гранулометрическому составу и прочностным характеристика
Тип варианта	Основной

Типичный ход событий для данного варианта представлен в табл. 29.

Таблица 29

Типичный ход событий для варианта использования «Управление анализом»

Действие исполнителя	Отклик системы
1 Пользователь обращается к управлению анализом	2 Система предоставляет настройку управления анализа
3 Пользователь настраивает управление анализом	4 Система запускает анализом

Альтернатива

4. Если при управлении анализом пользователь не запустит анализ, система перейдет в спящий режим.

3.3.2.2 Контекстная диаграмма классов

Диаграммы классов - центральное звено объектно-ориентированных методов разработки программного обеспечения, поэтому все существующие методы используют диаграммы классов в одной из известных нотаций. Однако в основном диаграммы классов в этих методах применяют на этапе проектирования, для того чтобы показать особенности построения конкретных классов [17].

Концептуальные модели в соответствии с определением оперируют понятиями предметной области, атрибутами этих понятий и отношениями между ними. Понятию в предметной области разрабатываемого

программного обеспечения могут соответствовать как материальные предметы, так и абстракции, которые применяют специалисты предметной области.

Основным понятиям в модели ставятся в соответствие классы. Класс при этом традиционно понимают, как совокупность общих признаков заданной группы объектов предметной области.

В соответствии с этим определением на диаграмме классов каждому классу соответствует группа объектов, общие признаки которых и фиксирует класс.

Контекстная диаграмма классов программного обеспечения прогнозирования прочностных характеристик кокса по его гранулометрическому составу представлена на рис. 51.

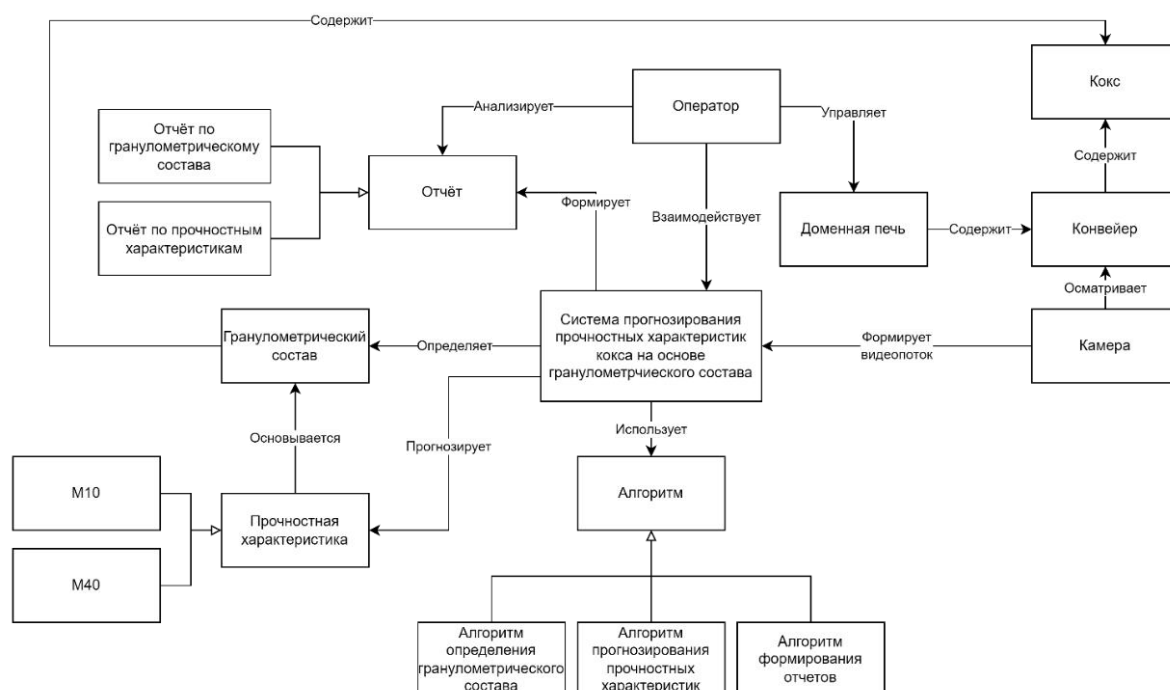


Рис. 51. Контекстная диаграмма классов

В данной диаграмме существуют следующие понятия, взятые из предметной области и их связи: «Оператор» осуществляет взаимодействие с «Системой прогнозирования прочностных характеристик кокса по его гранулометрическому составу», которая, в свою очередь использует

алгоритмы определения гранулометрического состава, прогнозирования прочностных характеристик и формирования отчётов. Для формирования видеопотока с конвейера будет использоваться камера.

Система прогнозирует прочностные характеристики (M10, M40) кокса, который движется по конвейеру доменной печи на основе определенного гранулометрического состава. После формирования отчётов по гранулометрическому составу и прочностным характеристикам, пользователь сможет их проанализировать.

Таким образом, можно сделать вывод о том, что система хорошо декомпозируется и использование объектно-ориентированного подхода приемлемо.

3.3.2.3 Диаграмма пакетов

Пакетом при объектном подходе называется совокупность описаний классов и других программных ресурсов, в том числе и самих пакетов. Объединение в пакеты используют для удобства создания больших проектов, количество классов в которых велико. При этом в один пакет обычно собирают классы и другие ресурсы одинакового назначения [17].

Диаграмма пакетов показывает, из каких частей состоит проектируемая программная система, и как эти части связаны друг с другом. Связь между пакетами определяется внешними связями классов и других ресурсов, объединенных в пакет.

Спецификация языка UML не налагает жёстких ограничений на то, как модель разбивается на пакеты, существует множество способов организации по функциональности, виду модели или по любому другому признаку. Допускается также иерархическое вкладывание одних UML-пакетов в другие, при этом вложенный пакет имеет полный доступ к содержимому своего контейнера и считается его частью [17].

Диаграмма пакетов серверной и клиентской частей представлена соответственно на рис. 52 и рис. 53. В табл. 30 представлено описание пакетов.

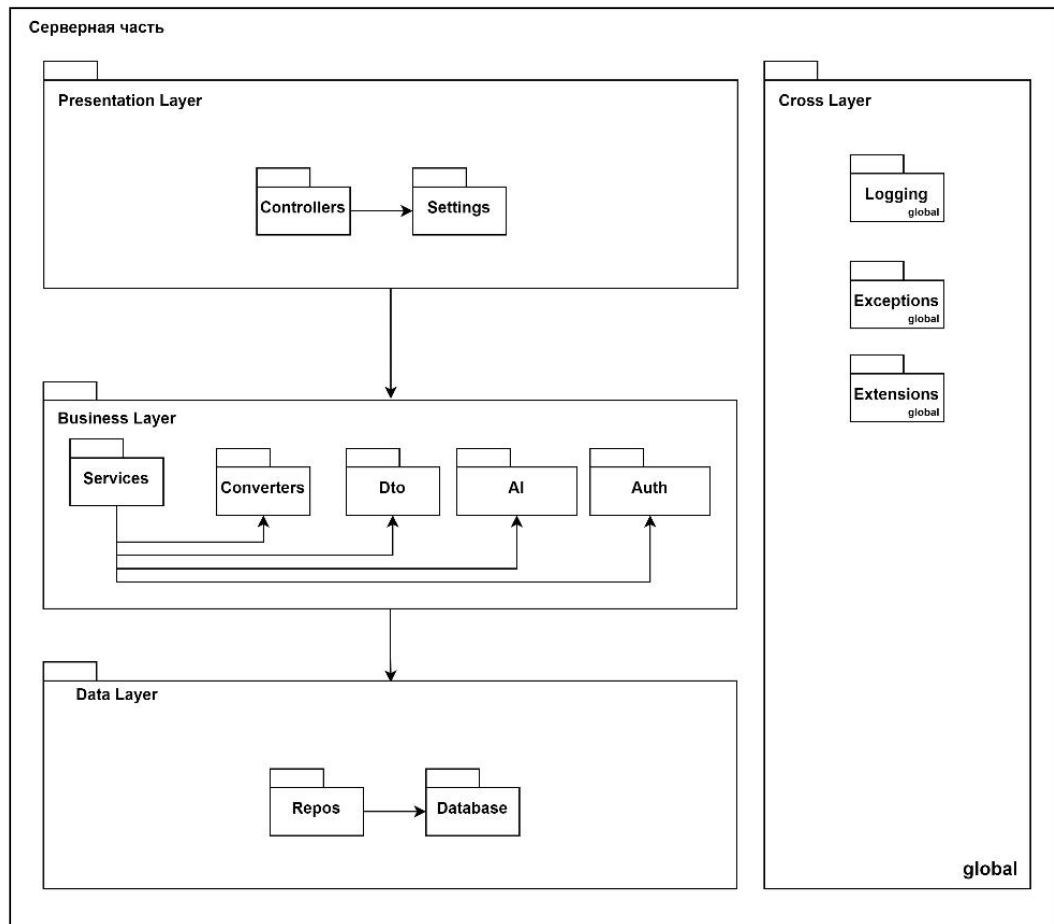


Рис. 52. Диаграмма пакетов серверной части

Серверная часть проектируется в соответствии с стандартами слоевой архитектуры. Данная архитектура содержит три основных слоя и один дополнительный.

Уровень представления (PL) — это самый верхний уровень трехуровневой архитектуры, и его основная роль заключается в отображении результатов пользователю, или, другими словами, в представлении данных, которые мы получаем из уровня бизнес-доступа, и предоставлении результатов интерфейсному пользователю.

Логический уровень (BAL) взаимодействует с уровнем доступа к данным и уровнем представления для обработки действий, которые приводят к

логическим решениям и оценкам. Основная задача этого уровня - обрабатывать данные между другими уровнями.

Основная функция уровня доступа к данным (DAL) заключается в доступе к данным из базы данных и их хранении, а обработка данных на уровне бизнес-доступа данные передаются на уровень представления по запросу пользователя.

Основная функция Сквозного уровня (CL) заключается в том, что он обеспечивает взаимодействие между различными слоями системы и позволяет им обмениваться информацией. Этот слой позволяет более эффективно и гибко управлять ресурсами и данными, обеспечивая более быструю обработку и передачу информации между слоями.

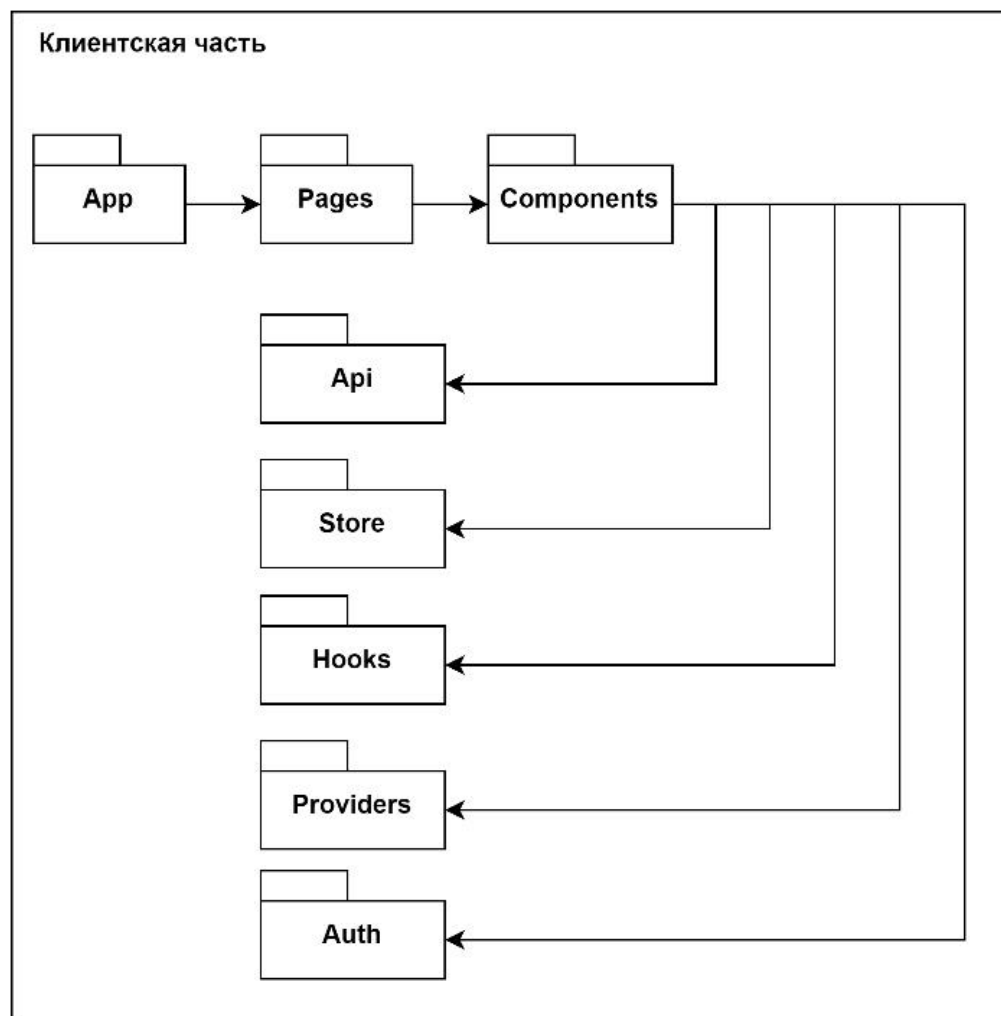


Рис. 53. Диаграмма пакетов клиентской части

Описание пакетов

Пакет	Описание
Серверная часть	
Controllers	Содержит набор классов для уточнения маршрутов и обработки HTTP-методов
Settings	Содержит набор классов, определяющих конфигурацию приложения
Services	Содержит набор классов, отвечающих за бизнес логику приложения
Converters	Содержит набор классов для работы с преобразованием объектов классов сущностей
Dto	Содержит набор классов для работы с транспортным уровнем
AI	Содержит набор классов для работы с нейронными сетями и математическими моделями
Auth	Содержит набор классов, отвечающих за аутентификацию
Repos	Содержит набор классов для абстрактной работы с таблицами базы данных
Database	Содержит набор классов для работы с базой данных
Logging	Содержит набор классов для легирования
Exceptions	Содержит набор классов для работы с исключениями
Extensions	Содержит набор классов, расширяющий базовый функционал стандартных конструкций
Клиентская часть	
App	Содержит набор классов для старта приложения
Pages	Содержит набор классов, отвечающих за маршруты имеющихся страниц
Components	Содержит набор классов универсальных компонентов приложения
Api	Содержит набор классов для работы с серверной частью приложения
Store	Содержит набор классов для работы с состоянием приложения
Hooks	Содержит набор классов «хуков»
Providers	Содержит набор классов «провайдеров»
Auth	Содержит набор классов, связанных с аутентификацией

3.3.2.3.1 Детальная диаграмма классов пакета «AI»

На рис. 54 отображена детальная диаграмма классов пакета AI серверной части, в табл. 31 представлено его описание.

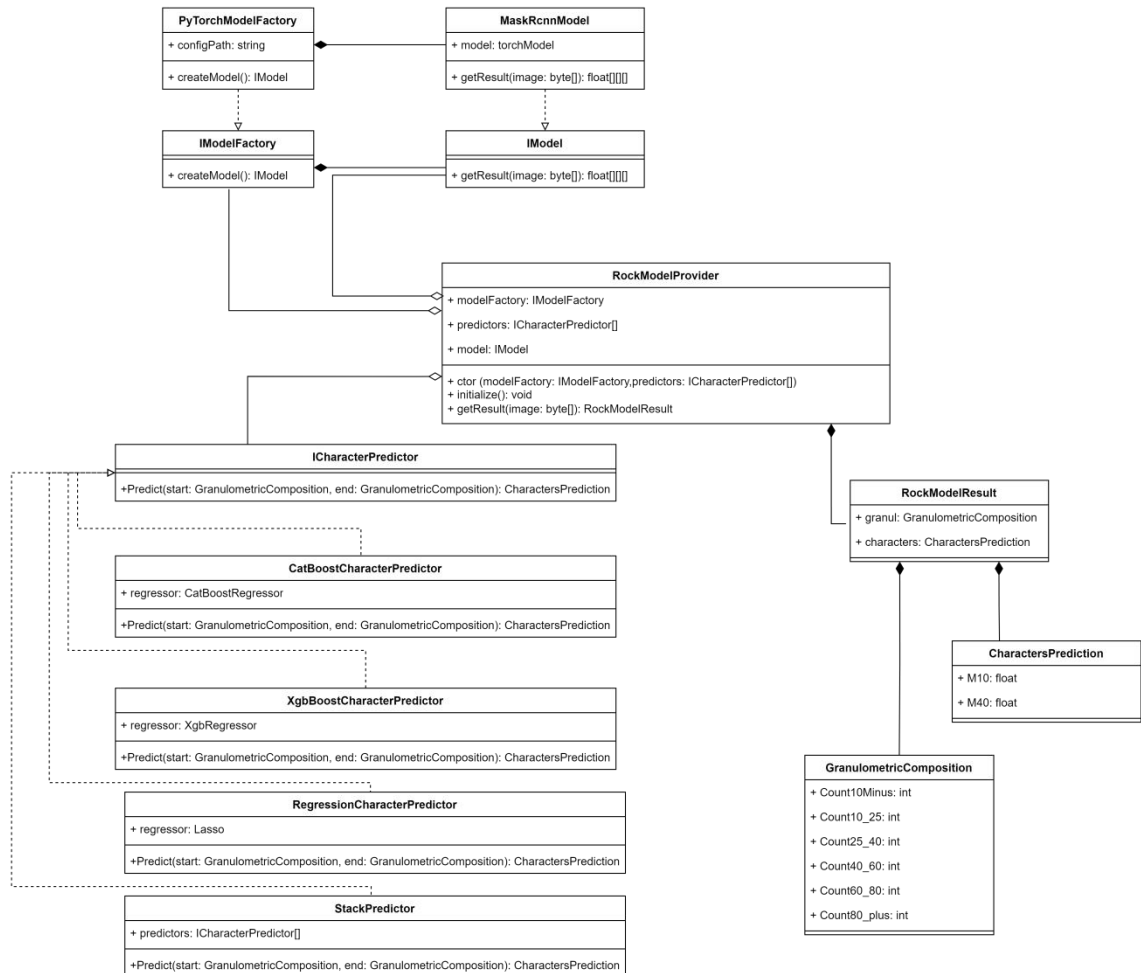


Рис. 54. Диаграмма классов пакета AI

Таблица 31

Описание классов пакета AI

Класс	Описание
1	2
PyTorchModelFactory	Фабрика создания модели с помощью фреймворка PyTorch
MaskRcnnModel	Модель Mask R-CNN
IModelFactory	Интерфейс фабрики создания модели
IModel	Интерфейс модели

Продолжение табл. 31

1	2
RockModelProvider	Компоновщик абстракций моделей, выполняющий обработку запроса по получению гранулометрического состава и прочностных характеристик с изображений
RockModelResult	Хранилище результата работы моделей
CharactersPrediction	Результат работы модели прогнозирования прочностных характеристик
GranulometricComposition	Результат работы модели определения гранулометрического состава
ICharacterPredictor	Интерфейс модели предсказания прочностных характеристик
CatBoostCharacterPredictor	Модель предсказания прочностных характеристик с помощью градиентного бустинга CatBoost
XgbBoostCharacterPredictor	Модель предсказания прочностных характеристик с помощью градиентного бустинга Xgb
RegressionCharacterPredictor	Модель предсказания прочностных характеристик с помощью линейной регрессии с методом нормализации Lasso
StackPredictor	Модель предсказания прочностных характеристик, основывающаяся на списке моделей

3.3.3 Разработка физической модели

3.3.3.1 Диаграмма компонентов

Диаграммы компонентов применяют при проектировании физической структуры разрабатываемого программного обеспечения. Эти диаграммы показывают, как выглядит программное обеспечение на физическом уровне, т.е. из каких частей оно состоит и как эти части связаны между собой [17]. Диаграммы компонентов оперируют понятиями компонент и зависимость. Под компонентами при этом понимают физические заменяемые части

программного обеспечения, которые соответствуют некоторому набору интерфейсов и обеспечивают их реализацию.

Ключевыми компонентами являются RocksClient.ts (клиентская часть), RockServer.exe (серверная часть), RockDetector.py (модель определения гранулометрического состава и прогнозирования прочностных характеристик). Для запуска клиентской части требуется серверный фреймворк Node.js и файл конфигурации config.ts. Для работы серверной части требуется фреймворк .NET 8 и файл конфигурации appsettings.json. Чтобы запустить модель необходимо подключить файл конфигурации нейронной сети Mask R-CNN config.pickle, файл весов нейронной сети weights.pth и файл модели машинного обучения прогнозирования прочностных характеристик stack_model.pickle.

Диаграмма компонентов системы представлена на рис. 55.

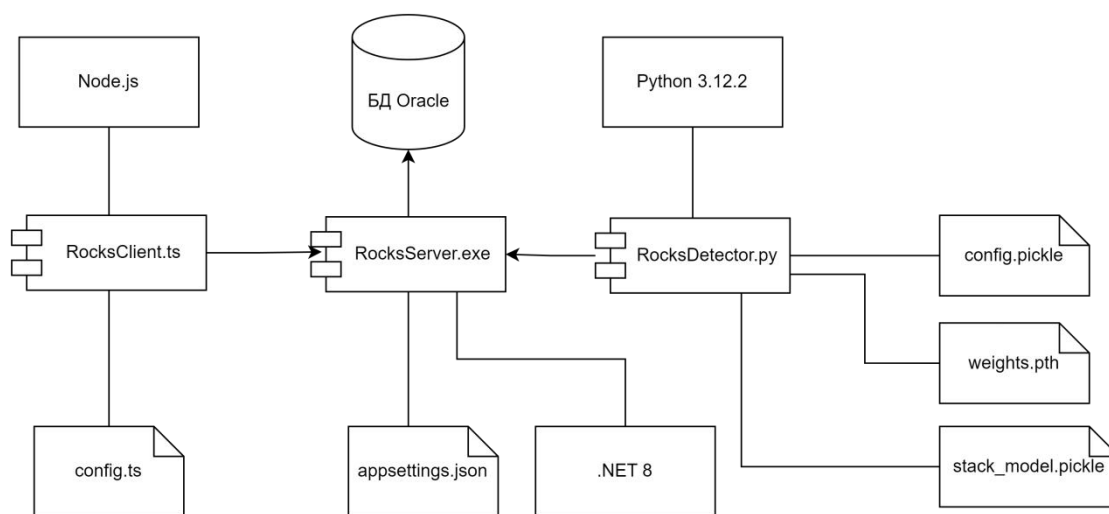


Рис. 55. Диаграмма компонентов

Краткие спецификации на разработанное приложение расположены в прил. 3. Текст программы расположен в прил. 2.

3.3.3.2 Диаграмма размещения

При физическом проектировании распределенных программных систем необходимо определить наиболее оптимальный вариант размещения программных компонентов на реальном оборудовании в локальной или глобальной сетях.

Для этого используют специальную модель UML - диаграмму размещения. Диаграмма размещения отражает физические взаимосвязи между программными и аппаратными компонентами системы [17].

Каждой части аппаратных средств системы, например, компьютеру или датчику, на диаграмме размещения соответствует узел. Диаграмма размещения проектируемой системы представлена на рис. 56.

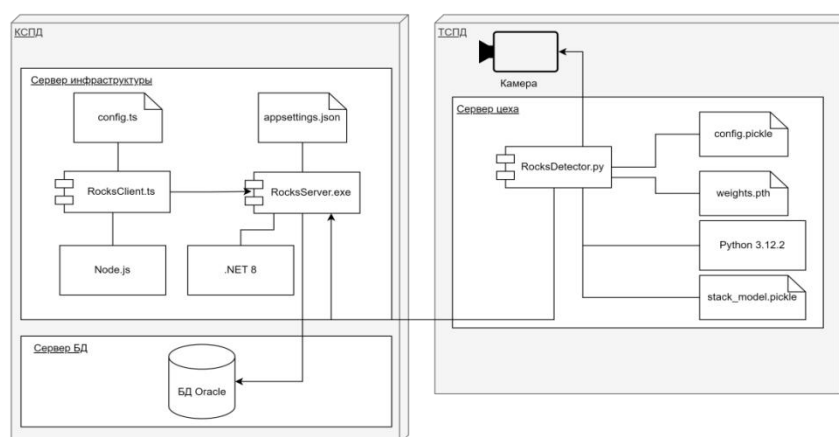


Рис. 56. Диаграмма размещения

На данной диаграмме имеются три сервера:

- сервер цеха, расположенный в технологическом сегменте сети, отвечающий за работу математической модели прогнозирования и определения;
- сервер инфраструктуры, расположенный в корпоративном сегменте сети, отвечающий за предоставления функционала просмотра и редактирования для пользователя
- сервер базы данных, расположенный в корпоративном сегменте сети, отвечающий за хранение истории измерений.

3.3.4 Разработка интерфейсов

3.3.4.1 Построение графа диалога

Граф диалога – ориентированный взвешенный граф, каждой вершине которого сопоставлена конкретная картина на экране (кадр) или определенное состояние диалога, характеризующееся набором доступных пользователю действий.

Дуги, исходящие из вершин, показывают возможные изменения состояний при выполнении пользователем указанных действий. Дуги, исходящие из вершин, показывают возможные изменения состояний при выполнении пользователем указанных действий. В качестве весов дуг указывают условия переходов из состояния в состояние и операции, выполняемые во время перехода. Под диалогом понимают регламентированный обмен информацией между человеком и программным обеспечением, осуществляемый в реальном масштабе времени и направленный на совместное решение конкретных задач.

Каждый диалог состоит из отдельных процессов ввода-вывода, которые физически обеспечивают связь между пользователем программного обеспечения и компьютером. Обмен информацией между пользователем и компьютером осуществляется с помощью передачи сообщений и управляющих сигналов [17].

Сообщение — это определенная порция информации в диалоговом обмене человека и компьютера. Пользователь в основном генерирует сообщения следующих типов: запрос информации, запрос операции или функции, ввод или изменение информации, выбор зоны и т. д.

В ответ он получает: подсказки или справки; информационные сообщения, не требующие ответа; приказы, требующие действий; сообщения об ошибках, нуждающиеся в ответных действиях; изменение формата кадра и т. д. Так как работа с программой имеет четкую последовательность

действий, то тип диалога пользователя с программой является «Управляемый программой».

Граф диалога программы представлен на рис. 57.

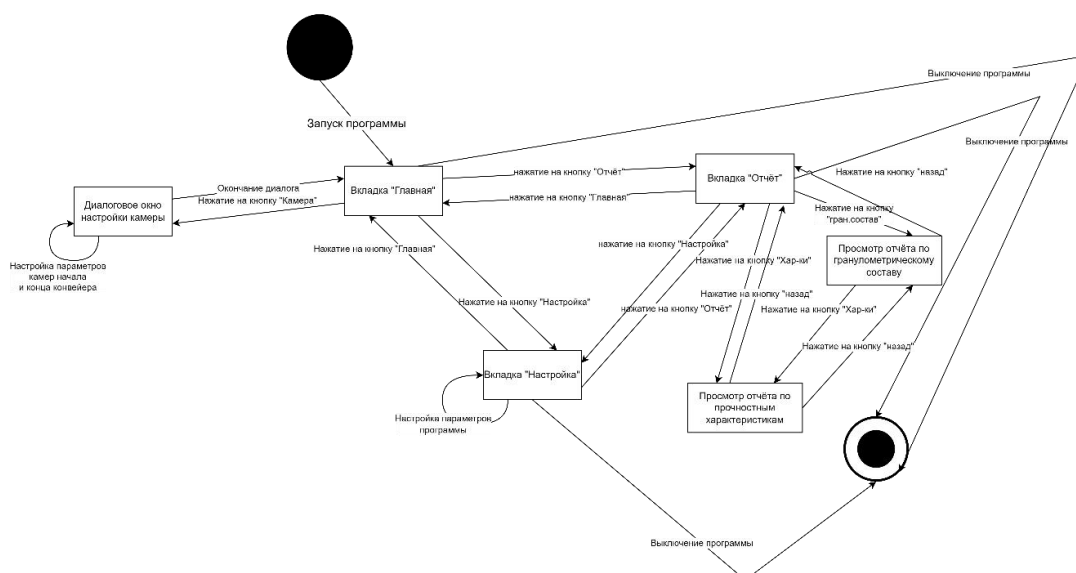


Рис. 57. Граф диалога

3.3.4.2 Разработка форм ввода вывода информации

Интерфейс разрабатываемого программного обеспечения является интерфейсом со свободной навигацией, который предназначен для анализа работы системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу. Для входа в систему предусмотрена форма авторизации, отображенная на рис. 58.

Вход

ВОЙТИ

РЕГИСТРАЦИЯ

Рис. 58. Вход в систему

После успешного входа пользователь попадает на главную страницу приложения (рис. 59). На ней расположены переходы между вкладками, возможность остановки системы, а также результат работы системы в реальном времени. В левой верхней части расположена информация о конвейере К19, а в правой - КД4. В нижней части расположены результаты прогнозирования прочностных характеристик, опираясь на которые оператор будет управлять доменной печью.

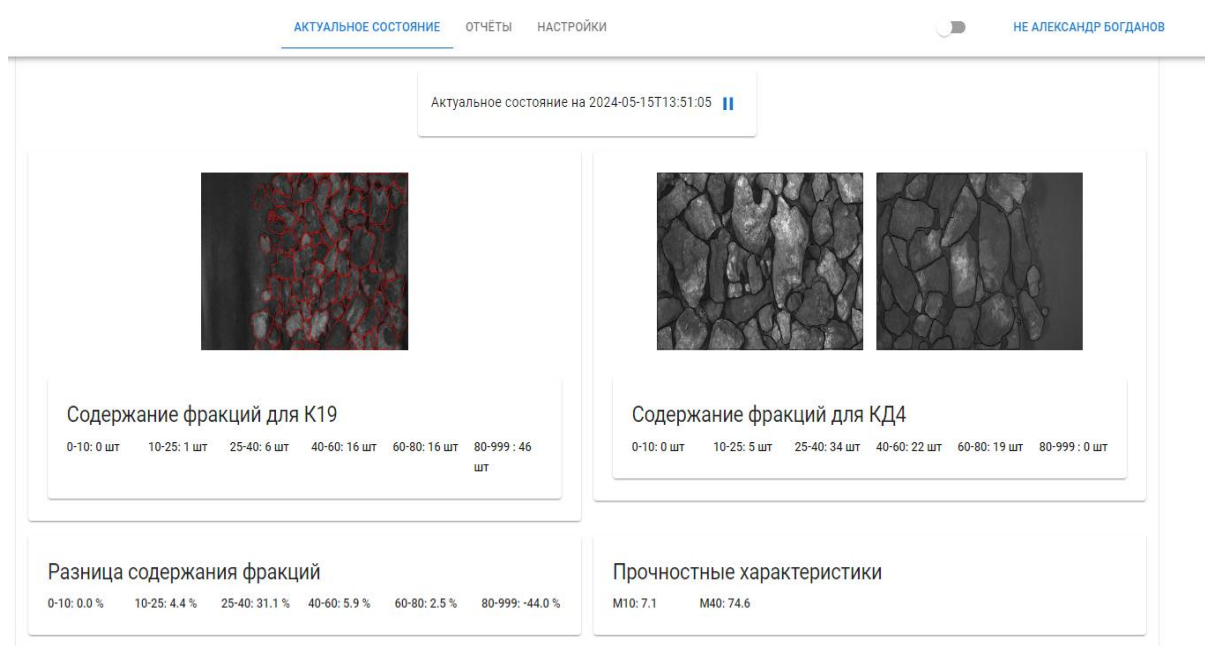


Рис. 59. Главная страница приложения

При переходе на вкладку «Отчеты» пользователь перейдет на страницу просмотра исторической информации (рис. 60). На данной странице расположен список произведенных измерений, в котором можно перейти к конкретному отчёту, нажав на его дату.

АКТУАЛЬНОЕ СОСТОЯНИЕ		ОТЧЕТЫ	НАСТРОЙКИ		НЕ АЛЕКСАНДР БОГДАНОВ		
Дата		0-10	10-25	25-40	40-60	60-80	80-999
	2024-05-15T13:50:31	0.0 %	9.3 %	16.2 %	9.1 %	3.3 %	-37.8 %
	2024-05-15T13:51:05	0.0 %	4.4 %	31.1 %	5.9 %	2.5 %	-44.0 %
	2024-05-15T13:52:14	-1.0 %	14.6 %	15.8 %	-2.4 %	-9.0 %	-18.0 %
	2024-05-15T13:52:48	-1.0 %	7.5 %	2.9 %	8.7 %	5.4 %	-23.6 %
	2024-05-15T13:53:22	0.0 %	3.9 %	21.2 %	11.5 %	-0.2 %	-36.4 %
	2024-05-15T13:53:56	0.2 %	7.0 %	13.2 %	8.9 %	0.4 %	-29.7 %
	2024-05-15T13:54:30	0.0 %	2.0 %	11.0 %	21.3 %	0.0 %	-34.3 %
	2024-05-15T13:56:13	-2.0 %	6.7 %	15.4 %	10.9 %	-8.6 %	-22.3 %
	2024-05-15T13:56:47	-1.0 %	12.5 %	15.2 %	7.2 %	-3.2 %	-30.7 %

Рис. 60. Страница «Отчеты»

При переходе к конкретному отчету (рис. 61), откроется страница аналогичная главной, то есть она будет содержать информацию о произведенном определении гранулометрического состава и прогнозирования прочностных характеристик.

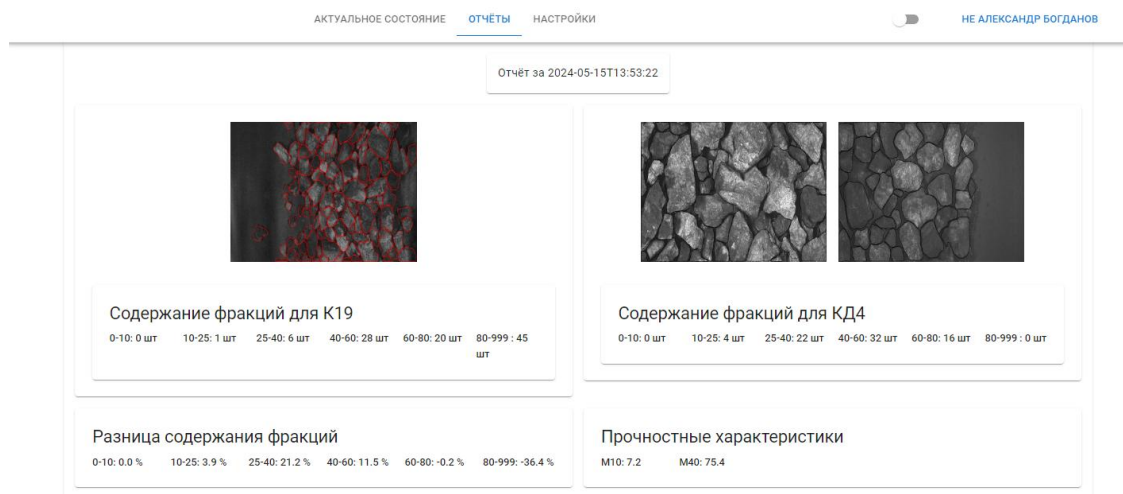


Рис. 61. Открытый отчет

При открытии вкладки настройки, пользователю предоставляется возможность редактирования настроек приложения (рис. 62). Для данной системы предоставляется возможность, настройки адресов для камер. Руководство пользователя расположено в прил. 4.

Рис. 62. Страница «Настройки»

Следующим шагом является экспериментальная проверка предложенных технических решений.

3.4 Выводы по разделу 3

В данном разделе выполнены все этапы проектирования разрабатываемого программного обеспечения на основе объектно-ориентированного подхода к анализу и проектированию систем. В качестве средств реализации использованы языки высокого уровня программирования C#, Python и TypeScript.

На этапе концептуального моделирования были разработаны модели «черного» и «белого» ящика, состав и структура системы. На этапе проектирования был произведен выбор подхода к разработке и модели жизненного цикла, а также используемых инструментов. С помощью языка моделирования UML были разработаны: диаграмма вариантов использования, контекстная диаграмма классов, диаграмма пакетов, диаграммы классов пакетов, диаграммы компонентов и диаграмма размещения.

В ходе разработки интерфейса был описан граф диалога пользователя и все формы ввода и вывода информации для программного обеспечения прогнозирования прочностных характеристик кокса по его гранулометрическому составу.

4 Экспериментальная проверка предложенных технических решений прогнозирования прочностных характеристик кокса по его гранулометрическому составу

4.1 Описание эксперимента и экспериментальных данных

Для проверки правильности прогнозирования прочностных характеристик необходимо проверить правильность работу составляющих подсистем: подсистему определения гранулометрического состава и подсистему прогнозирования прочностных характеристик.

Для подсистемы определения гранулометрического состава необходимо выполнить следующие действия:

- сформировать набор изображений с камеры не участвовавших в обучении модели за определённый период;
- разметить тестовые изображения и рассчитать класс крупности
- относительно тестовой выборки изображений рассчитать критерии AP с различными зонами перекрытия IoU.
- относительно тестовой выборки рассчитать класс крупности и критерии F1 и Accuracy
- точность определения класса крупности должна составлять не менее 90%.

Для того, чтобы адекватно оценить модель определения гранулометрического состава, которая обнаруживает гранулы кокса, необходимо применить расчет критерия Average precision для определения точности сегментации и определения границ.

Общее определение средней точности (AP) — это нахождение области под кривой точности-отзыва. mAP (средняя точность) — это среднее значение AP. В некоторых контекстах AP рассчитывается для каждого класса и усредняется для получения MAP. Но в других они означают то же самое. Средняя оценка точности (mAP) рассчитывается путем взятия среднего AP

по всем классам и / или общим пороговым значениям IoU, в зависимости от различных существующих проблем обнаружения.

IoU (Intersection over Union / Пересечение над объединением). Эта метрика равняется соотношению площади пересекающихся областей к площади областей объединенных.

F1 критерий рассчитывается как среднее гармоническое между Precision и Recall.

Вычисление этих метрик осуществляется на основе величин:

TP — количество объектов, классифицированных правильно;

TN — количество объектов, правильно отнесённых к другому классу;

FP — количество объектов другого класса, которые были соотнесены к рассматриваемому;

FN — количество объектов, которые не были отнесены к рассматриваемому классу.

В случае многоклассового классификатора строится матрица ошибок M (confusion matrix) размером (n, n) , где n - количество классов объектов. По горизонтали располагаются распознанные сетью классы, а по вертикали истинные (рис. 63).

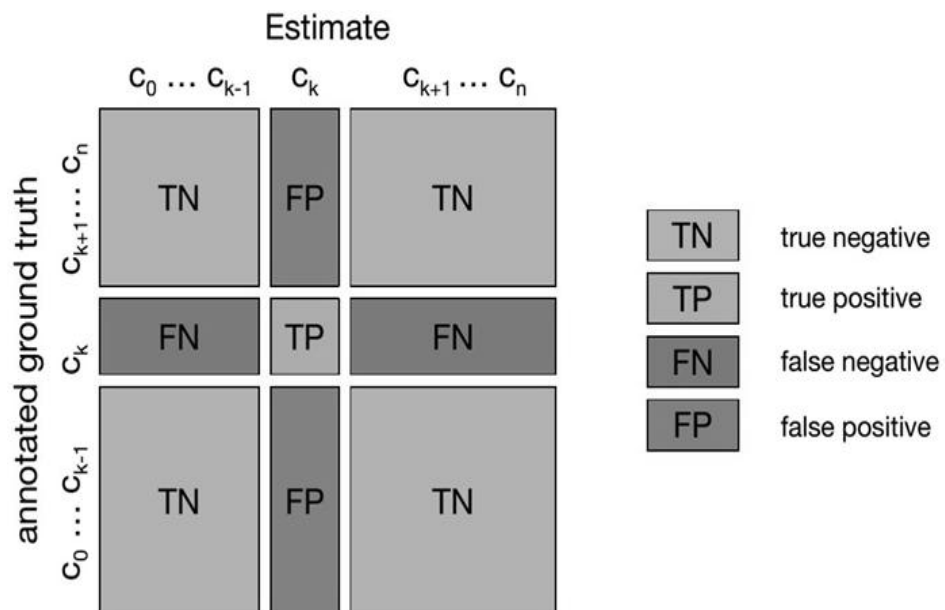


Рис. 63. Определение базовых метрик по матрице ошибок

Значение метрики точности тем меньше, чем больше появляется ложных детекций рассматриваемых классов. Тогда, точность, которая показывает долю верно определённых объектов среди всех распознанных, на основе этих обозначений можно записать следующим образом:

$$Precision = \frac{TP}{TP + FP}, \quad (16)$$

$$Recall = \frac{TP}{TP + FN}, \quad (17)$$

Для проверки качества определения класса размерности использовался критерий Ассигасу – доля правильных ответов алгоритма:

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (18)$$

Для подсистемы прогнозирования прочностных характеристик необходимо выполнить следующие действия:

- сформировать набор данных по гранулометрическому составу;
- сформировать набор данных по расчёту прочностных характеристик из лаборатории;
- относительно тестовой выборки рассчитать критерии MSE, MAE, R2 и MAPE
- точность прогнозирования прочностных характеристик должна составлять не менее 80%.

Для тестирования линейной регрессии, предсказывающей параметры прочности кокса, необходимо провести расчет критериев MSE, MAE.

Среднеквадратичная ошибка (Mean Squared Error) применяется в случаях, когда требуется подчеркнуть большие ошибки и выбрать модель, которая

дает меньше именно больших ошибок. Большие значения ошибок становятся заметнее за счет квадратичной зависимости.

$$MSE = \left(\frac{1}{n}\right) \sum (y_i - \hat{y}_i)^2, \quad (19)$$

где

n – количество наблюдений, по которым строится модель и количество прогнозов,

y_i – фактические значение зависимой переменной для i -го наблюдения,

$\hat{y}_i$ – значение зависимой переменной, предсказанное моделью.

MAE выражает среднюю абсолютную ошибку между фактическими и предсказанными значениями модели.

$$MAE = \left(\frac{1}{n}\right) \sum |y_i - \hat{y}_i|, \quad (20)$$

где

n – количество наблюдений, по которым строится модель и количество прогнозов,

y_i – фактические значение зависимой переменной для i -го наблюдения,

$\hat{y}_i$ – значение зависимой переменной, предсказанное моделью.

В наборе данных для определения гранулометрического состава использовались изображения в формате «.bmp» (информация о количестве изображений находится в п.2.2).

В наборе данных для прогнозирования прочностных характеристик использовалось два набора данных (информация о количестве измерений находится в п.2.2). Первый набор содержит измерения гранулометрического состава с полями: GRAN_0_10, GRAN_10_25, GRAN_25_40, GRAN_40_60, GRAN_60_80, GRAN_80_999 и DATE (Дата измерения). Второй набор

данных содержит лабораторные измерения прочностных характеристик с полями: M10, M40, DATE (Дата).

4.2 Разработка методики настройки программного обеспечения

Для проведения экспериментальных исследований, как обучения, так и получения результатов работы модели, необходимо использовать ПК, удовлетворяющий минимальным системным требованиям, указанным в прил. 1.

Для проведения проверки, необходимо установить разработанное программное обеспечение на компьютер, подключенный к корпоративной сети передачи данных, который будет связываться с сервером цеха из технологической сети (рис. 64).

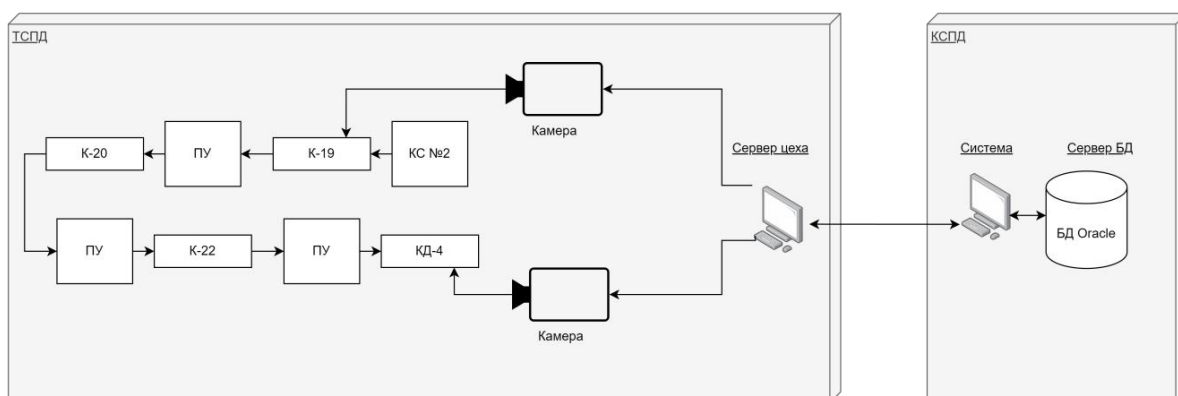


Рис. 64. Структурно-функциональная схема эксперимента

4.3 Результаты экспериментальной проверки модели определения гранулометрического состава

Данный этап тестирования заключается в определении гранулометрического состава кокса для каждого тестового изображения, а также вычисления класса фракции для каждого куска, с последующим сравнением с исходными данными и анализом метрик.

На рис. 65 - 67 отображено три примера изображений сегментации кусков кокса с различной загрузкой конвейера. При анализе можно сделать вывод о

том, что нейронная сеть обнаруживает даже те гранулы, которые не были отмечены, что исправляет неполную разметку данных.

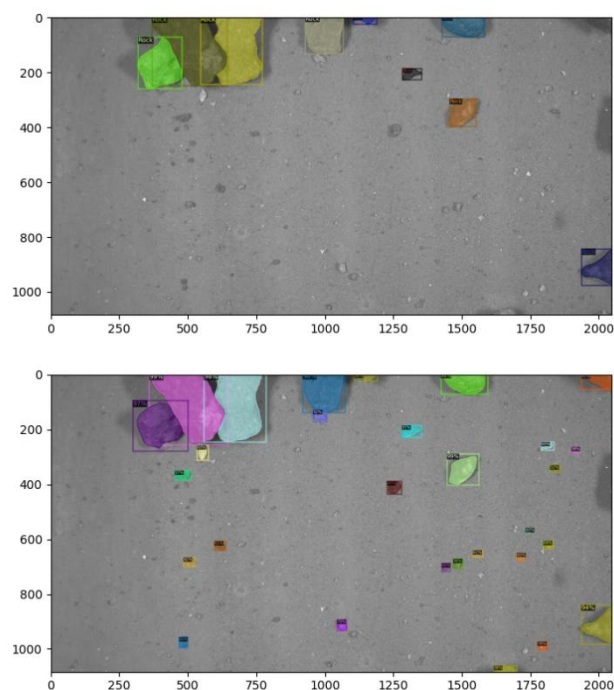


Рис. 65. Сравнение модели сегментации с разметкой с низкой загруженности конвейера

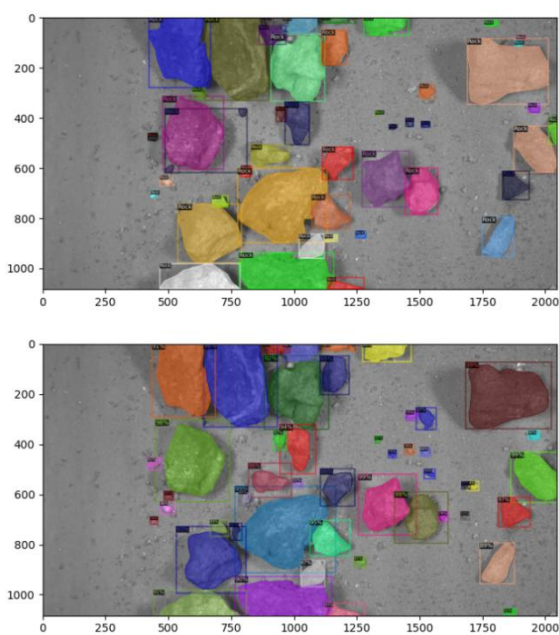


Рис. 66. Сравнение модели сегментации с разметкой с частичной загруженности конвейера

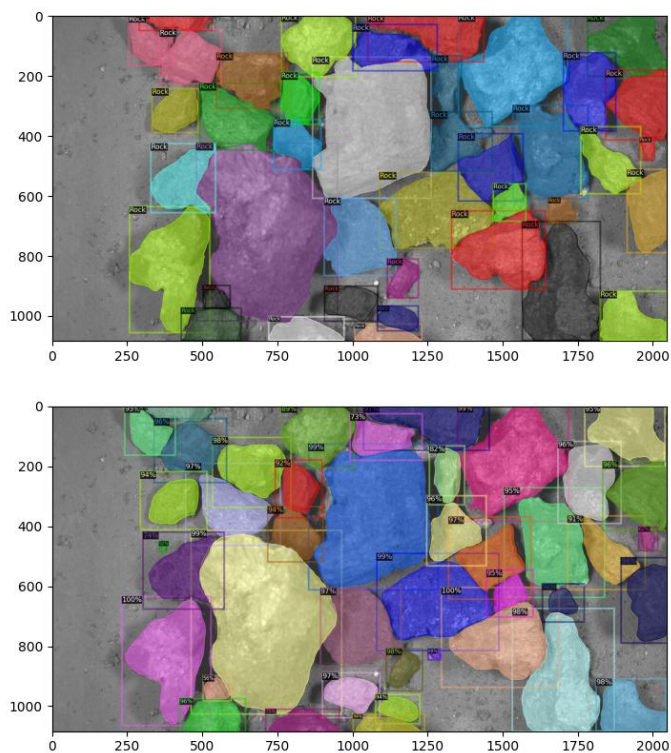


Рис. 67. Сравнение модели сегментации с разметкой с полной загруженностью конвейера

Результаты тестирования модели определения гранулометрического состава представлены в табл. 32.

Таблица 32

Проверка модели

Критерий	Значение
AP100	0.621
AP75	0.685
AP50	0.756
F1	0.944
Accuracy	0.961

Анализируя полученные метрики, можно сделать вывод о том, что точность системы определения гранулометрического состава более 93 %, что соответствует поставленным требованиям.

4.4 Результаты экспериментальной проверки модели прогнозирования прочностных характеристик

Данный этап тестирования заключается в прогнозировании прочностных характеристик кокса для каждого лабораторного измерения с последующим сравнением с исходными данными и анализом метрик.

В табл. 33 представлено сравнение между фактическим и расчетным значением параметра M10.

Таблица 33

Данные по пробам фактического и расчетного значения параметра M10

Номер пробы	M10 фактически	M10 расчет
1	2	3
1	7,8	7,892325046
2	8,2	7,917731425
3	8,4	8,252899414
4	8,2	8,033121402
5	7,4	7,345852994
6	7,6	7,64484553
7	7,4	7,541600069
8	8	8,082817313
9	7,5	7,613303496
10	7,4	7,405127655
11	7,2	7,619705913
12	8	8,064040308
13	7,2	7,184275765
14	7	7,003562501
15	7	7,313161154
16	7,8	7,648376102
17	8	8,029375875
18	7,2	7,201240338
19	8	8,043759059
20	7,6	7,638816541

Продолжение табл. 33

1	2	3
21	7,2	7,161673438
22	7,4	7,492450098
23	7	7,056427748
24	6,6	7,466303705
25	7,2	7,190390223
26	7,2	7,038913531
27	6,8	6,789778665
28	7,2	7,150627482
29	7,2	7,206915868
30	6,6	7,162877336
31	7,2	7,204673468
32	6,6	6,533939146
33	6,4	7,290241953
34	7,2	7,120118314
35	6,6	6,911067237
36	7	6,979453708
37	7,4	7,413089258
38	7	6,967865989
39	7,2	7,196310279
40	7,4	7,338304297

На рис. 68 отображен график с минимальным допуском, максимальным допуском и расчетным значением параметра $M10$. На данном этапе тестирования можно сказать, что модель соблюдает допустимые рамки погрешности.

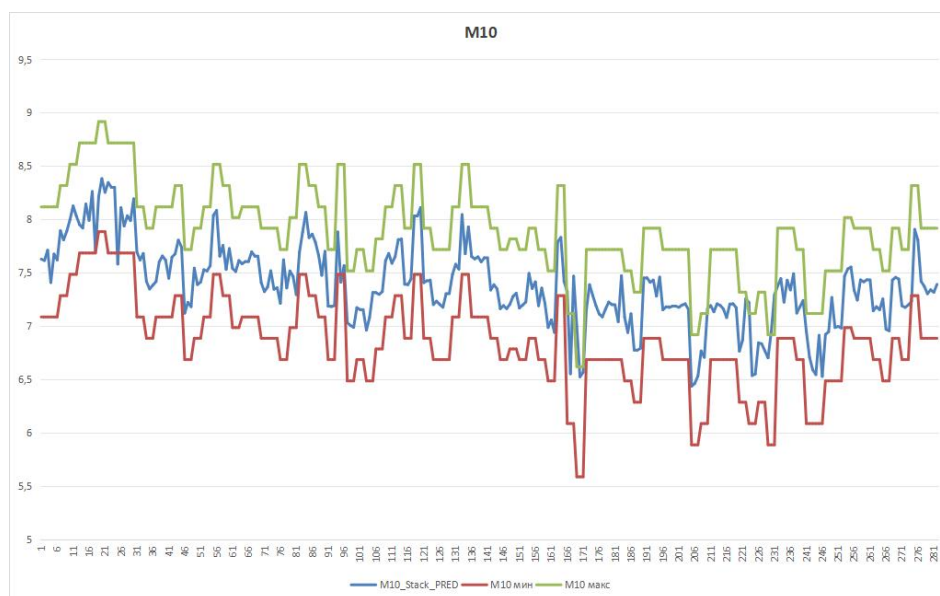


Рис. 68. Сравнение расчетного M10 с фактическим

Результаты тестирования модели прогнозирования параметра M10 представлены в табл. 34.

Таблица 34

Проверка модели

Критерий	Значение
MAE	0,097514732
MSE	0,031111689

Анализируя полученные метрики, можно сделать вывод о том, что точность системы прогнозирования M10 более 87 %, что соответствует поставленным требованиям.

В табл. 35 представлено сравнение между фактическим и расчетным значением параметра M40.

Таблица 35

Данные по пробам фактического и расчетного значения параметра M40

Номер пробы	M40 фактически	M40 расчет
1	2	3
1	75,8	75,6811855

Продолжение табл. 35

1	2	3
2	71	75,69903803
3	74	74,32411432
4	72,6	73,22777873
5	74,6	74,61146266
6	76,4	75,96046272
7	74,6	74,18671296
8	74,6	74,56489906
9	74,4	73,63479986
10	74,4	74,17988092
11	75,4	73,80482555
12	72,6	72,91950034
13	76	75,70312594
14	74,6	74,3875894
15	75,8	74,934208
16	75,7	74,56350354
17	73,6	73,61993902
18	73,4	73,62962238
19	71,6	71,71608527
20	73,8	73,52919729
21	76	76,30383449
22	75,6	72,91701969
23	76,4	75,4859773
24	78,5	75,95117659
25	75,8	75,97728131
26	73,9	73,9759548
27	73,6	73,51507368
28	74,3	73,96306071
29	74,4	74,75658135
30	74,8	76,53204908
31	77,8	77,54351434

1	2	3
32	76,2	76,60102248
33	75,9	75,84080389
34	74,8	74,77860966
35	75,7	76,12873352
36	76,6	76,71797406
37	75,2	75,26288154
38	75,8	76,14950183
39	76,8	76,6497028
40	76,2	76,51235645

На рис. 69 отображен график с минимальным допуском, максимальным допуском и расчетным значением параметра М40. На данном этапе тестирования можно сказать, что модель соблюдает допустимые рамки погрешности.

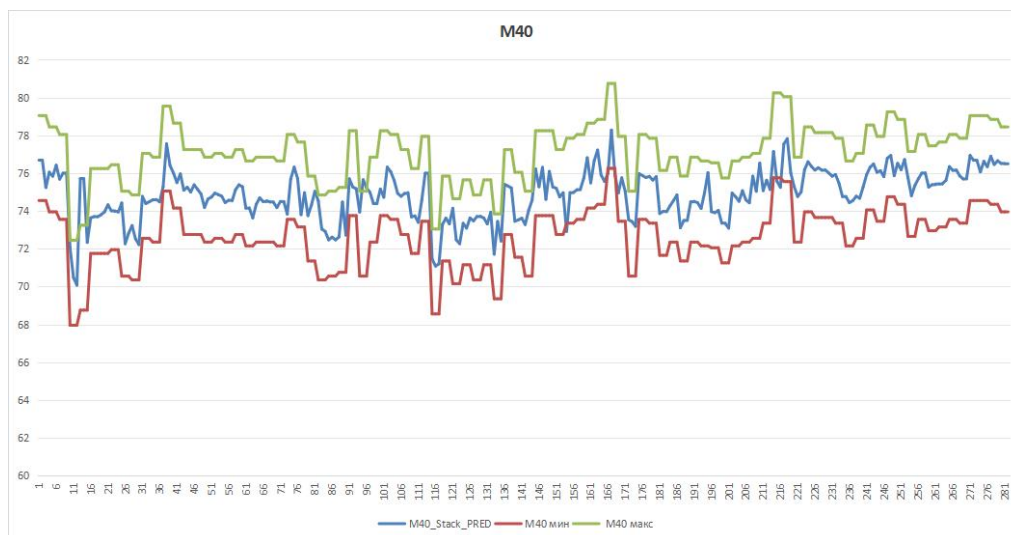


Рис. 69. Сравнение расчетного М40 с фактическим

Результаты тестирования модели прогнозирования параметра М40 представлены в табл. 36.

Проверка модели

Критерий	Значение
MAE	0,486114752
MSE	0,654777543

Анализируя полученные метрики, можно сделать вывод о том, что точность системы прогнозирования M40 более 84 %, что соответствует поставленным требованиям.

4.5 Тестирование программного обеспечения

Выбор стратегии тестирования, разработка тестов, программа и методика испытаний.

Объект и цель испытаний.

Объектом испытаний является программное обеспечение прогнозирования прочностных характеристик кокса по его гранулометрическому составу.

Цели испытаний:

- нахождение ошибок в программе;
- проверка правильности работы отдельных функций программы;
- проверка соответствия, разрабатываемого ПО требованиям, заявленным в техническом задании.

Требования к программному обеспечению.

Требования к программному обеспечению представлены в прил. 1.

Требования к функциональным характеристикам программного обеспечения.

У программного обеспечения следующие требования к функциональным характеристикам:

- настройка пользователем камер хвоста и головы конвейера;

- просмотр отчетов по гранулометрическому составу и прочностным характеристикам;
- расчет моделей на основе полученных данных с указанной периодичностью;
- запись результатов работы программы в лог-файл;
- возможность остановки и запуска анализа.

Требования к надежности

У программного обеспечения следующие требования к надежности:

- система должна иметь защиту от некорректных действий оператора и ошибочных исходных данных;
- система не должна во время работы модифицировать свой код или коды других программ;
- при удалении и передаче зафиксированных данных из базы данных система должна запрашивать подтверждения действия;
- логгирование.

Требования к составу и параметрам технических средств.

Требования к составу и параметрам технических средств представлены в прил. 1.

Требования к программной документации.

Программная документация должна включать в себя следующие элементы:

1. Расчетно-пояснительная записка;
2. Устав проекта;
3. Руководство пользователя.

Документация оформляется на листах формата А4 по действующим стандартам на создание документации к программному обеспечению (ЕСПД).

Состав и порядок испытаний.

Тестирование разработанного ПО проводилось согласно следующего состава мероприятий по тестированию:

- тестирование разработанных модулей;
- тестирование программы на соответствие функциональных требований, изложенных в прил. 1;
- тестирование надежности программы на соответствие требований к надежности, изложенных в прил. 1;
- проверка программной документации на соответствие требований ЕСПД.

Тестирование ПО проводилось на персональном компьютере, имеющем следующие характеристики:

- процессор Intel Core I5 12400f (6 ядер, 12 потоков, тактовая частота 3.5 ГГц);
- видеокарта Nvidia RTX 3070;
- объем оперативной памяти 32 Гб;
- твердотельный накопитель объемом 1024 Гб;
- операционная система Windows 10 Домашняя;
- монитор с разрешением 3440x1440.

Для тестирования программной документации использовался метод ручного контроля. Для тестирования программного обеспечения использовались два метода: функциональное тестирование и системное тестирование.

Для системного тестирования применялся метод ручного контроля. Ручной контроль обычно используется на ранних этапах разработки, так как с его помощью можно находить от 30 до 70 % ошибок логического проектирования и кодирования. Исходными данными для таких проверок являются: техническое задание, спецификации, структурная и функциональная схемы программного продукта, схемы отдельных компонентов.

Из методов ручного контроля был выбран метод проверки за столом. Этот метод не требует наличия группы специалистов. Проверка исходного

текста проводится одним человеком, который читает текст программы и проверяет его на наличие возможных ошибок по списку наиболее часто встречающихся ошибок.

Для тестирования модулей был выбран один из методов функционального тестирования – метод граничных значений. Исходными данными для этого метода являются: программные модули, спецификации на компоненты, правильные и неправильные данные для модулей.

Результаты проведения испытаний

В результате проверки программной документации методом ручного контроля были исправлены найденные грамматические и пунктуационные ошибки, было проверено соответствие документации требованиям ЕСПД. Также были проверены построенные схемы и диаграммы на соответствие ГОСТ.

Результаты функционального тестирования компонентов программного обеспечения представлены в табл. 37.

Таблица 37

Результаты тестирования компонентов программного обеспечения

Дата	Тестируемый пакет	Описание	Кто проводил	Способ	Результат
1	2	3	4	5	6
Серверная часть					
15.05.2024	Data Layer: Database	Правильное подключение к БД	Разработчик	Функциональное	Успех
15.05.2024	Data Layer: Repos	Корректная конвертация в сущности	Разработчик	Функциональное	Успех
15.05.2024	Business Layer: Services	Корректный доступ к репозиториям и конвертация в Dto	Разработчик	Функциональное	Успех
15.05.2024	Business Layer: Dto	Полнота классов с корректным доступом к информации	Разработчик	Функциональное	Успех

Продолжение табл. 37

1	2	3	4	5	6
15.05.2024	Business Layer: AI	Правильность работы модели определения гранулометрического состава	Разработчик	Функциональное	Успех
15.05.2024	Business Layer: AI	Правильность работы модели прогнозирования прочностных характеристик	Разработчик	Функциональное	Успех
15.05.2024	Business Layer: Auth	Аутентификация пользователя	Разработчик	Функциональное	Успех
15.05.2024	PresentationLayer: Controllers	Правильный доступ к методам по маршрутам	Разработчик	Функциональное	Успех
15.05.2024	PresentationLayer: Controllers	Конвертирование данных в JSON	Разработчик	Функциональное	Успех
15.05.2024	PresentationLayer: Settings	Настройка запуска	Разработчик	Функциональное	Успех
15.05.2024	Cross Layer: Logging	Логгирование в консоль	Разработчик	Функциональное	Успех
15.05.2024	Cross Layer: Logging	Логгирование в файл	Разработчик	Функциональное	Успех
15.05.2024	Cross Layer: Exception	Корректная передача информации	Разработчик	Функциональное	Успех
Клиентская часть					
16.05.2024	App	Запуск приложения	Разработчик	Функциональное	Успех
16.05.2024	Pages	Переход по страницам согласно маршрутам	Разработчик	Функциональное	Успех
16.05.2024	Components	Отрисовка отдельных частей страниц	Разработчик	Функциональное	Успех
16.05.2024	API	Обращение к серверной части	Разработчик	Функциональное	Успех

Продолжение табл. 37

1	2	3	4	5	6
16.05.2024	Store	Сохранение состояния приложения при переходе между страницами	Разработчик	Функциональное	Успех
16.05.2024	Auth	Аутентификация пользователя	Разработчик	Функциональное	Успех

После функционального тестирования, было проведено тестирование всей системы в целом на выполнение требований (прил. 1), т.е. было проведено системное тестирование.

Было проведено тестирование разработанного ПО на выполнение требований к функциональным характеристикам. Результаты данного тестирования представлены в табл. 38.

Таблица 38

Тестирование разработанного ПО на выполнение требований к функциональным характеристикам

Дата	Тестируемое требование	Кто проводил	Способ	Результат
17.05.2024	настройка пользователем камер хвоста и головы конвейера	Тестировщик	Системное тестирование	Успех
17.05.2024	просмотр отчетов по гранулометрическому составу и прочностным характеристикам	Тестировщик	Системное тестирование	Успех
17.05.2024	расчет моделей на основе полученных данных с указанной периодичностью	Тестировщик	Системное тестирование	Успех
17.05.2024	запись результатов работы программы в лог-файл	Тестировщик	Системное тестирование	Успех
17.05.2024	возможность остановки и запуска анализа	Тестировщик	Системное тестирование	Успех

Следующим было проведено тестирование разработанного ПО на выполнение требований к надежности. Результаты данного тестирования представлены в табл. 39.

Таблица 39

Тестирование разработанного ПО на выполнение требований к
надежности

Дата	Тестируемое требование из ТЗ	Кто проводил	Способ	Результат
18.05.2024	Система должна иметь защиту от некорректных действий оператора и ошибочных исходных данных	Тестировщик	Системное тестирование	Успех
18.05.2024	Система не должна во время работы модифицировать свой код или коды других программ	Тестировщик	Системное тестирование	Успех
18.05.2024	При удалении и передаче зафиксированных данных из базы данных система должна запрашивать подтверждения действия	Тестировщик	Системное тестирование	Успех
18.05.2024	Логгирование	Тестировщик	Системное тестирование	Успех

4.6 Перспективы применения

Данное исследование направлено на оценку значимости прогнозирования прочностных характеристик кокса по его гранулометрическому составу. В работе применяются методы машинного обучения и регрессионного анализа для решения этой задачи.

Цель исследования - разработка эффективного метода прогнозирования прочностных характеристик кокса по его гранулометрическому составу. Для достижения этой цели были проведены эксперименты и анализированы данные по гранулометрическому составу кокса и его прочностным характеристикам.

В результате исследований было установлено, что гранулометрический состав кокса является важным фактором, влияющим на его прочностные характеристики. Применение методов машинного обучения и регрессионного анализа позволяет установить связь между гранулометрическим составом угля и его прочностными характеристиками.

Полученные результаты исследования могут быть использованы для оптимизации процессов добычи и переработки угля, а также для разработки новых технологий в области угольной промышленности. Прогнозирование прочностных характеристик кокса по его гранулометрическому составу является актуальной задачей, имеющей практическую значимость для различных отраслей промышленности.

Применение методов машинного обучения и регрессионного анализа для прогнозирования прочностных характеристик угля по его гранулометрическому составу является перспективным направлением научных исследований. Полученные результаты исследования могут быть использованы для разработки новых методов контроля качества угля, что позволит повысить эффективность работы угольных предприятий.

4.7 Выводы по разделу 4

В данном разделе была выполнена апробация предложенных ранее моделей программного обеспечения прогнозирования прочностных характеристик кокса по его гранулометрическому составу.

Разработанный метод прогнозирования, основывающийся на визуальном анализе естественной способности кокса к разрушаемости, доказал свою

пригодность в сравнении с тем, что используется на данный момент, так как суммарное время выполнения анализа сократилось с 24 часов до 1 минуты. В совокупности с алгоритмом определения гранулометрического состава данная система позволит сократить количество простоев минимум на 30% в год.

Заключение

В данной исследовательской работе был разработан метод прогнозирования прочностных характеристик кокса, который основан на регрессионном анализе. Этот метод позволяет предсказать прочность кокса с высокой точностью.

Кроме того, был разработан алгоритм распознавания гранулометрического состава, который основан на использовании сегментационной нейронной сети. Этот алгоритм позволяет автоматически определять гранулометрический состав образцов кокса с высокой точностью.

Для реализации алгоритмов было проведено обучение нейронной сети на большом объеме данных и проведено тестирование для проверки эффективности и точности работы алгоритмов.

Также было спроектировано и разработано программное обеспечение, которое включает в себя серверное и клиентские приложения для удобного использования методов прогнозирования прочности кокса и определения гранулометрического состава в автоматизированном режиме.

Полученные результаты подтверждают эффективность и точность новых методов прогнозирования и определения состава кокса, что делает их перспективными для применения в промышленности.

По итогу выполнения работы были освоены следующие компетенции:

- способен осуществлять критический анализ проблемных ситуаций на основе системного подхода, вырабатывать стратегию действий;
- способен управлять проектом на всех этапах его жизненного цикла;
- способен организовывать и руководить работой команды, вырабатывая командную стратегию для достижения поставленной цели;
- способен применять современные коммуникативные технологии, в том числе на иностранном(ых) языке(ах), для академического и профессионального взаимодействия; способен анализировать и учитывать разнообразие культур в процессе межкультурного взаимодействия;

- способен определять и реализовывать приоритеты собственной деятельности и способы ее совершенствования на основе самооценки;
- способен самостоятельно приобретать, развивать и применять математические, естественнонаучные, социально-экономические и профессиональные знания для решения нестандартных задач, в том числе, в новой или незнакомой среде и в междисциплинарном контексте;
- способен разрабатывать оригинальные алгоритмы и программные средства, в том числе с использованием современных интеллектуальных технологий, для решения профессиональных задач;
- способен анализировать профессиональную информацию выделять в ней главное, структурировать, оформлять и представлять в виде аналитических обзоров с обоснованными выводами и рекомендациями;
- способен применять на практике новые научные принципы и методы исследований; способен разрабатывать и модернизировать программное и аппаратное обеспечение информационных и автоматизированных систем;
- способен самостоятельно приобретать с помощью информационных технологий и использовать в практической деятельности новые знания и умения, в том числе в новых областях знаний, непосредственно не связанных со сферой деятельности;
- способен применять при решении профессиональных задач методы и средства получения, хранения, переработки и трансляции информации посредством современных компьютерных технологий, в том числе, в глобальных компьютерных сетях;
- способен осуществлять эффективное управление разработкой программных средств и проектов;
- управление программно-техническими, технологическими и человеческими ресурсами.

Список литературы

1. Johnson J.W. Adapting Mask-RCNN for Automatic Nucleus Segmentation //IEEE Conference on Computer Vision and Pattern Recognition. 2018. P.7.
2. Филатов, С. В. Анализ влияния горячей прочности кокса (CSR) на показатели работы доменных печей / С. В. Филатов, И. Ф. Курунов, В. Н. Титов, А. М. Логинов // Сталь. – 2014. – № 10. – С. 10-14. – EDN UAHBNN.
3. ГОСТ 9434-75. Кокс каменноугольный. Классификация по размеру кусков: межгосударственный стандарт: издание официальное: утвержден и введен в действие Постановлением Государственного комитета стандартов Совета Министров СССР от 19 июня 1975 г. № 1586: дата введения 1977-01-01. - Москва: ИПК Издательство стандартов, 2002. - 3 с. - Текст: непосредственный.
4. Антонович, Е. В. Использование нейронной сети mask r-CNN для распознавания объектов / Е. В. Антонович // 75-я научная конференция студентов и аспирантов Белорусского государственного университета: Материалы конференции. В 3-х частях, Минск, 14–23 мая 2018 года / Редколлегия: В.Г. Сафонов [и др.]. – Минск: Белорусский государственный университет, 2018. – С. 508-511. – EDN YZLQRF.
5. Клячкин, В. Н. Прогнозирование состояния технического объекта с применением методов машинного обучения / В. Н. Клячкин, Д. А. Жуков // Программные продукты и системы. – 2019. – № 2. – С. 244-250. – EDN VBAGKR.
6. Емельянов, А. В. Анализ методов семантической сегментации изображений на основе нейронных сетей / А. В. Емельянов // Математические методы в технике и технологиях - ММТТ. – 2019. – Т. 12-1. – С. 195-201.
7. Alexandru Tatomir, Christopher McDermott, Jacob Bensabat, Holger Class, Katriona Edlmann. Conceptual model development using a generic Features, Events, and Processes (FEP) database for assessing the potential impact of

hydraulic fracturing on groundwater aquifers (англ.) // Advances in Geosciences. — Copernicus GmbH, 2018-08-22. — Т. 45. — С. 185–192

8. Демиденко Е.З. Линейная и нелинейная регрессия. — М.: Финансы и статистика, 1981. — 302 с

9. Алексеев Г.В. Основы системного анализа в пищевой промышленности: монография / Алексеев Г.В., Холявин И.И. — Саратов: Вузовское образование, 2017. — 112 с

10. Цветков В.Я. Модели в информационном поле. Palmarium Academic Publishing, 2018. 129 с.

11. Киселев Д.Ю., Киселев Ю.В., Макарьев В.Д. Структурный анализ потоков данных (Data Flow Diagrams – DFD). Метод. указания - Самара: СГАУ, 2014. – 12 с.

12. 610.12-1990 - IEEE Standard Glossary of Software Engineering Terminology

13. Agile – [Электронный ресурс] – Режим доступа: <https://www.atlassian.com/ru/agile> (Дата обращения: 05.12.2022)

14. Вендров А.М. CASE-технологии. Современные методы и средства проектирования информационных систем. – Москва: Финансы и статистика, 1998. – 176 с

15. Иванова Г.С. Технология программирования: Учебник для вузов. - М.: Изд-во МГТУ им. Н.Э. Баумана, 2002. -320 с.: ил. (Сер. Информатика в техническом университете.)

16. Лешек А. Мацяшек. Анализ и проектирование информационных систем с помощью UML 2.0. - Вильямс, 2008. – 816 с

17. Леоненков А.В. «Самоучитель UML – 2-е изд.» – СанктПетербург.: БХВ-Петербург, 2013. - 432 с.

18. Алексеев П. П., Квятковская И. Ю. Применение нейронных сетей в системе распознавания промысловых гидробионтов в условиях повышенной флуктуации // Вестник Астраханского государственного технического

университета. Серия: Управление, вычислительная техника и информатика. 2022. № 2. С. 76-86. URL:. DOI: 10.24143/2072-9502-2022-2-76-86

19. Архипов, В.А. Движение частиц дисперсной фазы в несущей среде: учебное пособие. [Электронный ресурс] / В.А. Архипов, А.С. Усанина. Томск: ТГУ, 2014. 252 с., с. 199- 200. Доступно на сайте: <http://e.lanbook.com/book/76789>.

20. Бархатов В. А. Распознавание дефектов с помощью искусственной нейронной сети специального типа //Дефектоскопия. – 2006. – №. 2. – С. 28-39.

21. Белоусов, И. С. Разработка глубокой нейронной сети для сегментации проблемных участков сельскохозяйственных полей / И. С. Белоусов, А. Ф. Рогачев // Инженерный вестник Дона. – 2022. – № 8(92). – С. 162-174.

22. Бирюков, А. В. Аналитическая гранулометрия / А. В. Бирюков // Вестник Кузбасского государственного технического университета. – 2012. – № 4(92). – С. 118-119.

23. Буляндра А.Ф., Майстренко И.А. Особенности взаимодействия коксов александровских бурых углей с кислородом воздуха // Энерготехнологии и ресурсосбережение. 2010. № 3. С. 3-8.

24. В.А.Головко. Нейронные сети: обучение, организация и применение. М., ИПРЖР, 2001

25. Вакуленко Д., Кравец А. Реинжиниринг бизнес-процессов агропромышленных предприятий в условиях сквозной цифровой трансформации // Вестник Астраханского государственного технического университета. Серия: Управление, вычислительная техника и информатика. 2021. №. 3. С. 115-125. URL: (дата обращения: 02.08.2022). DOI: 10.24143/2072-9502-2021-3-115-125

26. Влияние гранулометрии и вида заполнителя на свойства ремонтных составов / М. Ш. Саламанова, А. А. Узаева, Э. Д. Муртазаева, А. Р. Ахматов // Научные технологии и инновации : Электронный сборник докладов Международной научно-практической конференции, посвященной 65-летию

БГТУ им. В.Г. Шухова, Белгород, 29 апреля 2019 года. – Белгород: Белгородский государственный технологический университет им. В.Г. Шухова, 2019. – С. 361-368. – DOI 10.12737/conferencearticle_5cacedc40eafc8.07426207.

27. Головкин, В.А. Нейронные сети: обучение, организация и применение / В.А. Головкин. - М.: ИПРЖ, 2001. - 256 с.

28. Горбань А.Н. Обучение нейронных сетей / Горбань А.С. - М.: СП ПараГраф, 1991. 276 с.

29. Григорченко С. А., Капустин В. И. Классификация дефектов при автоматизированном радиографическом контроле сварных соединений. - Дефектоскопия, 2009, № 9, с. 73 - 87.

30. Гурвич А. К., Васильев В. А. Оценка конфигурации дефектов металлопродукции с плоскопараллельными поверхностями усеченным дельта-методом. - Контроль. Диагностика, 2013, № 10, с. 68 - 70.

31. Д.А.Тархов. Нейронные сети. Модели и алгоритмы. М., Радиотехника, 2005. (Научная серия "Нейрокомпьютеры и их применение", ред. А.И.Галушкин. Кн.18.)

32. Зейналабдыева, И. С. Модель принятия решений по управлению движением кассетного конвейера в производстве механообработки / И. С. Зейналабдыева // Вестник Уфимского государственного авиационного технического университета. – 2022. – Т. 26. – № 2(96). – С. 126-133.

33. Исследование особенностей горения смесей коксов углей различной степени метаморфизма и коксов биомассы / И. В. Бесценный, Т. С. Щудло, Н. И. Дунаевская, А. И. Топал // Теплоэнергетика. – 2013. – № 12. – С. 4-8.

34. Исследование синергетических эффектов при факельном сжигании угольных смесей / И.В. Бесценный, Д.Л. Бондзик, Т.С. Щудло, Н.И. Дунаевская // Современная наука: идеи, исследования, результаты, технологии. 2011. № 3 (8). С. 119-124.

35. Квачко, С. К. Литолого-фациальный анализ нижнемеловых отложений Ванкорского газонефтяного месторождения (Западная Сибирь) / С. К. Квачко

// Вестник Томского государственного университета. – 2010. – № 338. – С. 223-226.

36. Коузов П.А. Основы анализа дисперсного состава промышленных пылей и измельченных материалов. 3-е изд. Л.: Химия, 1987. 264 с., с.11.

37. Лиля В.Б. Алгоритм и программная реализация адаптивного метода обучения искусственных нейронных сетей // Инженерный вестник Дона. 2012. №1. URL: ivdon.ru/magazine/archive/n1y2012/626.

38. Липухин, Ю.В. Автоматизация металлургических агрегатов / Ю.В. Липухин, Ю.И. Булатов, К. Адельман, М. Кнорр. - М.: Металлургия, 1992. - 303 с.

39. Львов, А.С. Алгоритмы многопараметрического контроля металлургических процессов вычислителями низких конфигураций / А.С. Львов, А.В. Полянский // Информатизация процессов формирования открытых систем на основе СУБД, САПР, АСНИ и СИИ: материалы 5-й науч.-техн. конф. - Вологда. - 2009 - С. 159-163.

40. Майстренко А.Ю., Соболев В.С., Голенко И.Л. Установка для исследования кинетики взаимодействия угольных коксов с кислородом и диоксидом углерода в кипящем слое // Высокотемпературное преобразование энергии: Сб. науч. тр. Киев: ИПЭ АН УССР, 1989. С. 102-106.

41. Макс Ж. Методы и техника обработки сигналов при физических измерениях: В двух томах. Пер. с франц.- М.: Мир, 1983.

42. Отырба, Р. Р. Использование методов глубокого обучения для анализа изображений аэрокосмического мониторинга в интересах обнаружения элементов новизны на местности / Р. Р. Отырба, М. А. Дрюченко, А. А. Сирота // Вестник Воронежского государственного университета. Серия: Системный анализ и информационные технологии. – 2022. – № 2. – С. 110-134.

43. Парфентьев, К. П. Построение моделей подстилающей поверхности с использованием сегментационных нейронных сетей / К. П. Парфентьев, К. А. Неусыпин // Проблемы экоинформатики : Материалы XV Международного

симпозиума, Москва, 06–08 декабря 2022 года / Под редакцией Ф.А. Мкртчяна. – Москва: Московское НТО радиотехники, электроники и связи им. А.С. Попова, 2022. – С. 87-89.

44. Полянский А. В. и др. Повышение быстродействия в обучении нейронной сети многопараметрическому контролю процессов в металлургии на базе вычислителей низкой конфигурации //Вестник Череповецкого государственного университета. – 2010. – №. 3.

45. Соловьев Р.А., Тельпухов Д.В., Кустов А.Г. Автоматическая сегментация спутниковых снимков на базе модифицированной свёрточной нейронной сети UNET. Инженерный вестник Дона, 2017, №4. URL: ivdon.ru/ru/magazine/archive/n4y2017/4433.

46. Сырямкин В. И., Горбачев С. В. Применение нейросетевых алгоритмов анализа изображений материалов в рентгеновских микротомографах. - Изв. вузов. Физика, 2013, т. 56, № 10/2, с. 29 - 34.

47. Тарасов, А. С. Применение сверточных сегментационных нейронных сетей для экологического мониторинга земной поверхности / А. С. Тарасов, М. Б. Никифоров, Н. И. Бакамбис // Известия Тульского государственного университета. Технические науки. – 2021. – № 6. – С. 3-10.

48. Тенденции применения систем автоматизированного распознавания изображений в металлургической промышленности / И. И. Мацко, И. А. Демьяненко, В. В. Павлов, О. С. Логунова // Инженерная поддержка инновации и модернизации : Материалы Международной заочной конференции, посвященной 15-летию со дня создания РУО АИН, Екатеринбург, 01–10 декабря 2010 года. – Екатеринбург, 2010. – С. 77-79.

49. Уоссерман Ф. Нейрокомпьютерная техника: Теория и практика / Уоссерман Ф. - М. Мир, 1992. 202 с.

50. Уоссермен Ф. Нейрокомпьютерная техника.- М.: Мир, 1992.

51. Фигуровский Н.А. Седиментометрический анализ / Н.А. Фигуровский; под ред. и с предисл. П.А. Ребиндера; АН СССР, Ин-т физич. химии. М.; Л.: Изд-во АН СССР, 1948. 332 с., с. 35-3

52. Фокин В. А. и др. Оценка эффективности двух алгоритмов сегментации цифрового радиационного изображения объекта контроля. – 2017.

53. Харисова З.И., Фетисов В.С. Система для экспрессного определения гранулометрического состава суспензий на основе видеотехнических средств и искусственной нейросети, дообучаемой в процессе работы // Приборы и системы. Управление, контроль, диагностика. 2017. № 2. С. 57-64.

54. Харисова, З. И. Конструкция проточной измерительной камеры в составе системы гранулометрического анализа жидких дисперсных сред / З. И. Харисова // Электротехнические и информационные комплексы и системы. – 2017. – Т. 13. – № 3. – С. 75-78.

55. Ходаков Г. С. Основные методы дисперсионного анализа порошков / Г. С. Ходаков. М.: Стройиздат, 1968. 199 с., с. 10.

56. A martian case study of segmenting images automatically for granulometry and sedimentology, Part 2: Assessment / S. Karunatilake, J. R. Skok, S. M. McLennan [et al.] // Icarus. – 2014. – Vol. 229. – P. 408-417.

57. Современные информационные технологии. Теория и практика: Материалы VI Всероссийской научно-практической конференции «Современные информационные технологии. Теория и практика» (г. Череповец, 29 ноября 2023 г.) / под редакцией Е. В. Ершова. Череповец: ЧГУ, 2024 – 510 с.

Устав проекта

Программное обеспечение прогнозирования прочностных характеристик кокса по его гранулометрическому составу

Общие положения

Данный документ является соглашением между основными участниками проекта, определяющим цели проекта, рамки проекта, обязанности участников проекта, основные этапы выполнения работ по проекту.

Соблюдение положения устава является обязательным для всех участников проекта.

Данный документ может меняться в ходе выполнения проекта. Внесение изменений и дополнений в устав проекта инициируется руководителем проекта и утверждается заказчиком (решением управляющего комитета).

Настоящий устав действует, начиная с момента подписания договора на проектные работы и до окончания срока реализации проекта (до подписания акта сдачи-приемки всех работ по проекту).

Требования, удовлетворяющие потребности, пожелания и ожидания заказчика и других участников проекта

Программный продукт должен обеспечить снижение затрат времени и упрощение процесса анализа гранулометрического состава кокса выполнении следующих требований:

- динамическое отслеживание и определение класса гранул на конвейере;
- анализ гранулометрического состава для дальнейшего расчёта прочностных характеристик;

- ведение базы данных снимков гранулометрического состава с конвейера.

Имеющиеся на рынке программные продукты не в полной мере учитывают специфику и потребности АО «Северсталь менеджмент», а точнее «Дирекции по техническому развитию и качества» и не предлагают комплексного решения для поставленной задачи.

Основной целью проекта является оптимизация процесса управления доменной печью за счёт сокращения расхода кокса.

Производственная необходимость

Перед акционерным обществом «Северсталь менеджмент» стоит задача не только обеспечить новым инструментом персонал компании, но и сократить расход кокса у доменных печей.

В настоящее время отслеживание, сбор данных и формирование отчетности происходит вручную, что занимает большой объем времени.

Требования к продукту, который является предметом проекта

Требования к функциональным характеристикам:

- выбор языка и среды программирования, соответствующих ИТ-концепции организации заказчика, по возможности среди свободного программного обеспечения;
- возможность обработка видеозаписи с помощью сегментационной свёрточной нейронной сети с целью нахождения гранул на конвейере и определения их размера;
- наличие визуального анализа полученных данных в виде настраиваемых графиков;
- наличие системы предсказания прочностных характеристик твёрдости и стойкости.

Цель проекта

Разработка программного обеспечения для расширения анализа и обработки данных акционерного общества «Северсталь менеджмент», которая позволит строить интерактивные отчеты с целью упрощения формирования данных по гранулометрическому составу, которая позволит добиться повышения качества продукции с помощью технологии многоуровневой системы обнаружения, а также обеспечивать точность прогнозирования прочностных характеристик кокса в интервале от 80 до 85 %.

Целью исследования системы является изучение предметной области определения гранулометрического состава кокса, формирование модели данных, создание методов динамического отслеживания, анализа и идентификации гранулометрического состава кокса на конвейере, выявлении размера с учетом последующего анализа.

Критерии оценки успешной реализации проекта

- стабильность работы программы;
- чёткая идентификация гранул кокса в кадре и выделение их контура;
- построение графиков на основе результатов работы модуля обнаружения;
- сохранение данных во время каждого периода анализа гранулометрического состава кокса на конвейере.

Расписание контрольных событий представлено в табл. П1.1.

Таблица П1.1

Расписание контрольных событий

Этап	Срок	Ответственный
1	2	3
1 этап: Анализ предметной области, постановка задачи (разработка требований)	ноябрь – декабрь 2022	Богданов А.П.

Продолжение табл. П1.1

1	2	3
2 этап: Выбор и описание подхода и инструментальных средств разработки программного обеспечения	январь - май 2023	Богданов А.П.
3 этап: Выбор и описание математического аппарата, используемого в проекте	май - октябрь 2023	Богданов А.П.
4 этап: Проектирование программного обеспечения: разработка логической модели (модель использования, логическая модель, модель процессов) и физической модели (модель реализации и модель развертывания)	октябрь – декабрь 2023	Богданов А.П.
5 этап: Разработка программного обеспечения	январь – март 2024	Богданов А.П.
6 этап: Экспериментальная проверка программного обеспечения, оформление результатов проекта	март – май 2024	Богданов А.П.

Информация об участниках проекта

Информация об участниках проекта представлена в табл. П1.2 - П1.4.

Таблица П1.2

Заказчик проекта

Организация:	АО «Северсталь менеджмент»
Полномочия:	● формирование общих требований к результату ● проекта; ● назначение руководителя проекта; ● контроль хода выполнения работ по проекту; ● приемка результатов проекта;
Ответственность:	● за своевременное обеспечение проекта ресурсами;

Руководитель (менеджер) проекта

Таблица П1.3

Организация:	ФГБОУ ВПО «Череповецкий государственный университет»
ФИО, должность	Ершов Евгений Валентинович, заведующий кафедрой МПО ЭВМ
Контактная информация	Тел: +7(8202) 51-90-69
Полномочия:	● формирование команды проекта, распределение ответственности, организация взаимодействия между участниками проектной команды и заказчиком; ● распределение ресурсов по стадиям (этапов) проекта; ● управление рисками проекта, управление процессом решения проблем; ● участие в разрешении противоречий в проектных решениях; ● планирование, организация и контроль выполнения работ по достижению целей проекта с требуемыми затратами, качеством и в заданный срок; ● достижение запланированных результатов ● оценка результативности и разработка отчетных документов по проекту.
Ответственность:	● качественное и своевременное исполнение работ по проекту, достижение запланированных результатов

Разработчик проекта

Таблица П1.4

1	2
Организация:	ФГБОУ ВПО «Череповецкий государственный университет»
ФИО, должность	Богданов Александр Павлович
Контактная информация	Тел: +7(921) 544-93-23

Продолжение табл. П1.4

1	2
Полномочия:	● разработка, проектирование программного продукта согласно требованиям технического задания ● создание дорожной карты для планирования и ● организации выполнения работ по достижению целей ● коммуникация с заказчиком, получение обратной связи по реализации проекта ● распределение ресурсов команды по стадиям проекта
Ответственность:	● качественное и своевременное исполнение работ по проекту, достижение запланированных результатов

Функциональные организации и их участие

Проект реализуется на базе ФГБОУ ВО «Череповецкий государственный университет».

Заказчиком проекта является АО «Северсталь-менеджмент».

Стратегия «Северстали» заключается в повышении финансовой эффективности, создании максимальной добавленной стоимости и увеличении вознаграждения акционеров в сочетании с заботой о сотрудниках компании и минимизацией негативного воздействия на окружающую среду. В число стратегических приоритетов «Северстали» входят высочайшее качество обслуживания клиентов, сохранение лидерства в отрасли по себестоимости производства и реализация новых возможностей. В основе реализации этих приоритетов лежит передовая корпоративная культура «Северстали».

Бюджет проекта

Проект реализуется в рамках освоения магистерской программы «Искусственный интеллект» по направлению 09.04.04 Программная инженерия.

В рамках реализации данного проекта отсутствует необходимость приобретения программных продуктов специального назначения, т.к. все требования для разработки можно реализовать на свободно распространяемом программном обеспечении.

Для разработки программного обеспечения необходим компьютер с характеристиками выше среднего, к примеру на базе процессора Intel i5-12400f и видеокарты NVidia RTX 3070 (100 000 рублей).

Программное решение предполагает вложение финансовых средств, поэтому возникает вопрос рационального их применения и выбора наиболее эффективного решения задачи.

Расчет экономической эффективности и срока окупаемости проектируемой программы для ЭВМ начинается с расчета трудовых затрат.

Состав разработчиков: один fullstack-программист.

В себестоимость разработки входят: основная заработная плата разработчиков, дополнительная заработная плата, единый социальный налог, затраты на использование машинного времени, затраты на носители информации, затраты на текущий и профилактический ремонт вычислительной техники, прочие эксплуатационные расходы.

В итоге формула для вычисления себестоимости программного продукта выглядит следующим образом:

$$C_{пп} = Z_{тр} + Z_{м.вр} + Z_{н.и} + Z_{рем} + Z_{пр}, \quad (1)$$

где

$C_{пп}$ – себестоимость программного продукта, руб.;

$З_{тр}$ – затраты на оплату труда, руб;

$З_{м.вр}$ – затраты на использование машинного времени, руб;

$З_{н.и}$ – затраты на носители информации, руб;

$З_{рем}$ – затраты на текущий и профилактический ремонт вычислительной техники, руб;

$З_{пр}$ – прочие эксплуатационных расходов, руб.

Требуется рассчитать затраты на оплату труда $З_{тр}$.

Над созданием программного продукта работал 1 программист-выпускника и руководитель ВКР, которые работали с зарплатой по окладу, вычисляемой по формуле:

$$З_{тр} = (З_{общ} + Отч) * Т_n, \quad (2)$$

где

$З_{общ}$ – общая зарплата работника за час, руб.;

Отч – отчисления с зарплаты, %;

$Т_n$ – время написания программы, ч.

$$З_{пр} = С_{Т_{пр}}, \quad (3)$$

Дополнительная зарплата рассчитывается по формуле:

$$З_{доп} = \frac{З_{пр} * Н_{доп}}{100\%}, \quad (4)$$

где

$З_{пр}$ – заработная плата работника за час, руб.;

$H_{\text{доп}}$ – норма отчислений на дополнительную зарплату (10%).

В итоге общая зарплата формируется на основе их суммы:

$$Z_{\text{общ}} = Z_{\text{пр}} + Z_{\text{доп}}. \quad (5)$$

Отчисления «Отч на соцстрах», фонд занятости и пенсионный фонд вычисляются по следующей формуле:

$$\text{Отч} = O_{\text{сс}} + O_{\text{фз}} + O_{\text{пф}}, \quad (6)$$

Где

$O_{\text{сс}}$ – отчисления на соцстрах (0,5% от $Z_{\text{общ}}$);

$O_{\text{фз}}$ – отчисления в фонд занятости (0,5% от $Z_{\text{общ}}$);

$O_{\text{пф}}$ – отчисления в пенсионный фонд (2% от $Z_{\text{общ}}$).

Далее необходимо рассчитать заработную плату программиста с окладом 80000 руб., который занимался разработкой с 1 февраля 2024 года до 15 июня 2024 года по 6 часов в рабочие дни недели. Расчеты производятся по формулам (2) – (6):

$$Z_{\text{доп}} = 80000 * 10\% = 8000 \text{ (руб./мес.)}$$

$$Z_{\text{общ}} = 80000 + 8000 = 88000 \text{ (руб./мес.)}$$

$$O_{\text{сс}} = 88000 * 0,5\% = 440 \text{ (руб.)}$$

$$O_{\text{фз}} = 88000 * 0,5\% = 440 \text{ (руб.)}$$

$$O_{\text{пф}} = 88000 * 2\% = 1760 \text{ (руб.)}$$

$$\text{Отч} = 440 + 440 + 1760 = 2640 \text{ (руб.)}$$

$$Z_{\text{тр}} = (88000 + 2640) * 4,5 = 407880 \text{ (руб.)}$$

После этого необходимо рассчитать заработную плату руководителя выпускной квалификационной работы с окладом 31226 руб., который

занимался разработкой с 1 февраля 2024 года до 15 июня 2024 года. Расчеты производятся по формулам (2) – (6):

$$З_{\text{доп}} = 31226 * 10\% = 3122,6 \text{ (руб./мес.)}$$

$$З_{\text{общ}} = 31226 + 3122,6 = 34348,6 \text{ (руб./мес.)}$$

$$О_{\text{сс}} = 34348,6 * 0,5\% = 171,74 \text{ (руб.)}$$

$$О_{\text{фз}} = 34348,6 * 0,5\% = 171,74 \text{ (руб.)}$$

$$О_{\text{пф}} = 34348,6 * 2\% = 686,97 \text{ (руб.)}$$

$$Отч = 171,74 + 171,74 + 697,4 = 1030,45 \text{ (руб.)}$$

$$З_{\text{тр}} = (34348,6 + 1030,45) * 4,5 = 159205,73 \text{ (руб.)}$$

Таким образом, затраты на заработную плату составляют 567085,73 рубля.

Все данные по заработной плате сводятся в таблице (табл. П1.5).

Таблица П1.5

Данные по заработной плате

Должность разработчика	Время работы, мес.	Ст, руб.	Здоп, руб.	Зобщ, руб.	Отч, руб.	Зтр, руб.
Программист-выпускник	4.5	80000	8000	88000	2640	407880
Руководитель ВКР	4.5	31226	3122,6	34348,6	1030,45	159205,73
Итого:		111226	11122,6	122348,6	3670,45	567085,73

Затраты на использование машинного времени вычисляются по формуле:

$$З_{\text{м.вр}} = C_{\text{м.вр}} * Вр_{\text{в.т}}, \quad (7)$$

где

$З_{\text{м.вр}}$ – затраты на использование машинного времени, руб.;

$C_{м.вр}$ – стоимость одного часа машинного времени, руб./ч;

$Вр_{в.т}$ – время использования вычислительной техники, ч.

Стоимость одного часа машинного времени рассчитывается по формуле:

$$C_{м.вр} = \frac{Ц_k}{C_{сл.к} * K_{р.д} * Вр_c} + C_{т_э} * M_{вс}, \quad (8)$$

где

$C_{м.вр}$ – стоимость одного часа машинного времени, руб./ч;

$Ц_k$ – покупная цена компьютера, руб.;

$C_{сл.к}$ – срок службы компьютера, год;

$K_{р.д}$ – количество рабочих дней в году;

$Вр_c$ – время работы компьютера в течение суток, ч;

$C_{т_э}$ – стоимость одного кВт*ч электроэнергии, руб.;

$M_{вс}$ – мощность вычислительной системы, кВт.

Время использования вычислительной техники рассчитывается по следующей формуле:

$$Вр_{в.т} = K_{д.р.} * Вр_c, \quad (9)$$

где

$Вр_{в.т}$ – время использования вычислительной техники, ч;

$K_{д.р.}$ – количество дней разработки ПО.

Затраты на носители информации принимаются в размере 2% от цены вычислительной техники.

Затраты на текущий и профилактический ремонт принимаются в размере 4% от цены вычислительной техники.

Прочие эксплуатационные расходы включают в себя затраты на освещение, отопление, охрану, уборку и текущий ремонт помещений. Они принимаются в размере 10% от стоимости помещения (или его аренды), где происходит разработка программного продукта.

Время работы компьютера в течение суток составляет 6 часов, а количество дней на разработку ПО составляет 90 рабочих дней за 4,5 месяца с февраля по июнь.

Таким образом, по формуле (9) получаем, что время работы вычислительной техники:

$$V_{p.v.t.} = 90 * 6 = 540 \text{ (ч.)}$$

Покупная цена персонального компьютера разработчика составляет 100000 руб. и имеет срок службы 4 года, количество рабочих дней в году составляет 247 дней, в течение суток он будет работать 6 часов. Его мощность составляет 0,311 кВт, тогда как стоимость электроэнергии составляет 3 рубля за 1 кВт*ч.

Тогда по формуле (8) можно вычислить стоимость одного часа машинного времени:

$$C_{m.vp} = \frac{100\,000}{4 * 247 * 6} + 3 * 0,311 \approx 17,8 \text{ (руб./ч)}$$

По формуле (7) вычислим затраты на использование машинного времени:

$$Z_{m.vp} = 17,8 * 540 = 9613 \text{ (руб.)}$$

Затраты на носители информации:

$$100\,000 * 2\% = 2000 \text{ (руб.)}$$

Затраты на текущий и профилактический ремонт:

$$100\,000 * 4\% = 4000 \text{ (руб.)}$$

Прочие эксплуатационные расходы:

$$11000 * 0,1 = 1100 \text{ (руб.)}$$

Таким образом по формуле (1) рассчитаем себестоимость разработки ПО:

$$C_{пп} = 567085 + 9613 + 2000 + 4000 + 1100 = 583798 \text{ (руб.)}$$

Расчет цены программного продукта

Для определения минимальной цены, ниже которой разработчику будет невыгодно продавать программный продукт, используется следующая формула:

$$C_{п.п} = C_{п.п} * (1 + N_{пр}), \quad (10)$$

где

$C_{п.п}$ – цена программного продукта, руб.;

$C_{п.п}$ – себестоимость программного продукта, руб.;

$N_{пр}$ – норматив прибыли (20 %, в формуле $N_{пр} = 0,2$).

Тогда по этой формуле получаем стоимость разрабатываемой системы для пользователя:

$$C_{п.п} = 583798 * (1 + 0,2) = 700557,6 \text{ (руб.)}$$

Расчет экономической эффективности

В расчет экономической эффективности входит определение эксплуатационных расходов и капитальных затрат потребителя, годовой экономии эксплуатационных расходов у одного потребителя, срока окупаемости программного продукта, годового экономического эффекта.

Расходы потребителя, связанные с эксплуатацией программы, определяются по формуле (11).

$$P_{\text{э.п.}} = Vp_{\text{п.п.}} * C_{\text{м.вр}} + \frac{Ц_{\text{п.п.}}}{C_{\text{сл}}}, \quad (11)$$

где

$P_{\text{э.п.}}$ – эксплуатационные расходы потребителя, руб.;

$Vp_{\text{п.п.}}$ – объем машинного времени в течение года, необходимый для решения данной задачи с использованием программы, ч;

$C_{\text{м.вр}}$ – стоимость одного часа машинного времени, руб./ч;

$Ц_{\text{п.п.}}$ – цена программного продукта, руб.;

$C_{\text{сл}}$ – срок службы программного продукта, год.

Если у пользователей будет компьютер стоимостью 30000 рублей, который обладает мощностью 0,224 кВт, тогда как стоимость электроэнергии составляет 3 рубля за кВт*ч, то по формуле (8) можно вычислить стоимость одного часа машинного времени у пользователя:

$$C_{\text{м.вр}} = \frac{30000}{2*247*1} + 3 * 0,224 = 61,3 \text{ (руб./ч)}$$

Пусть программа будет работать 247 дней по 1 часу в день и иметь срок службы в два года. Тогда по формуле (11) можно вычислить расходы потребителя при использовании разработанного ПО:

$$P_{\text{э.п.}} = 247 * 61,3 + \frac{700557}{2} = 365419,6 \text{ (руб.)}$$

Если на момент внедрения программного продукта у потребителя все работы выполнялись вручную, тогда капитальные затраты рассчитываются по формуле:

$$P_{\text{кап}} = \frac{Vp_{\text{п.п.}} * K_{\text{эвм}}}{\Phi_{\text{вр}}} + Ц_{\text{п.п.}}, \quad (12)$$

где

$P_{\text{кап}}$ – капитальные расходы потребителя, руб.;

$\Phi_{\text{вр}}$ – полезный годовой фонд времени работы вычислительной техники, принимается условно 2000 ч в год;

$K_{\text{ЭВМ}}$ – капитальные затраты на вычислительную технику, для которой предназначена программа, руб.

Капитальные затраты на вычислительную технику рассчитываются по формуле:

$$K_{\text{ЭВМ}} = C_{\text{ЭВМ}} + P_{\text{п.п}}, \quad (13)$$

где

$C_{\text{ЭВМ}}$ – цена вычислительной техники, руб.;

$P_{\text{п.п}}$ – прочие расходы потребителя, связанные с помещением (отопление, освещение, уборка и т.д.), принимаются в размере 10 % от стоимости помещения потребителя (или его аренды), руб.

По формуле (13) можно вычислить капитальные затраты пользователя на вычислительную технику:

$$K_{\text{ЭВМ}} = 30000 + 2000 = 32000 \text{ (руб.)}$$

По формуле (12) можно узнать все капитальные расходы потребителя:

$$P_{\text{кап}} = \frac{247 \cdot 30000}{2000} + 700557,6 = 704262,6 \text{ (руб.)}$$

Для расчета годовой экономии эксплуатационных расходов потребителя вычисляются эксплуатационные затраты потребителя при решении задачи вручную применяют формулу:

$$P_{\text{э.руч}} = 1,21 \cdot \Phi_{\text{ЗП}} \cdot 12, \quad (14)$$

где

$P_{э.руч}$ – эксплуатационные расходы потребителя при решении задачи вручную, руб.;

ФЗП – фонд заработной платы персонала, обслуживающего решение задачи вручную, руб.;

12 – количество месяцев в году;

1,21 – поправочный коэффициент.

$$P_{э.руч} = 1,21 * 72000 * 12 = 1045440 \text{ (руб.)}$$

Тогда годовая экономия эксплуатационных расходов у одного потребителя рассчитывается по формуле:

$$\mathcal{E} = P_{э.руч} - P_{э.п.}, \quad (15)$$

Срок окупаемости программного продукта рассчитывается по формуле:

$$T_{ок} = \frac{P_{кап}}{\mathcal{E}}, \quad (16)$$

Годовой экономический эффект, получаемый одним потребителем, рассчитывается по формуле:

$$\mathcal{E}\mathcal{E} = \mathcal{E} - E_n * P_{кап}, \quad (17)$$

где

E_n – нормативный коэффициент эффективности дополнительных капитальных вложений, равный 0,15.

Годовая экономия эксплуатационных расходов у одного потребителя составляет:

$$\mathcal{E} = 1045440 - 704262,6 = 341177,4 \text{ (руб.)}$$

Срок окупаемости ИС составляет:

$$T_{\text{ок}} = \frac{704262,6}{341117,4} = 2,06 \text{ (года)}$$

Годовой экономический эффект составляет:

$$\text{ЭЭ} = 341117,4 - 0,15 * 704262,6 = 235478,01 \text{ (руб.)}$$

Основные риски проекта

Риски в рамках реализуемого проекта представлены в табл. П1.6.

Таблица П1.6

Риски проекта

№ п/п	Риски проекта	Мероприятия по управлению рисками (по предотвращению риска / по реагированию при возникновении риска)
1	Риски, связанные с не выполнением участниками взятых в рамках проекта обязательств по причине форс мажорных обстоятельств (не зависящих от воли участников проекта)	Поиск замены участника/участников проекта Пересмотр сроков реализации проекта
2	Риски, связанные с недостаточным объемом финансированием реализуемого проекта	Пересмотр сроков реализации проекта Приостановление проекта

Реальная бизнес-ситуация, служащая обоснованием проекта с данными о
прибыли на инвестиции

Проект не является коммерческим, доходы по проекту в период его реализации не предусмотрены.

Текст программы

Текст модуля train_with_optuna представлен на рис. П2.1.

```

import json
import logging
from math import sqrt
import pickle
import warnings
import numpy as np
import optuna
import pandas as pd
from sklearn.ensemble import StackingRegressor
from sklearn.exceptions import ConvergenceWarning
from sklearn.linear_model import ElasticNet,
LinearRegression
from sklearn.model_selection import KFold, StratifiedKFold,
train_test_split
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    mean_squared_log_error,
    r2_score,
)
from catboost import CatBoostRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from xgboost import XGBRegressor
from scipy.linalg import LinAlgWarning
from optuna.samplers import TPESampler
warnings.filterwarnings(action="ignore")
logging.getLogger("optuna").setLevel(logging.ERROR)
dataset_path = "data/parq/kd_dataset.parquet"
df = pd.read_parquet(dataset_path, engine="fastparquet")
feature_cols = [
    "GRAN_0_10",
    "GRAN_10_25",
    "GRAN_25_40",
    "GRAN_40_60",
    "GRAN_60_80",
    "GRAN_80_999",
]
# target_cols = ['M10', 'M40', 'WI', 'CSR', 'CRI', 'C. +80MM',
'C. 60-80MM', 'C. 40-60MM', 'C. 25-40MM', 'C. 0-25MM',
'WA', 'SD', 'VDAF', 'AD', 'VD']
target_cols = ["M10", "M40"]
df = df[feature_cols, target_cols]
stats_df = pd.DataFrame()
def get_mlp_model(params):
    model_params = {
        "hidden_layer_sizes": [
            params["hidden_layer_sizes_" + str(i)] for i in
range(params["n_layers"])
        ],
        "activation": params["activation"],
        "solver": params["solver"],
        "alpha": params["alpha"],
        "max_iter": params["max_iter"],
        "learning_rate": params["learning_rate"],
        "learning_rate_init": params["learning_rate_init"],
    }
    return MLPRegressor(**model_params)
def get_cat_boost_model(params):
    model_params = {
        "depth": params["depth"],
        "learning_rate": params["learning_rate"],
        "iterations": params["iterations"],
        "l2_leaf_reg": params["l2_leaf_reg"],
        "boosting_type": params["boosting_type"],
        "bootstrap_type": params["bootstrap_type"],
        "leaf_estimation_method":
params["leaf_estimation_method"],
        "loss_function": params["loss_function"],
        "subsample": params["subsample"],
        "grow_policy": params["grow_policy"],
        "rsm": params["rsm"],
        "random_strength": params["random_strength"],
    }
    return CatBoostRegressor(verbose=False, **model_params,
allow_writing_files=False)
def get_xgb_model(params):
    model_params = {
        "max_depth": params["max_depth"],
        "learning_rate": params["learning_rate"],
        "n_estimators": params["n_estimators"],
        "subsample": params["subsample"],
        "colsample_bytree": params["colsample_bytree"],
        "reg_alpha": params["reg_alpha"],
        "reg_lambda": params["reg_lambda"],
        "min_child_weight": params["min_child_weight"],
    }
    return XGBRegressor(**model_params)
def get_linear_regression(params):
    pol_params = {
        "degree": params["degree"],
        "interaction_only": params["interaction_only"],
        "include_bias": params["include_bias"],
    }
    reg_params = {
        "fit_intercept": params["fit_intercept"],
        "copy_X": params["copy_X"],
        "alpha": params["alpha"],
        "l1_ratio": params["l1_ratio"],
        "max_iter": params["max_iter"],
        "tol": params["tol"],
        "selection": params["selection"],
    }
    return Pipeline(
        [
            ("Pol", PolynomialFeatures(**pol_params)),
            ("Reg", ElasticNet(**reg_params)),
        ]
    )
def get_mlp_model_trial(trial: optuna.Trial):
    model_params = {
        "hidden_layer_sizes": [
            trial.suggest_int("hidden_layer_sizes_" + str(i), 2, 32,
log=True)
            for i in range(trial.suggest_int("n_layers", 5, 10))
        ],
        "activation": trial.suggest_categorical(
            "activation", ["identity", "logistic", "tanh", "relu"]
        ),
        "solver": trial.suggest_categorical("solver", ["lbfgs",
"sgd", "adam"]),
        "alpha": trial.suggest_float("alpha", 1e-6, 1, log=True),
        "max_iter": trial.suggest_int("max_iter", 10, 1000,
log=True),
        "learning_rate": trial.suggest_categorical(
            "learning_rate", ["constant", "invscaling", "adaptive"]
        ),
        "learning_rate_init": trial.suggest_float(
            "learning_rate_init", 0.001, 0.1, log=True
        ),
    }

```

Рис. П2.1. Текст модуля train_with_optuna

П2.1. Продолжение

```

return MLPRegressor(
    **model_params,
)
def get_cat_boost_model_trial(trial: optuna.Trial):
    model_params = {
        "depth": trial.suggest_int("depth", 1, 7),
        "learning_rate": trial.suggest_float("learning_rate", 0.001,
0.1, log=True),
        "iterations": trial.suggest_int("iterations", 30, 200,
log=True),
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-6, 1,
log=True),
        "boosting_type": trial.suggest_categorical(
            "boosting_type", ["Plain", "Ordered"])
    ),
    "bootstrap_type": trial.suggest_categorical(
        "bootstrap_type", ["Bernoulli", "MVS"])
    ),
    "leaf_estimation_method": trial.suggest_categorical(
        "leaf_estimation_method", ["Gradient", "Newton",
"Exact", "Simple"])
    ),
    "loss_function": trial.suggest_categorical(
        "loss_function", ["RMSE", "MAE", "MAPE"])
    ),
    "subsample": trial.suggest_float("subsample", 0.5, 1),
    "grow_policy": trial.suggest_categorical(
        "grow_policy", ["SymmetricTree", "Depthwise",
"Lossguide"])
    ),
    "rsm": trial.suggest_float("rsm", 0.5, 1),
    "random_strength": trial.suggest_float("random_strength",
0.5, 1),
    }
    return CatBoostRegressor(
        verbose=False,
        allow_writing_files=False,
        early_stopping_rounds=15,
        **model_params,
    )
def get_xgb_model_trial(trial: optuna.Trial):
    model_params = {
        "max_depth": trial.suggest_int("max_depth", 1, 8),
        "learning_rate": trial.suggest_float("learning_rate", 0.001,
0.1, log=True),
        "n_estimators": trial.suggest_int("n_estimators", 30, 200,
log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1),
        "colsample_bytree":
trial.suggest_float("colsample_bytree", 0.5, 1),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-6, 1,
log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-6, 1,
log=True),
        "min_child_weight":
trial.suggest_int("min_child_weight", 1, 20, log=True),
        "gamma": trial.suggest_float("gamma", 0, 1),
        "eta": trial.suggest_float("eta", 0.001, 0.1, log=True),
        "booster": trial.suggest_categorical("booster", ["gbtree",
"gblinear", "dart"]),
        "tree_method": trial.suggest_categorical(
            "tree_method", ["auto", "exact", "approx", "hist"])
    ),
    "grow_policy": trial.suggest_categorical(
        "grow_policy", ["depthwise", "lossguide"])
    ),
    }
    return XGBRegressor(
        **model_params,
        validate_parameters=True,
        early_stopping_rounds=15
    )
def get_linear_regression_trial(trial: optuna.Trial):
    pol_params = {
        "degree": trial.suggest_int("degree", 1, 2),
        "interaction_only":
trial.suggest_categorical("interaction_only", [True]).

```

```

        "include_bias": trial.suggest_categorical("include_bias",
[True]),
    },
    reg_params = {
        "fit_intercept": trial.suggest_categorical("fit_intercept",
[False]),
        "copy_X": trial.suggest_categorical("copy_X", [True]),
        "alpha": trial.suggest_float("alpha", 1e-6, 1, log=True),
        "l1_ratio": trial.suggest_float("l1_ratio", 1e-6, 1,
log=True),
        "max_iter": trial.suggest_int("max_iter", 100, 2000,
log=True),
        "tol": trial.suggest_float("tol", 1e-6, 1, log=True),
        "selection": trial.suggest_categorical("selection",
["cyclic", "random"]),
    }
    a = PolynomialFeatures()
    return Pipeline(
        [
            ("Pol", PolynomialFeatures(**pol_params)),
            ("Reg", ElasticNet(**reg_params)),
        ]
    )
    models_infos = [
        {"trials": 800, "type": "CatBoost"},
        {"trials": 800, "type": "XGB"},
        {"trials": 500, "type": "MLP"},
        {"trials": 500, "type": "LinearRegression"},
    ]
    stack_trials = 200
    k_folds = 3
    use_thought = True
    models_names = ["Stack"] + [model_info["type"] for
model_info in models_infos]
    x = df[feature_cols]
    y = df[target_cols]
    x_train, x_val, y_train, y_val = train_test_split(x, y,
test_size=0.2)
    for target in target_cols:
        y_train_target = y_train[target]
        y_val_target = y_val[target]
        errors = []
        equations = []
        models = []
        for model_info in models_infos:
            model_type = model_info["type"]
            def optimize_model(trial: optuna.Trial):
                metrics = {"mse": [], "mae": []} #, "r2": []
                for train_index, test_index in
KFold(n_splits=k_folds).split(
                    x_train, y_train_target
                ):
                    x_cv_train, x_cv_val = (
                        x_train.iloc[train_index],
                        x_train.iloc[test_index],
                    )
                    y_cv_train, y_cv_val = (
                        y_train_target.iloc[train_index],
                        y_train_target.iloc[test_index],
                    )
                    try:
                        if model_type == "CatBoost":
                            model = get_cat_boost_model_trial(trial)
                            model.fit(x_cv_train, y_cv_train,
eval_set=(x_cv_val, y_cv_val))
                        elif model_type == "XGB":
                            model = get_xgb_model_trial(trial)
                            model.fit(
                                x_cv_train,
                                y_cv_train,
                                eval_set=(
                                    (x_cv_train, y_cv_train),
                                    (x_cv_val, y_cv_val),
                                ),
                                verbose=False,
                            )

```

П2.1. Продолжение

```

elif model_type == "MLP":
    model = get_mlp_model_trial(trial)
    model.fit(x_cv_train, y_cv_train)
else:
    model = get_linear_regression_trial(trial)
    model.fit(x_cv_train, y_cv_train)
except BaseException as e:
    raise optuna.TrialPruned()
y_cv_pred = model.predict(x_cv_val)

metrics["mse"].append(mean_squared_error(y_cv_val, y_cv_pred))

metrics["mae"].append(mean_absolute_error(y_cv_val, y_cv_pred))
# metrics["r2"].append(1 - r2_score(y_cv_val,
y_cv_pred))
metric_values =
np.array(list(metrics.values())).mean(axis=1)
return 1 / np.mean(np.power(metric_values, -1))
sampler = TPESampler()
study = optuna.create_study(direction="minimize",
sampler=sampler)
study.optimize(
    optimize_model, n_trials=model_info["trials"],
show_progress_bar=True
)
if model_type == "CatBoost":
    model = get_cat_boost_model(study.best_params)
elif model_type == "XGB":
    model = get_xgb_model(study.best_params)
elif model_type == "MLP":
    model = get_mlp_model(study.best_params)
else:
    model = get_linear_regression(study.best_params)
model.fit(x_train, y_train_target)
models.append((model_type, model))
if model_type == "CatBoost":
    eq = list(zip(feature_cols, model.feature_importances_))
elif model_type == "XGB":
    eq = list(zip(feature_cols, model.feature_importances_))
elif model_type == "MLP":
    eq = [
        study.best_params["hidden_layer_sizes_" + str(i)]
        for i in range(study.best_params["n_layers"])
    ]
else:
    coefficients = model.named_steps["Reg"].coef_
    intercept = model.named_steps["Reg"].intercept_
    degrees =
model.named_steps["Pol"].get_feature_names_out()
equation_parts = [
    f"({coeff:.4f} {degree})".replace(" ", "")
    for coeff, degree in zip(coefficients, degrees)
]
eq = f"{intercept:.4f} + " + " ".join(equation_parts)
equations.append((model_type, str(eq)))
errors.append((model_type, round(study.best_value, 3)))
print(
    f"[{target}, {model_type}]:
{study.best_value:.5f}\n{study.best_params}\n"
)
with open(f"models/{target}_{model_type}.pkl", "wb")
as file:
    pickle.dump(model, file)
def optimize_stack_model(trial: optuna.Trial):
    metrics = {"mse": [], "mae": []} #, "r2": []
    for train_index, test_index in
KFold(n_splits=k_folds).split(
    x_train, y_train_target
):
    x_cv_train, x_cv_val = (x_train.iloc[train_index],
x_train.iloc[test_index])
    y_cv_train, y_cv_val = (
        y_train_target.iloc[train_index],
        y_train_target.iloc[test_index],
    )
    try:

```

```

model = StackingRegressor(
    models,
    get_linear_regression_trial(trial),
    cv="prefit",
    passthrough=use_throught,
)
model.fit(x_cv_train, y_cv_train)
except BaseException as e:
    raise optuna.TrialPruned()
y_cv_pred = model.predict(x_cv_val)
metrics["mse"].append(mean_squared_error(y_cv_val,
y_cv_pred))
metrics["mae"].append(mean_absolute_error(y_cv_val,
y_cv_pred))
# metrics["r2"].append(1 - r2_score(y_cv_val,
y_cv_pred))
metric_values =
np.array(list(metrics.values())).mean(axis=1)
return 1 / np.mean(np.power(metric_values, -1))
sampler = TPESampler()
study = optuna.create_study(direction="minimize",
sampler=sampler)
study.optimize(optimize_stack_model, n_trials=stack_trials,
show_progress_bar=True)
model = StackingRegressor(
    models,
    get_linear_regression(study.best_params),
    cv="prefit",
    passthrough=use_throught,
)
model.fit(x_train, y_train_target)
coefficients =
model.final_estimator_named_steps["Reg"].coef_
intercept =
model.final_estimator_named_steps["Reg"].intercept_
degrees =
model.final_estimator_named_steps["Pol"].get_feature_names_out(
    np.array(
        list(model.named_estimators_.keys())
        + (feature_cols if use_throught else [])
    )
)
equation_parts = [
    f"({coeff:.4f} {degree})".replace(" ", "")
    for coeff, degree in zip(coefficients, degrees)
]
eq = f"{intercept:.4f} + " + " ".join(equation_parts)
equations.append(("Stack", str(eq)))
errors.append(("Stack", round(study.best_value, 3)))
print(f"[{target},
{study.best_value:.5f}\n{study.best_params}\n")
print(eq)
with open(f"models/{target}_Stack.pkl", "wb") as file:
    pickle.dump(model, file)
with open(f"stats/errors_{target}.json", "w") as file:
    json.dump(errors, file)
with open(f"stats/cq_{target}.json", "w") as file:
    json.dump(equations, file)
for model_name in models_names:
    with open(f"models/{target}_{model_name}.pkl", "rb")
as f:
        model = pickle.load(f)
        y_target = y[target]
        y_val_pred = model.predict(x_val)
        y_pred = model.predict(x)
        stats_df = pd.concat(
            [
                stats_df,
                pd.DataFrame(
                    [
                        {
                            "TARGET": target,
                            "TYPE": "test",
                            "PARAMETER": "MAE",

```

П2.1. Продолжение

```

"VALUE": mean_absolute_error(y_val_target, y_val_pred),
    },
    {
        "TARGET": target,
        "TYPE": "test",
        "PARAMETER": "MSE",
        "VALUE":
mean_squared_error(y_val_target, y_val_pred),
    },
    {
        "TARGET": target,
        "TYPE": "test",
        "PARAMETER": "R2",
        "VALUE":
r2_score(y_val_target,
y_val_pred),
    },
    {
        "TARGET": target,
        "TYPE": "all",
        "PARAMETER": "MAE",
        "VALUE":
mean_absolute_error(y_target,
y_pred),
    },
    {
        "TARGET": target,
        "TYPE": "all",
        "PARAMETER": "MSE",
        "VALUE":
mean_squared_error(y_target,
y_pred),
    },
    {
        "TARGET": target,
        "TYPE": "all",
        "PARAMETER": "R2",
        "VALUE":
r2_score(y_target, y_pred),
    },
    ],
    ),
    ],
    ignore_index=True,
)
df["{target}_{model_name}_PRED"] = y_pred
with pd.ExcelWriter("data/xlsx/results/stack_results.xlsx") as
writer:
    df.to_excel(writer, sheet_name="data")
    stats_df.to_excel(writer, sheet_name="stats", index=False)

```

Текст модуля inference_oracle представлен на рис. П2.2.

```

from datetime import datetime, timedelta
import json
import pickle
import time
import pandas as pd
import schedule
import cx_Oracle
import os
from dotenv import load_dotenv
d = r"D:\instantclient_21_6"
cx_Oracle.init_oracle_client(lib_dir=d)
load_dotenv()
feature_cols = [
    "GRAN_0_10",
    "GRAN_10_25",
    "GRAN_25_40",
    "GRAN_40_60",
    "GRAN_60_80",
    "GRAN_80_999",
]
target_cols = ["M10", "M40"]
models_names = ["LinearRegression", "CatBoost", "XGB",
"Stack"]
models_infos = {}
for target in target_cols:
    models_infos[target] = {}
    for model_name in models_names:
        with open(f"models/{target}_{model_name}.pkl", "rb")
as f:
        models_infos[target][model_name] = pickle.load(f)
def predict(row: pd.Series):
    return {
        target: {
            model_name:
model.predict(pd.DataFrame(row).T).item()
            for model_name, model in target_models_info.items()
        }
        for target, target_models_info in models_infos.items()
    }
def filter_and_map_kd_data(df: pd.DataFrame, kd_name: str):
    df = df.copy()
    df = df.pivot_table(
        index=["EVENT_ID", "DATE"],
        columns="ATTRIB_CODE",
        values=["DOUBLE_VALUE", "STRING_VALUE"],
        aggfunc="first",
    )
    df = df.reset_index()
    df = df[
        [
            ("EVENT_ID", ""),
            ("DATE", ""),
            ("DOUBLE_VALUE", "GRAN_0_10"),
            ("DOUBLE_VALUE", "GRAN_10_25"),
            ("DOUBLE_VALUE", "GRAN_25_40"),
            ("DOUBLE_VALUE", "GRAN_40_60"),
            ("DOUBLE_VALUE", "GRAN_60_80"),
            ("DOUBLE_VALUE", "GRAN_80_999"),
            ("STRING_VALUE", "CAMERA_NO"),
        ]
    ]
    df.columns = [
        "EVENT_ID",
        "DATE",
        "GRAN_0_10",
        "GRAN_10_25",
        "GRAN_25_40",
        "GRAN_40_60",
        "GRAN_60_80",
        "GRAN_80_999",
        "CAMERA_NO",
    ]
    uniq_cameras = sorted(pd.unique(df["CAMERA_NO"]),
reverse=True)
cameras_index = {cam: i for i, cam in
enumerate(uniq_cameras)}
df["EVENT_ID"] = df.apply(
    lambda x: x["EVENT_ID"] -
cameras_index[x["CAMERA_NO"]], axis=1
)
df = df.groupby("EVENT_ID").filter(lambda x: len(x) ==
len(uniq_cameras))
df = df.groupby("EVENT_ID").agg(
    {
        "GRAN_0_10": "sum",
        "GRAN_10_25": "sum",
        "GRAN_25_40": "sum",
        "GRAN_40_60": "sum",
        "GRAN_60_80": "sum",
        "GRAN_80_999": "sum",
        "DATE": "mean",
    }
)

```

Рис. П2.2. Текст модуля inference_oracle

П2.2. Продолжение

```

    }
    )
    df = df.reset_index()
    df = df.drop(["EVENT_ID"], axis=1)
    gran_cols = [
        "GRAN_0_10",
        "GRAN_10_25",
        "GRAN_25_40",
        "GRAN_40_60",
        "GRAN_60_80",
        "GRAN_80_999",
    ]
    sum_df = df[gran_cols].sum(axis=1)
    df = df[sum_df > 30]
    for col in gran_cols:
        df[col] = df[col] / sum_df
    df = df[(df[gran_cols] <= 0.7).all(axis=1)]
    # df = df.drop(["GRAN_80_999"], axis=1)
    # df["GRAN_40_80"] = df["GRAN_40_60",
"GRAN_60_80"].sum(axis=1)
    # df = df.drop(["GRAN_40_60", "GRAN_60_80"], axis=1)
    if df.empty:
        raise Exception(f"{kd_name} is empty")
    print(f"[SRC, {kd_name}]")
    print(df)
    print(f"[MEAN, {kd_name}]")
    df = df.mean().to_frame().T
    df[feature_cols] = df[feature_cols].astype(float)
    print(df)
    return df

def get_connection():
    return
cx_Oracle.connect(os.getenv("CONNECTION_STRING"))
kd_types = {
    "EVENT_ATTRID_ID": int,
    "EVENT_ID": int,
    "ATTRIB_CODE": str,
    "STRING_VALUE": str,
    "DOUBLE_VALUE": float,
    "EVENT_DATE": str,
}
def get_data_from_oracle(query, params):
    conn = None
    cursor = None
    try:
        conn = get_connection()
        cursor = conn.cursor()
        output_cursor = cursor.var(cx_Oracle.CURSOR)
        cursor.execute(query, cur=output_cursor, **params)
        result = output_cursor.getvalue()
        df = pd.DataFrame(result.fetchall(),
columns=kd_types.keys())
        df = df.astype(kd_types)
        df["EVENT_DATE"] =
pd.to_datetime(df["EVENT_DATE"])
        df = df.rename({"EVENT_DATE": "DATE"}, axis=1)
        return df
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()
    def get_kd4_data_from_oracle(params):
        query_kd4 = "BEGIN :cur :=
kadp_cv_model.PKG GRANULO_KD.FNC SELECT_EVENT_BE
TWEEN_DATES_KD4(:pBeginDate, :pEndDate); END;"
        return get_data_from_oracle(query_kd4, params)
    def get_kd19_data_from_oracle(params):
        query_kd19 = "BEGIN :cur :=
kadp_cv_model.PKG GRANULO_K.FNC SELECT_EVENT_BE
TWEEN_DATES_K19(:pBeginDate, :pEndDate); END;"
        return get_data_from_oracle(query_kd19, params)
    def write_data_to_oracle(result):
        conn = None
        cursor = None
        try:
            conn = get_connection()
            cursor = conn.cursor()
            query = "BEGIN
kadp_cv_model.PKG GRANULO_KD.ins_strength(:p1, :p2, :p3, :p
4, :p5, :p6, :p7, :p8); END;"
            cursor.execute(
                query,
                p1=result["M10"]["LinearRegression"],
                p2=result["M10"]["CatBoost"],
                p3=result["M10"]["XGB"],
                p4=result["M40"]["LinearRegression"],
                p5=result["M40"]["CatBoost"],
                p6=result["M40"]["XGB"],
                p7=result["M10"]["Stack"],
                p8=result["M40"]["Stack"],
            )
        finally:
            if cursor:
                cursor.close()
            if conn:
                conn.close()
    n_minutes = 15
    kd_delta = timedelta(minutes=11, seconds=30)
    def func():
        try:
            params_kd4 = {
                "pBeginDate": (datetime.now() -
timedelta(minutes=n_minutes)),
                "pEndDate": datetime.now(),
            }
            params_kd19 = {
                "pBeginDate": (params_kd4["pBeginDate"] - kd_delta),
                "pEndDate": params_kd4["pEndDate"] - kd_delta,
            }
            for params in [params_kd4, params_kd19]:
                for param in ["pBeginDate", "pEndDate"]:
                    params[param] = params[param].strftime("%d-%m-
%Y %H:%M:%S")
            print("start get queries")
            kd4_start_df = get_kd4_data_from_oracle(params_kd4)
            kd19_start_df =
get_kd19_data_from_oracle(params_kd19)
            print("end get queries")
            print("start filter")
            kd4_df = filter_and_map_kd_data(kd4_start_df, "KD4")
            kd19_df = filter_and_map_kd_data(kd19_start_df,
"KD19")
            print("end filter")
            feature_df: pd.Series = (kd4_df - kd19_df).squeeze()
            print("[INPUT]")
            print(feature_df[feature_cols])
            print("start inference")
            result = predict(feature_df[feature_cols].astype(float))
            print("end inference")
            print(result)
            print("start set query")
            write_data_to_oracle(result)
            print("end set query")
        except Exception as e:
            print(f"Exception: {e}")
    schedule.every(n_minutes).minutes.do(func)
    while True:
        schedule.run_pending()
        time.sleep(1)

```

Текст модуля join_data представлен на рис. П2.3.

```

from math import log, sqrt
import numpy as np
import pandas as pd
from optuna.samplers import TPESampler
import logging
import optuna
from scipy.special import expit
logging.getLogger("optuna").setLevel(logging.ERROR)

kd4_df =
pd.read_parquet("data/parq/src/kd4_mapped.parquet",
engine="fastparquet")

kd19_df =
pd.read_parquet("data/parq/src/kd19_mapped.parquet",
engine="fastparquet")

target_df = pd.read_parquet("data/parq/src/target.parquet",
engine="fastparquet")

time_delta = pd.Timedelta(minutes=11, seconds=30)
# kd4_df["DATE"] = kd4_df["DATE"].dt.round(time_delta)
# kd19_df["DATE"] = kd19_df["DATE"].dt.round(time_delta)
kd4_df = kd4_df.sort_values(by=["DATE"], ascending=True)
kd19_df = kd19_df.sort_values(by=["DATE"],
ascending=True)

merged_df = pd.merge_asof(
    kd4_df,
    kd19_df,
    on="DATE",
    tolerance=pd.Timedelta(minutes=1),
    allow_exact_matches=True,
    suffixes=("_kd4", "_kd19"),
)

merged_df = merged_df.dropna()
# merged_df.to_excel("data/img/merged.xlsx")
# exit()

feature_df = pd.DataFrame()
feature_df["DATE"] = merged_df["DATE"]
feature_cols = [
    "GRAN_0_10",
    "GRAN_10_25",
    "GRAN_25_40",
    "GRAN_40_60",
    "GRAN_60_80",
    "GRAN_80_999",
]

for col in feature_cols:
    feature_df[col] = merged_df[f"{col}_kd4"] -
merged_df[f"{col}_kd19"]
    # feature_df = feature_df.loc[
    #     ((feature_df[feature_cols] > 0) ==
(feature_df[feature_cols].mean() > 0)).all(
    #     axis=1
    # )
    # ]
    feature_df["DATE"] -= pd.Timedelta(hours=4)
    feature_df["DATE"] =
feature_df["DATE"].dt.floor(pd.Timedelta(days=1))
    feature_df["RND_BY_DAY"] = np.random.randint(0, 3,
feature_df.shape[0])
    feature_df: pd.DataFrame = feature_df.groupby(["DATE",
"RND_BY_DAY"]).agg(
        {
            "GRAN_0_10": "mean",
            "GRAN_10_25": "mean",
            "GRAN_25_40": "mean",
            "GRAN_40_60": "mean",
            "GRAN_60_80": "mean",
            "GRAN_80_999": "mean",
        }
    )
    feature_df = feature_df.reset_index()
    feature_df = feature_df.drop(["RND_BY_DAY"], axis=1)
    feature_df = feature_df.sort_values(by=["DATE"],
ascending=True)
    target_df = target_df.sort_values(by=["DATE"],
ascending=True)
    df = pd.merge_asof(
        feature_df,
        target_df,
        on="DATE",
        tolerance=pd.Timedelta(0),
        allow_exact_matches=True,
    )
    df = df.set_index("DATE")
    df = df.dropna()
    df.to_parquet("data/parq/kd_dataset.parquet",
engine="fastparquet")
    df.to_excel("data/xlsx/kd_dataset.xlsx")

```

Рис. П2.3. Текст модуля join_data

Текст модуля rock_publisher представлен на рис. П2.4.

```

import base64
import glob
from itertools import combinations
from detectron2.engine import DefaultPredictor
import json
import pickle
import os
import cv2
import numpy as np
import zmq
from time import sleep
import torch
from pypylon import pylon
from dto import *
torch.cuda.empty_cache()
# rock_scale = 0.06025
rock_scale = 0.09018
show_frame = True
cfg_save_path = "IS_rocks_cfg.pickle"
with open(cfg_save_path, "rb") as f:
    cfg = pickle.load(f)
cfg.MODEL.WEIGHTS = os.path.join(cfg.OUTPUT_DIR,
"model_0049999.pth")
#         cfg.MODEL.WEIGHTS
'dataset/old_weights/model_final.pth'
cfg.MODEL.ROI_HEADS.SCORE_THRESH_TEST = 0.5
predictor = DefaultPredictor(cfg)
camera =
pylon.InstantCamera(pylon.TIFactory.GetInstance()).CreateFirstDevice()
camera.StartGrabbing(pylon.GrabStrategy_LatestImageOnly)
converter = pylon.ImageFormatConverter()
converter.OutputPixelFormat =
pylon.PixelType_BGR8packed
converter.OutputBitAlignment =
pylon.OutputBitAlignment_MsbAligned
context = zmq.Context()
with context.socket(zmq.PUB) as socket:
    socket.bind("tcp://127.0.0.1:7777")
    while camera.IsGrabbing():
        grabResult = camera.RetrieveResult(5000,
pylon.TimeoutHandling_ThrowException)
        if not grabResult.GrabSucceeded():
            continue
        image = converter.Convert(grabResult)
        mat = image.GetArray()
        outputs = predictor(mat)
        predictions = outputs["instances"].to("cpu")
        detectonResult = get_result(predictions, mat.shape[0],
mat.shape[1])
        for detection in detectonResult.detections:
            for pol in detection.polygon:
                contour = [[p.x, p.y] for p in pol]
                contour = np.array(contour).reshape((-1, 1, 2))
                cv2.drawContours(mat, [contour], -1, (0, 0, 255), 6)
            polygones = [detection.polygon for detection in
detectonResult.detections]
            sizes = []
            for polygone in polygones:
                for pol in polygone:
                    np.int32)
                    contour = np.array([np.array([p.x, p.y]) for p in pol],
                    epsilon = 0.001 * cv2.arcLength(contour, True)
                    approx = cv2.approxPolyDP(contour, epsilon, True)
                    max_value = np.max(
                        np.array(
                            [np.linalg.norm(a - b) for a, b in
combinations(approx, 2)]
                        )
                    )
                    sizes.append(rock_scale * max_value)
                sizes = [size for size in sizes if size > 5]
                res = {
                    "measure": [
                        {
                            "id": 1,
                            "meta": "0-10 mm",
                            "value": len(list(filter(lambda x: x < 10, sizes))),
                        },
                        {
                            "id": 2,
                            "meta": "10-25 mm",
                            "value": len(list(filter(lambda x: 10 <= x < 25,
sizes))),
                        },
                        {
                            "id": 3,
                            "meta": "25-40 mm",
                            "value": len(list(filter(lambda x: 25 <= x < 40,
sizes))),
                        },
                        {
                            "id": 4,
                            "meta": "40-60 mm",
                            "value": len(list(filter(lambda x: 40 <= x < 60,
sizes))),
                        },
                        {
                            "id": 5,
                            "meta": "60-80 mm",
                            "value": len(list(filter(lambda x: 60 <= x < 80,
sizes))),
                        },
                        {
                            "id": 6,
                            "meta": "80+ mm",
                            "value": len(list(filter(lambda x: 80 <= x, sizes))),
                        },
                    ],
                    "total_count": len(sizes),
                    "image": base64.b64encode(cv2.imencode(".jpg",
mat)[1].tobytes()).decode(
                        "ascii"
                    ),
                }
                with open("data.json", "w") as f:
                    json.dump(res, f)
            if show_frame:
                cv2.imshow("Rocks", mat)
                cv2.waitKey(30)
                socket.send_json(res)

```

Рис. П2.4. Текст модуля rock_publisher

Текст модуля train представлен на рис. П2.5.

```
import pickle
import os
from detectron2.utils.logger import setup_logger
from rockDatasets import *
import torch
from utils import *
import torch
from trainer import Trainer
setup_logger()
config_file_path = "COCO-
InstanceSegmentation/mask_rcnn_R_50_FPN_3x.yaml"
checkpoint_url = config_file_path
output_dir = "output/object_segmentation"
num_classes = 1
train_device = "cuda" if torch.cuda.is_available() else "cpu"

#train_device = "cpu"
def main():
    torch.cuda.empty_cache()
    cfg = get_train_cfg(config_file_path, checkpoint_url,
train_dataset_name,
test_dataset_name, num_classes, train_device,
output_dir)
    with open(cfg_save_path, 'wb') as f:
        pickle.dump(cfg, f)
    os.makedirs(cfg.OUTPUT_DIR, exist_ok=True)
    trainer = Trainer(cfg)
    trainer.resume_or_load(resume=True)
    trainer.train()
if __name__ == '__main__':
    main()
```

Рис. П2.5. Текст модуля train

Текст модуля trainer представлен на рис. П2.6.

```
import imp
import os
import torch
from detectron2.data import detection_utils as utils
import copy
import detectron2.data.transforms as T
import detectron2.utils.comm as comm
from detectron2.data import MetadataCatalog,
build_detection_train_loader
from detectron2.engine import DefaultTrainer
from randomBackground import RandomBackground
from negative import Negative
from detectron2.evaluation import (
    CityscapesInstanceEvaluator,
    CityscapesSemSegEvaluator,
    COCOEvaluator,
    DatasetEvaluators,
    LVISEvaluator,
    SemSegEvaluator,
)
def custom_mapper(dataset_dict):
    dataset_dict = copy.deepcopy(dataset_dict)
    image = utils.read_image(dataset_dict["file_name"],
format="BGR")
    transform_list = [
        T.RandomApply(T.RandomCrop(
            "relative_range", (0.4, 0.4)), prob=0.15),
        T.RandomApply(T.RandomRotation(
            angle=list(range(10, 180, 10)), sample_style="choice"),
prob=0.15),
        T.Resize((850, 850)),
        T.RandomFlip(prob=0.25, horizontal=True,
vertical=False),
        T.RandomFlip(prob=0.25, horizontal=False,
vertical=True),
        T.RandomApply(T.RandomContrast(0.5, 1.5), prob=0.2),
        T.RandomApply(T.RandomBrightness(0.5, 1.5),
prob=0.2),
    ]
    image, transforms = T.apply_transform_gens(transform_list,
image)
    dataset_dict["image"] = torch.as_tensor(
        image.transpose(2, 0, 1).astype("float32"))
    annos = [
        utils.transform_instance_annotations(
            obj, transforms, image.shape[:2])
        for obj in dataset_dict.pop("annotations")
        if obj.get("iscrowd", 0) == 0
    ]
    instances = utils.annotations_to_instances(annos,
image.shape[:2])
    dataset_dict["instances"] =
utils.filter_empty_instances(instances)
    return dataset_dict
class Trainer(DefaultTrainer):
    @classmethod
    def build_evaluator(cls, cfg, dataset_name,
output_folder=None):
        if output_folder is None:
            output_folder = os.path.join(cfg.OUTPUT_DIR,
"inference")
        return COCOEvaluator(dataset_name, cfg, False,
output_dir=output_folder)
    @classmethod
    def build_train_loader(cls, cfg):
        return build_detection_train_loader(cfg,
mapper=custom_mapper)
```

Рис. П2.6. Текст модуля trainer

Текст модуля Program представлен на рис. П2.7.

```

using Gran.Server.Abstractions;
using Gran.Server.Auth;
using Gran.Server.Configuration;
using Gran.Server.Database;
using Gran.Server.Database.Abstractions;
using Gran.Server.Database.Entities;
using Gran.Server.Database.Implementation;
using Gran.Server.Implementations;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using System.Security.Claims;
var builder = WebApplication.CreateBuilder(args);
builder.WebHost.UseKestrel();
builder.Services.AddOptions<GranOptions>()
    .BindConfiguration(GranOptions.Section);
builder.Services.AddOptions<AuthInputOptions>()
    .BindConfiguration(AuthInputOptions.Section);
builder.Services.AddHostedService<IterateCurrentBackgroundService>()
    .AddScoped<IGranulService, GranulService>()
    .AddScoped<IEncryptionService, EncryptionService>()
    .AddScoped<IUserService, UserService>();
builder.Services.AddMemoryCache();
builder.Services.AddControllers();
builder.Services.AddScoped(typeof(IRepo<,>),
    typeof(ProductDbRepo<,>))
    .AddScoped<IUserRepo, UserRepo>()
    .AddScoped<IRepo<User, int>>(s =>
        s.GetRequiredService<IUserRepo>());
builder.Services.AddCors(options =>
    options.AddDefaultPolicy(policy => policy.AllowAnyOrigin()
        .AllowAnyHeader()));
builder.Services.AddEndpointsApiExplorer()
    .AddSwaggerGen(setup =>
    {
        OpenApiSecurityScheme jwtSecurityScheme =
        new()
        {
            BearerFormat = "JWT",
            Name = "JWT Authentication",
            In = ParameterLocation.Header,
            Type = SecuritySchemeType.Http,
            Scheme =
            JwtBearerDefaults.AuthenticationScheme,
            Description = "Put **_ONLY_** your JWT
            Bearer token on textbox below!",
            Reference = new OpenApiReference
            {
                Id =
                JwtBearerDefaults.AuthenticationScheme,
                Type = ReferenceType.SecurityScheme
            }
        };
        setup.AddSecurityDefinition(jwtSecurityScheme.Reference.Id,
            jwtSecurityScheme);
        setup.AddSecurityRequirement(new
            OpenApiSecurityRequirement
            {
                { jwtSecurityScheme, [] }
            });
    });
    });
builder.Services
    .AddOptions<JwtBearerOptions>(JwtBearerDefaults.AuthenticationScheme)
    .Configure<IOptions<AuthInputOptions>>((options,
        authInputOptions) =>
    {
        options.TokenValidationParameters = new
            TokenValidationParameters
            {
                ValidateIssuer = true,
                ValidIssuer = authInputOptions.Value.Issuer,
                ValidateAudience = true,
                ValidAudience = authInputOptions.Value.Audience,
                ValidateLifetime = true,
                IssuerSigningKey =
                authInputOptions.Value.SymmetricSecurityKey,
                ValidateIssuerSigningKey = true,
            };
    }).Configure<IServiceProvider>((options,
        serviceProvider) =>
    {
        options.Events = new JwtBearerEvents
        {
            OnTokenValidated = async context =>
            {
                await using var scope =
                serviceProvider.CreateAsyncScope();
                var userService =
                scope.ServiceProvider.GetRequiredService<IUserService>();
                var identity = context.Principal.Identity as
                ClaimsIdentity;
                string login =
                identity.FindFirst(Claims.Login).Value;
                string password =
                identity.FindFirst(Claims.HashedPassword).Value;
                if (!await userService.UserExist(login, password,
                    default))
                {
                    context.Fail("user doesnt exist");
                }
            };
        });
    });
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer();
builder.Services.AddDbContext<ProductsDbContext>(options
    =>
    {
        options.UseNpgsql(builder.Configuration.GetConnectionString("Database"));
    });
var app = builder.Build();
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseCors();
app.UseAuthorization();
app.MapControllers();
app.Run();

```

Рис. П2.7. Текст модуля Program

Текст модуля UserService представлен на рис. П2.8.

```

using Gran.Server.Abstractions;
using Gran.Server.Auth;
using Gran.Server.Configuration;
using Gran.Server.Database.Abstractions;
using Gran.Server.Database.Entities;
using Gran.Server.Dto;
using Gran.Server.Exceptions;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
namespace Gran.Server.Implementations;
internal class UserService(IUserRepo userRepo,
IOptions<AuthInputOptions> authInputOption, IEncryptionService
encryptionService) : IUserService
{
    public async Task<UserInfoDto> GetByLogin(string login,
CancellationToken cancellationToken)
    {
        var existUser = await userRepo.FindByLogin(login,
cancellationToken) ?? throw new NotFoundException();
        return new UserInfoDto
        {
            Login = existUser.Login,
            Name = existUser.Name,
            Id = existUser.Id,
        };
    }
    public async Task<UserLoginInfoDto>
Login(UserLoginDto user, Cancellation token)
    {
        var existUser = await userRepo.FindByLogin(user.Login,
cancellationToken) ?? throw new NotFoundException();
        var hashedPassword = encryptionService.Encrypt(new
EncryptionData
        {
            Login = user.Login,
            Password = user.Password,
            GeneralSalt = authInputOption.Value.GeneralSalt,
            Salt = existUser.Salt,
        });
        if (existUser.Password != hashedPassword)
        {
            throw new WrongPasswordException();
        }
        DateTime now = DateTime.Now;
        JwtSecurityToken jwt = new
        (
            issuer: authInputOption.Value.Issuer,
            audience: authInputOption.Value.Audience,
            notBefore: now,
            expires:
now.Add(TimeSpan.FromMinutes(authInputOption.Value.TokenLife
TimeInMinutes)),
            claims: [new(Claims.Login, existUser.Login),
new(Claims.HashedPassword, hashedPassword)],
            signingCredentials:
new(authInputOption.Value.SymmetricSecurityKey,
SecurityAlgorithms.HmacSha256)
            );
        return new UserLoginInfoDto
        {
            User = new UserInfoDto
            {
                Login = existUser.Login,
                Name = existUser.Name,
                Id = existUser.Id,
            },
            Token = new
new
JwtSecurityTokenHandler().WriteToken(jwt)
        };
    }
    public async Task Register(UserRegistrationDto user,
CancellationToken cancellationToken)
    {
        if (await userRepo.FindByLogin(user.Login,
cancellationToken) is not null)
        {
            throw new UserExistException();
        }
        string salt = new
encryptionService.GenerateSalt(authInputOption.Value.SaltSize);
        var hash = encryptionService.Encrypt(new
EncryptionData
        {
            Login = user.Login,
            Password = user.Password,
            GeneralSalt = authInputOption.Value.GeneralSalt,
            Salt = salt,
        });
        userRepo.Add(new User
        {
            Login = user.Login,
            Name = user.Name,
            Password = hash,
            Salt = salt
        });
        await userRepo.SaveChanges(cancellationToken);
    }
    public async Task<bool> UserExist(string login, string
hashedPassword, Cancellation token)
    {
        var existUser = await userRepo.FindByLogin(login,
cancellationToken);
        return existUser is not null && existUser.Password ==
hashedPassword;
    }
}

```

Рис. П2.8. Текст модуля UserService

Текст модуля LoginPage представлен на рис. П2.9.

```

import Button from '@mui/material/Button';
import TextField from '@mui/material/TextField';
import Box from '@mui/material/Box';
import Typography from '@mui/material/Typography';
import Container from '@mui/material/Container';
import { Grid } from '@mui/material';
import { Link, useNavigate, } from 'react-router-dom';
import { UserLoginDto } from 'types/dtoTypes';
import { useBearerSignIn } from 'hooks/authHooks';
import { useIsAuthenticated } from 'react-auth-kit';
import { useEffect, useState } from 'react';

import { useUserStore } from 'store/userStore';
export default function LoginPage() {
    const signIn = useBearerSignIn();
    const navigate = useNavigate();
    const isAuthenticated = useIsAuthenticated();
    const login = useUserStore(state => state.login)
    const [error, setError] = useState<{ isError: boolean, error?:
string }>({ isError: false })
    useEffect(() => {
        if (isAuthenticated()) {
            navigate("/");
        }
    }, [isAuthenticated]);
}

```

Рис. П2.9. Текст модуля LoginPage

```

import { Grid, Paper, Typography } from "@mui/material";
import ImageFromCamera from "components/Gran/ImageFromCamera";
import { useInterval } from "hooks/themeHooks";
import { useAuthHeader } from "react-auth-kit";
import {
  useQuery,
  useQueryClient,
} from "react-query";
import { useParams, useSearchParams } from "react-router-dom";
import GranService from "services/userService copy";
export default function ReportPage() {
  const getToken = useAuthHeader();
  const { reportDate } = useParams();
  const { data: current } = useQuery(
    ["report", reportDate],
    async () => (await GranService.getReport(reportDate!,
  getToken())).data
  );

  return (
    <
      <Grid
        container
        justifyContent='center'
        alignItems='center'
        spacing={2}
      >
        <Grid item xs={12} justifySelf='center'>
          <Grid container justifyContent='center'>
            <Paper sx={{ p: 2 }}>
              Отчет за {current.date}
            </Paper>
          </Grid>
        </Grid>
        <Grid item xs={6}>
          <ImageFromCamera name="K19"
            images={['k19/${current.filePathFromK19}']>
        </Grid>
      </Grid>
    </
  >
  );
}

```

Рис. П2.10. Текст модуля ReportPage

```

granFrom0To10={current.granFrom0To10FromK19}

granFrom10To25={current.granFrom10To25FromK19}

granFrom25To40={current.granFrom25To40FromK19}

granFrom40To60={current.granFrom40To60FromK19}

granFrom60To80={current.granFrom60To80FromK19}

granFrom80To999={current.granFrom80To999FromK19}
  isCorrect={current.isK19Correct} />
</Grid>
<Grid item xs={6}>
  <ImageFromCamera name="Kd4"
images={['kd4-0/${current.filePathFromKd4Camera0}', 'kd4-
1/${current.filePathFromKd4Camera1}']]

granFrom0To10={current.granFrom0To10FromKd4}

granFrom10To25={current.granFrom10To25FromKd4}

granFrom25To40={current.granFrom25To40FromKd4}

granFrom40To60={current.granFrom40To60FromKd4}

granFrom60To80={current.granFrom60To80FromKd4}

granFrom80To999={current.granFrom80To999FromKd4}
  isCorrect={current.isKd4Correct} />
</Grid>
<Grid item xs={6}>
  <Paper sx={{ p: 3 }}>
    {current.isCorrect ? <
gutterBottom>
  {`Разница содержания фракций`}
</Typography>
<Grid container
  alignItems='center'
  spacing={1}>
  <Grid item xs>
    <Typography variant='subtitle2'>0-
10: {(current.granFrom0To10! * 100).toFixed(1)} %</Typography>
  </Grid>
  <Grid item xs>
    <Typography
variant='subtitle2'>10-25: {(current.granFrom10To25!
100).toFixed(1)} %</Typography>
  </Grid>
  <Grid item xs>
    <Typography
variant='subtitle2'>25-40: {(current.granFrom25To40!
100).toFixed(1)} %</Typography>
  </Grid>
  <Grid item xs>

```

П2.10. Продолжение

```

<Typography variant='subtitle2'>40-60: {(current.granFrom40To60!
* 100).toFixed(1)} %</Typography>
</Grid>
<Grid item xs>
  <Typography
variant='subtitle2'>60-80: {(current.granFrom60To80!
100).toFixed(1)} %</Typography>
</Grid>
<Grid item xs>
  <Typography
variant='subtitle2'>80-999: {(current.granFrom80To999!
100).toFixed(1)} %</Typography>
</Grid>
</Grid>
</> :
  <Typography variant='h6' height={50}
gutterBottom color='red'>
  {`Разница фракций не может
  быть рассчитана`}
</Typography>
</>
</Paper>
</Grid>
<Grid item xs={6}>
  <Paper sx={{ p: 3 }}>
    {current.isCorrect ? <
gutterBottom>
  {`Прочностные характеристики`}
</Typography>
<Grid container
  alignItems='center'
  spacing={1}>
  <Grid item xs={2}>
    <Typography
variant='subtitle2'>M10: {(current.m10Stack ?? 0).toFixed(1)}
</Typography>
  </Grid>
  <Grid item xs={2}>
    <Typography
variant='subtitle2'>M40: {(current.m40Stack ??
0).toFixed(1)}</Typography>
  </Grid>
  </Grid>
</> :
  <Typography variant='h6' height={50}
gutterBottom color='red'>
  {`Прочностные характеристики
не могут быть рассчитаны`}
</Typography>
</>
</Paper>
</Grid>
</Grid>
</Grid>
</>
</>
  )
}

```


Текст модуля ReportsPage представлен на рис. П2.11.

```

import { Link, Paper, Table, TableBody, TableCell,
TableContainer, TableHead, TablePagination, TableRow, Tooltip }
from "@mui/material";
import { useInterval } from "hooks/themeHooks";
import { useMemo } from "react";
import { useAuthHeader } from "react-auth-kit";
import {
  useQuery,
  useQueryClient,
} from 'react-query';
import GranService from "services/userService copy";
import { useQueryParam } from "use-query-params";
export default function ReportsPage() {
  const getToken = useAuthHeader();
  const [pageParam, setPage] = useQueryParam<string |
undefined>("page")
  const page = useMemo(() => parseInt(pageParam ?? '0'),
[pageParam])
  const [rowsPerPageParam, setRowsPerPage] =
useQueryParam<string | undefined>("rowsPerPage")
  const rowsPerPage = useMemo(() =>
parseInt(rowsPerPageParam ?? '50'), [rowsPerPageParam])
  const client = useQueryClient()
  const { data: reports } = useQuery
  (
    ["reports"],
    async () => (await
GranService.getReports(getToken())).data
  );
  const paginationReports = useMemo(() =>
reports?.slice(page * rowsPerPage, page * rowsPerPage +
rowsPerPage), [reports, page, rowsPerPage])
  return (
    <
      {
        (reports &&
          <TableContainer component={Paper}>
            <Table>
              <TableHead>
                <TableRow>
                  <TableCell align="left"></TableCell>
                  <TableCell
align="center">Дара</TableCell>
10</TableCell>
25</TableCell>
40</TableCell>
60</TableCell>
80</TableCell>
999</TableCell>
                </TableRow>
              </TableHead>
                <TableBody>
                  {paginationReports?.map((report) => (
                    <TableRow>
                      <TableCell align="left">
                        <img
src={`$${import.meta.env.VITE_API_ADDRESS}/api/gran/kd4-
0/${report.filePathFromKd4Camera0}`} style={{ width: 60, height:
40 }} />
                        </TableCell>
                        <TableCell align="center">
                          <Link
href={`reports/${report.date}`}>
                            {report.date}
                          </Link>
                        </TableCell>
                        <TableCell
align="center">{report.isCorrect ? `${(report.granFrom0To10! *
100).toFixed(1)} %` :
"Отсутствует"}</TableCell>
                        <TableCell
align="center">{report.isCorrect ? `${(report.granFrom10To25! *
100).toFixed(1)} %` :
"Отсутствует"}</TableCell>
                        <TableCell
align="center">{report.isCorrect ? `${(report.granFrom25To40! *
100).toFixed(1)} %` :
"Отсутствует"}</TableCell>
                        <TableCell
align="center">{report.isCorrect ? `${(report.granFrom40To60! *
100).toFixed(1)} %` : "Отсутствует"}</TableCell>
                        <TableCell
align="center">{report.isCorrect ? `${(report.granFrom60To80! *
100).toFixed(1)} %` : "Отсутствует"}</TableCell>
                        <TableCell
align="center">{report.isCorrect ? `${(report.granFrom80To999! *
100).toFixed(1)} %` : "Отсутствует"}</TableCell>
                      </TableRow>
                    )))
                  </TableBody>
                </Table>
                <TablePagination
page={page}
count={reports.length}
onPageChange={(_ , p) => {
return setPage(p.toString(), 'pushIn');
}}
rowsPerPage={rowsPerPage}
onRowsPerPageChange={(e) =>
setRowsPerPage(e.target.value)} />
              </TablePagination>
            </TableContainer>
          )
        }
      </>
    )
  )
}

```

Рис. П2.11. Текст модуля ReportsPage

Текст модуля GranPage представлен на рис. П2.12.

```

import { Button, Grid, IconButton, Paper, Typography } from
"@mui/material";
import ImageFromCamera from
"components/Gran/ImageFromCamera";
import { useInterval } from "hooks/themeHooks";
import { useState } from "react";
import { useAuthHeader } from "react-auth-kit";
import {
  useQuery,
  useQueryClient,
} from 'react-query';
import GranService from "services/userService copy";
import PauseIcon from '@mui/icons-material/Pause';
import PlayArrowIcon from '@mui/icons-material/PlayArrow';
export default function GranPage() {
  const getToken = useAuthHeader();
  const [paused, setPaused] = useState(false);
  const client = useQueryClient()
  const { data: current } = useQuery
    ( ["actual"], async () => (await
GranService.getCurrentGran(getToken()))),data,
    {
      enabled: !paused
    } );
  useInterval(() => {
    if (paused) {
      return
    }
    client.invalidateQueries(["actual"])
  }, 500, [paused])
  return (
    <Grid container
spacing={2} justifyContent='center'>
      <Grid item
xs={12} justifySelf='center'>
        <Grid container justifyContent='center'
alignItems='center'>
          <Paper sx={{ p: 2 }}>
            Актуальное состояние на
{current.date}
            <IconButton
color="primary"
onClick={() => setPaused(s => !s)}
>
              {paused ? <PlayArrowIcon /> :
<PauseIcon/></PauseIcon>}
            </IconButton>
          </Paper>
        </Grid>
      </Grid>
      <Grid item xs={6}>
        <ImageFromCamera name="K19"
images={['k19/$ {current.filePathFromK19}']
granFrom0To10={current.granFrom0To10FromK19}
granFrom10To25={current.granFrom10To25FromK19}
granFrom25To40={current.granFrom25To40FromK19}
granFrom40To60={current.granFrom40To60FromK19}
granFrom60To80={current.granFrom60To80FromK19}
granFrom80To999={current.granFrom80To999FromK19}
isCorrect={current.isK19Correct} />
      </Grid>
      <Grid item xs={6}>
        <ImageFromCamera name="Kd4"
images={['kd4-0/$ {current.filePathFromKd4Camera0}', `kd4-
1/$ {current.filePathFromKd4Camera1}`]}
granFrom0To10={current.granFrom0To10FromKd4}
granFrom10To25={current.granFrom10To25FromKd4} granFrom25
To40={current.granFrom25To40FromKd4} granFrom40To60={curre
nt.granFrom40To60FromKd4}
granFrom60To80={current.granFrom60To80FromKd4}
granFrom80To999={current.granFrom80To999FromKd4}
isCorrect={current.isKD4Correct} />
      </Grid>
      <Grid item xs={6}>
        <Paper sx={{ p: 3 }}>
          {current.isCorrect ? <
gutterBottom color='red'>
            {'Разница содержания фракций'}
          </Typography>
          </Grid>
          <Grid item xs=
{6}>
            <Paper
sx={{ p: 3 }}>
              {current.isCorrect ? <
gutterBottom color='red'>
                {'Прочностные характеристики не могут быть рассчитаны'}
              </Typography>
              </Grid>
              <Grid>
                <Grid item xs={6}>
                  <Typography variant='h5">
                    {'Разница содержания фракций'}
                  </Typography>
                  <Grid>
                    <Grid item xs=
{6}>
                      <Typography
variant='subtitle2'>0-10: {(current.granFrom0To10! * 100).toFixed(1)} %</Typography>
                    </Grid>
                    <Grid item xs=
{6}>
                      <Typography
variant='subtitle2'>10-25: {(current.granFrom10To25! *
100).toFixed(1)} %</Typography>
                    </Grid>
                    <Grid item xs=
{6}>
                      <Typography
variant='subtitle2'>25-40: {(current.granFrom25To40! *
100).toFixed(1)} %</Typography>
                    </Grid>
                    <Grid item xs=
{6}>
                      <Typography
variant='subtitle2'>40-60: {(current.granFrom40To60! *
100).toFixed(1)} %</Typography>
                    </Grid>
                    <Grid item xs=
{6}>
                      <Typography
variant='subtitle2'>60-80: {(current.granFrom60To80! *
100).toFixed(1)} %</Typography>
                    </Grid>
                    <Grid item xs=
{6}>
                      <Typography
variant='subtitle2'>80-999: {(current.granFrom80To999! *
100).toFixed(1)} %</Typography>
                    </Grid>
                  </Grid>
                </Grid>
              </Paper>
            </Grid>
          </Grid>
        </Paper>
      </Grid>
    </Grid>
  )
}
)

```

Рис. П2.12. Текст модуля GranPage

Спецификации

Настоящее приложение содержит спецификацию на разработанную программную документацию и компоненты программного обеспечения. Спецификация на разработанную программную документацию приведена в табл. ПЗ.1.

Таблица ПЗ.1

Спецификации на разработанную программную документацию

Обозначение	Наименование	Примечание
1	2	3
Документация		
Богданов_ВКР.docx	Расчетно-пояснительная записка выпускной квалификационной работы «Прогнозирование прочностных характеристик кокса по его гранулометрическому составу»	
Приложение 2	Текст программы	
Приложение 4	Руководство пользователя	
Компоненты		
App.tsx	Файл конфигурации запуска клиентского приложения	
ReportPage.tsx	Файл страницы отчета клиентского приложения	
GranPage.tsx	Файл страницы актуального состояния клиентского приложения	
LoginPage.tsx	Файл страницы входа в аккаунт клиентского приложения	
ReportsPage.tsx	Файл страницы выбора отчета клиентского приложения	
SettingsPage.tsx	Файл страницы настроек клиентского приложения	
train.py	Файл обучения нейронной сети определения гран состава	

Продолжение табл. ПЗ.1

1	2	3
trainer.py	Файл, содержащий класс для обучения нейронной сети определения гран состава	
rockPublisher.py	Файл, содержащий класс для работы нейронной сети определения гран состава в реальном времени с использованием камер	
train_with_optuna.py	Файл, содержащий алгоритм обучения модели прогнозирования прочностных характеристик	
join_data.py	Файл, содержащий алгоритм объединения данных по гран составу и лабораторных измерений прочностных характеристик	
filter_and_convert_data.py	Файл, содержащий алгоритм фильтрации данных по гран составу	
Program.cs	Файл конфигурирования запуска серверного приложения	
UserService.cs	Файл содержащий бизнес логику работы с пользователем на серверной части	
GranulService.cs	Файл содержащий бизнес логику работы с гран составом и прочностными характеристиками на серверной части	
EncryptionService.cs	Файл содержащий бизнес логику работы с шифрованием данных на серверной части	
UserController.cs	Файл, содержащий определение доступных HTTP запросов работы с пользователем на серверной части	
GranulController.cs	Файл, содержащий определение доступных HTTP запросов работы с гран составом и прочностными характеристиками на серверной части	
AuthInputOptions.cs	Файл, содержащий настройки авторизации серверной части	
GranulDataDto.cs	Файл, содержащий класс для передачи данны по измерению гран состава и прочностных характеристик	

Продолжение табл. ПЗ.1

1	2	3
UserInfoDto.cs	Файл, содержащий класс для передачи данны по пользователю	

Руководство пользователя

1. Описание программного обеспечения

Программный продукт предназначен для получения визуального результата работы системы прогнозирования прочностных характеристик кокса по его гранулометрическому составу. Программное обеспечение предназначается для операторов доменной печи №4.

2. Минимальные системные требования

Для корректной работы системы необходимы персональные компьютеры, удовлетворяющие следующим минимальным требованиям:

- объем оперативной памяти 16 Гб;
- монитор с разрешением 1920x1080;
- операционная система Windows10 Домашняя;
- твердотельный накопитель объемом 512 Гб;
- процессор AMD Ryzen 5 2600 (4 ядер, 8 потоков, тактовая 3.4 ГГц);
- видеокарта Gigabyte GeForce GTX 1660Ti с поддержкой DirectX 12;
- наличие установленного Python 3.12;
- наличие установленного Node.js.

3. Запуск программы

Для запуска программы необходимо запустить файл сервера «Gran.Server.exe» (рис. П4.1).

	Gran.Server.dll	04.06.2024 9:53	Расширение при...	59 КБ
	Gran.Server.exe	04.06.2024 9:53	Приложение	140 КБ
	Gran.Server.pdb	04.06.2024 9:53	Program Debug D...	38 КБ

Рис. П4.1. Файл запуска сервера

Также необходимо запустить файл детектора «rockPublisher.py» с помощью командной строки (рис. П4.2).

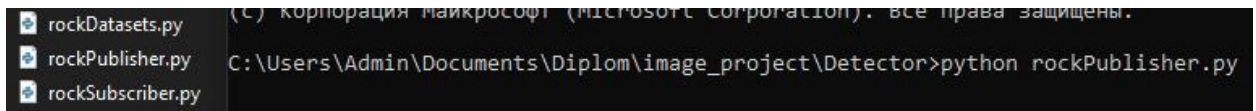


Рис. П4.2. Запуск детектора

Также необходимо запустить файл клиентского приложения (рис. П4.3).

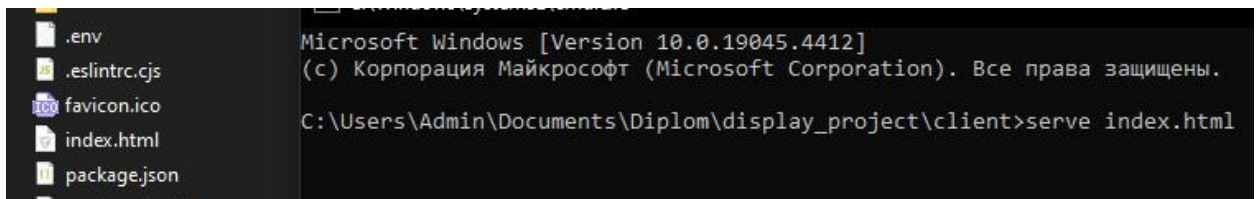


Рис. П4.3. Запуск клиентского приложения

После произведенных действий пользователям в сети предоставляется доступ к системе. Первой страницей при открытии является страница входа в аккаунт (рис. П4.4).

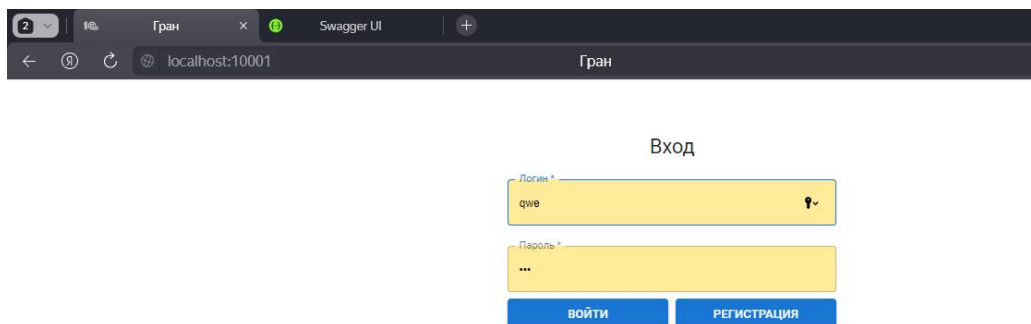


Рис. П4.4. Страница входа

После успешного входа пользователь попадает на страницу актуального состояния, на которой расположены результаты работы системы в реальном времени (рис. П4.5). На данной странице отображены текущие кадры с камер конвейеров КД4 и К19, содержания фракций и результаты работы модели прогнозирования прочностных характеристик.

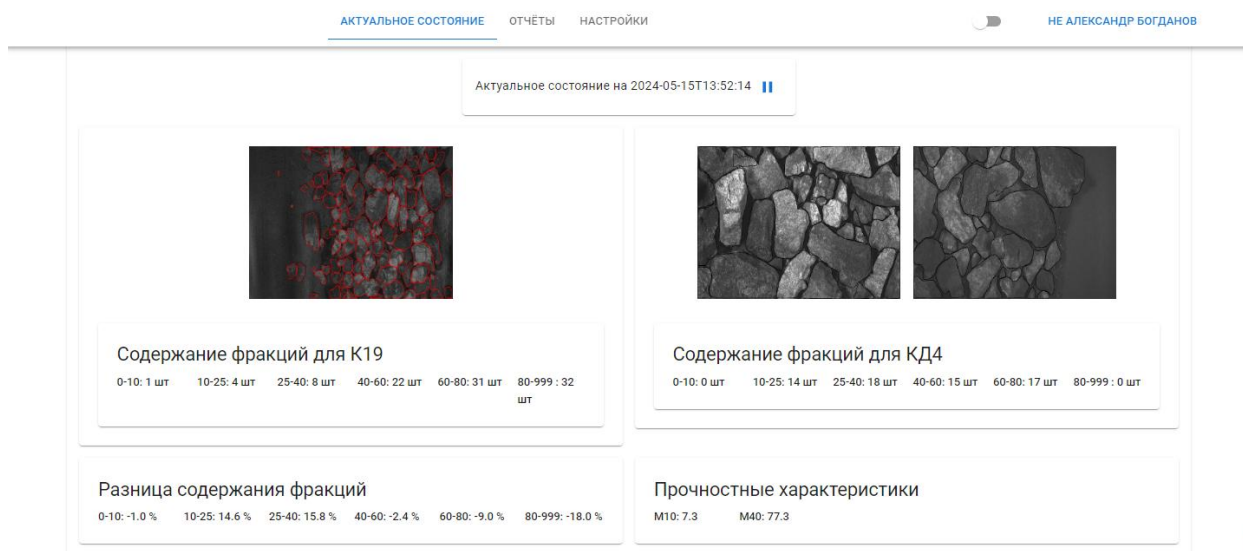


Рис. П4.5. Страница «Актуальное состояние»

В правой верхней части экрана присутствует возможность выхода из учетной записи (рис. П4. 6), а также кнопка изменения темы приложения (рис. П4.7).

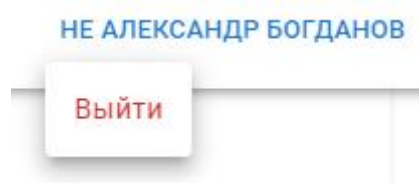


Рис. П4.6. Кнопка выхода из учетной записи

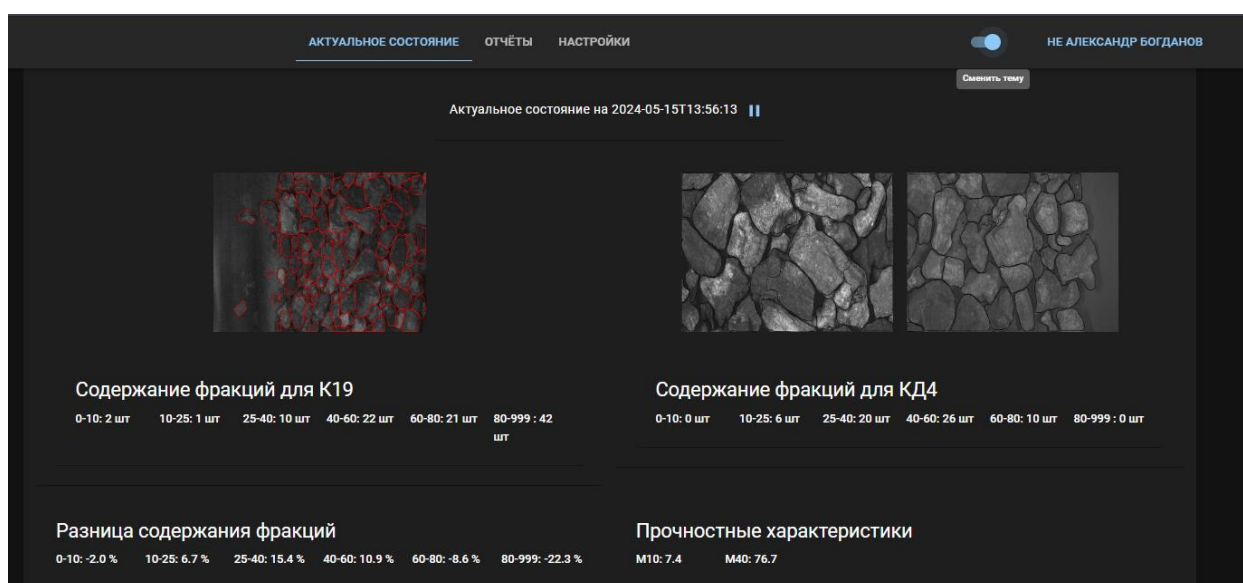


Рис. П4.7. Смена темы

При переходе на страницу «Отчеты» пользователь перейдет в режим просмотра исторической информации (рис. П4.8). На данной странице расположен список произведенных измерений, в котором можно перейти к конкретному отчёту, нажав на его дату.

АКТУАЛЬНОЕ СОСТОЯНИЕ ОТЧЁТЫ НАСТРОЙКИ							НЕ АЛЕКСАНДР БОГДАНОВ
Дата	0-10	10-25	25-40	40-60	60-80	80-999	
 2024-05-15T13:50:31	0.0 %	9.3 %	16.2 %	9.1 %	3.3 %	-37.8 %	
 2024-05-15T13:51:05	0.0 %	4.4 %	31.1 %	5.9 %	2.5 %	-44.0 %	
 2024-05-15T13:52:14	-1.0 %	14.6 %	15.8 %	-2.4 %	-9.0 %	-18.0 %	
 2024-05-15T13:52:48	-1.0 %	7.5 %	2.9 %	8.7 %	5.4 %	-23.6 %	
 2024-05-15T13:53:22	0.0 %	3.9 %	21.2 %	11.5 %	-0.2 %	-36.4 %	
 2024-05-15T13:53:56	0.2 %	7.0 %	13.2 %	8.9 %	0.4 %	-29.7 %	
 2024-05-15T13:54:30	0.0 %	2.0 %	11.0 %	21.3 %	0.0 %	-34.3 %	
 2024-05-15T13:56:13	-2.0 %	6.7 %	15.4 %	10.9 %	-8.6 %	-22.3 %	
 2024-05-15T13:56:47	-1.0 %	12.5 %	15.2 %	7.2 %	-3.2 %	-30.7 %	

Рис. П4.8. Страница «Отчеты»

При переходе к конкретному отчету (рис. П4.9), откроется страница аналогичная главной, то есть она будет содержать информацию о произведенном определении гранулометрического состава и прогнозирования прочностных характеристик.

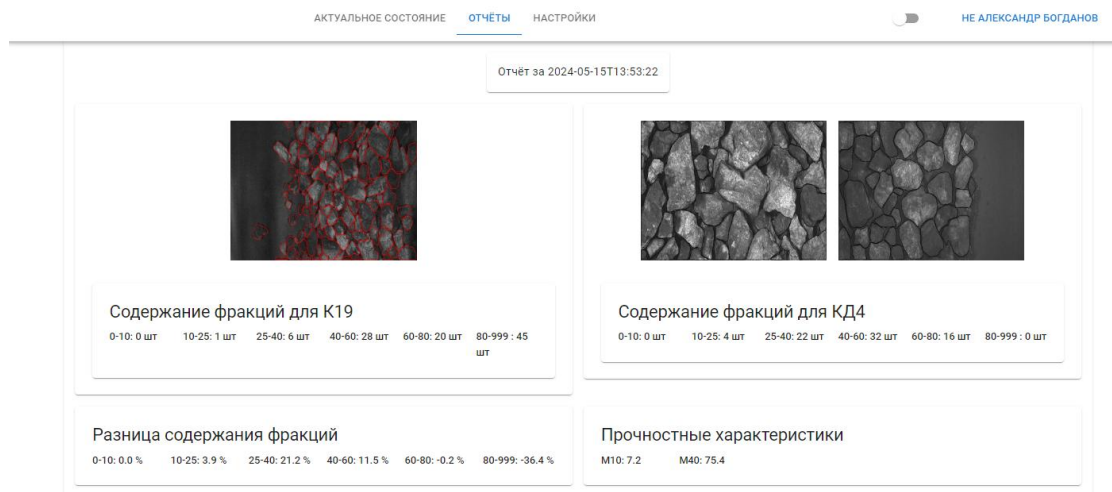


Рис. П4.9. Открытый отчет

При открытии вкладки настройки, пользователю предоставляется возможность редактирования настроек приложения (рис. П4.10). Для данной системы предоставляется возможность, настройки путей для камер.

АКТУАЛЬНОЕ СОСТОЯНИЕ ОТЧЁТЫ НАСТРОЙКИ

НЕ АЛЕКСАНДР БОГДАНОВ

Камера для КТ9 *

127.0.0.0:5030

Первая камера для КД4 *

127.0.0.0:5020

Вторая камера для КД4 *

127.0.0.0:5010

СОХРАНИТЬ ОТМЕНИТЬ

Рис. П4.10. Страница «Настройки»